

Arquitectura de Software

Arquitectura, arquitecto y *architecting* · atributos de calidad · patrones de diseño

Síntesis integrada de tres fuentes:

1. C. B. Reynoso — *Introducción a la Arquitectura de Software* (UBA, 2004)
2. L. Bass, P. Clements, R. Kazman — *Software Architecture in Practice*, 4.^a ed.
3. E. Gamma, R. Helm, R. Johnson, J. Vlissides — *Design Patterns* (GoF, 1994-95)
4. *Refactoring.Guru* — Design Patterns (A. Shvets) · ISO/IEC/IEEE 42010

Contenido

Parte I — El objeto de estudio	3	29. Virtualización: VMs, contenedores, pods, serverless	73
1. ¿Qué es una arquitectura de software?	4	30. La nube y la computación distribuida	75
2. Arquitectura frente a diseño: la demarcación	7	31. Sistemas móviles	78
3. El arquitecto de software	10	Parte VI — La práctica del arquitecto	80
4. Architecting: la arquitectura como verbo	12	32. Requerimientos arquitectónicamente significativos	81
Parte II — Génesis y fundamentos de la disciplina	15	33. Diseñar la arquitectura: el método ADD	83
5. Breve historia de la Arquitectura de Software	16	34. Evaluar la arquitectura: ATAM y evaluaciones livianas	85
6. Definiciones de la disciplina	19	35. Documentar la arquitectura	88
7. Conceptos fundamentales: estilos, ADLs, vistas, escenarios	21	36. Gestionar la deuda arquitectónica	90
8. Campos de la Arquitectura de Software	25	37. El rol del arquitecto en los proyectos y en Agile	92
9. Las seis corrientes o escuelas	26	38. Competencia arquitectónica	93
10. Arquitectura frente a diseño (las tesis de la literatura)		39. Un vistazo al futuro: computación cuántica	94
11. Problemas abiertos y relevancia de la disciplina	2928	Parte VII — Patrones de diseño (GoF)	95
Parte III — La arquitectura como conjunto de estructuras	32	40. Qué es un patrón de diseño	96
12. Qué es (y qué no es) la arquitectura de software	33	41. El catálogo: clasificación y los 23 patrones	98
13. Estructuras arquitectónicas y vistas	35	42. Cómo los patrones resuelven los problemas de diseño	
14. Qué hace "buena" a una arquitectura	37	43. Caso de estudio: el editor Lexi	103100
15. Las trece razones por las que la arquitectura importa	39	44. Comparaciones y discusión de los patrones	104
16. Atributos de calidad, escenarios, tácticas y patrones	41	45. Los patrones hoy: la lectura de Refactoring.Guru	107
Parte IV — Los diez atributos de calidad	44	Parte VIII — Síntesis integradora	110
17. Disponibilidad	45	46. Correlaciones entre las tres fuentes	111
18. Deployabilidad	48	47. Glosario esencial	114
19. Eficiencia energética	51	Parte IX — Catálogo de estilos arquitectónicos	116
20. Integrabilidad	52	48. Flujo de datos: tubería y filtro, y batch secuencial	118
21. Modificabilidad	54	49. Centrados en datos: repositorio y pizarra	119
22. Performance	57	50. Jerárquicos: capas y microkernel	120
23. Safety (inocuidad)	60	51. Invocación implícita: publish-subscribe y broker	122
24. Seguridad	62	52. Cliente-servidor y peer-to-peer	123
25. Testabilidad	64	53. Intérprete y máquina virtual	124
26. Usabilidad	66	54. Comparación, composición y el caso KWIC	125
27. Trabajar con otros atributos de calidad	68	Parte X — Las 23 fichas del catálogo GoF	127
Parte V — Interfaces e infraestructura moderna	69	55. Cómo leer una ficha	128
28. Interfaces de software	70	56. Los 5 patrones creacionales	129
		57. Los 7 patrones estructurales	132
		58. Los 11 patrones de comportamiento	137

Cómo leer esta guía. La **Parte I** fija el vocabulario (arquitectura, diseño, arquitecto, architecting) y puede leerse como resumen ejecutivo de todo el documento. Las **Partes II** (historia y teoría), **III–VI** (la ingeniería practicable) y **VII** (el catálogo de patrones) desarrollan cada fuente en profundidad. La **Parte VIII** muestra cómo las tres fuentes se corresponden entre sí, y las **Partes IX y X** son los dos catálogos de referencia: los **estilos arquitectónicos** y las **23 fichas de patrones**, para consultar patrón por patrón. Las cajas **violeta** marcan ideas clave; las **ámbar**, definiciones exactas; las **verdes**, ejemplos desarrollados; las **rojas**, advertencias y trampas típicas de examen; las **grises**, citas textuales.

PARTE I

El objeto de estudio

*Los cuatro conceptos que ordenan todo lo demás: qué es una **arquitectura**, en qué se diferencia del **diseño**, quién es el **arquitecto** y qué actividad es **architecting**. Con las definiciones canónicas de IEEE 1471 / ISO 42010, Perry & Wolf, Shaw & Garlan, Bass-Clements-Kazman, Booch, Parnas, Brooks, Ralph Johnson y Fowler.*

-
- 1 ¿Qué es una arquitectura de software?
.....
 - 2 Arquitectura frente a diseño: la demarcación
.....
 - 3 El arquitecto de software
.....
 - 4 Architecting: la arquitectura como verbo
.....

1. ¿Qué es una arquitectura de software?

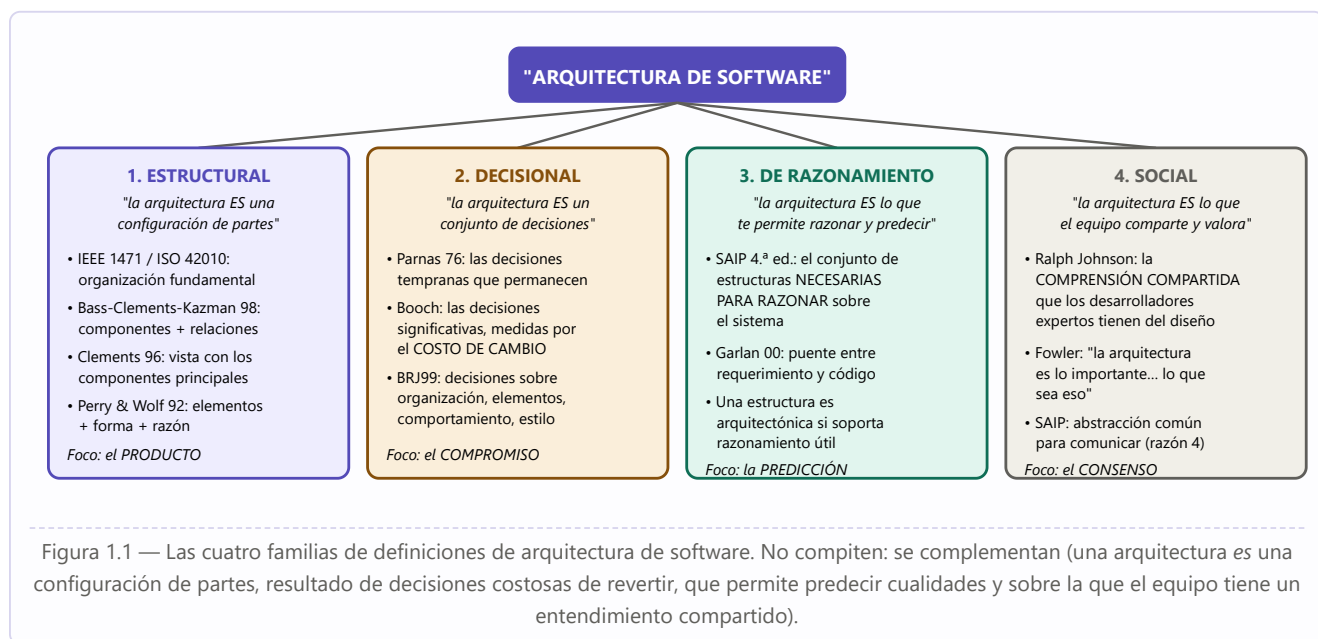
Antes de estudiar tácticas, patrones o métodos conviene detenerse en la palabra misma. "Arquitectura" es, en software, un término **prestado**: llegó por analogía con la arquitectura de edificios y arrastra consigo tanto una intuición muy productiva como un conjunto de malentendidos. Reynoso lo documenta con precisión: fue **Perry y Wolf en 1992** quienes propusieron concebir la disciplina "por analogía con la arquitectura de edificios"; **Brooks** ya en 1975 decía que el arquitecto de software "es un agente del usuario, igual que lo es quien diseña su casa"; el movimiento de patrones nació de **Christopher Alexander**, arquitecto *de edificios*. Pero de esa misma analogía algunos abusaron (WWISA), y **Jason Baragry** llegó a cuestionarla como columna vertebral de la disciplina, atribuyéndole la falta de acuerdo sobre la cantidad y calidad de las vistas arquitectónicas y la ausencia de un mecanismo formal de coordinación entre ellas.

Cuidado — el término más usado en exceso de la ingeniería de software

Clements y Northrop lo advirtieron: "el significado de la palabra *arquitectura* se ha ido diluyendo simplemente **porque parece estar de moda**". Es posible encontrar referencias a arquitectura específica de dominio, de megaprogramación, de destinatario, de sistemas, de redes, de infraestructura, de aplicaciones, de operaciones, técnica, de framework, conceptual, de referencia, empresarial, de factoría, C4I, de manufactura, de edificio, de máquina-herramienta... "A menudo la palabra se usa de maneras inapropiadas". Y el número de definiciones circulantes de *arquitectura de software* "alcanza un orden de tres dígitos, amenazando llegar a cuatro".

1.1 Las cuatro familias de definiciones

Las definiciones no son arbitrarias: se agrupan en cuatro familias según **dónde ponen el acento**. Reconocerlas evita discusiones estériles, porque casi siempre dos personas que "no se ponen de acuerdo sobre qué es la arquitectura" están simplemente parados en familias distintas.



1.2 Las definiciones canónicas, textuales

Fuente	Definición	Familia
IEEE Std 1471-2000 (la "oficial"; adoptada por Microsoft y por el C4ISR del DoD; hoy continuada por ISO/IEC/IEEE 42010)	"La Arquitectura de Software es la organización fundamental de un sistema , encarnada en sus componentes , las relaciones entre ellos y con su entorno , y los principios que orientan su diseño y evolución."	Estructural
Bass, Clements & Kazman SAIP 4.ª ed.	"La arquitectura de software de un sistema es el conjunto de estructuras necesarias para razonar sobre el sistema . Estas estructuras comprenden elementos de software, relaciones entre ellos y propiedades de ambos ."	De razonamiento
Perry & Wolf (1992) el paper fundacional	Un modelo de tres componentes: elementos (de procesamiento, de datos o de conexión), forma (las propiedades de y las relaciones entre los elementos, es decir, restricciones operadas sobre ellos) y razón o <i>rationale</i> (la base subyacente, derivada de las restricciones del sistema, que a su vez suelen derivar de sus requerimientos).	Estructural + decisional
Clements (1996)	"Una vista del sistema que incluye los componentes principales, la conducta de esos componentes según se la percibe desde el resto del sistema, y las formas en que los componentes interactúan y se coordinan para alcanzar la misión del sistema. La vista arquitectónica es una vista abstracta : aporta el más alto nivel de comprensión y la supresión o diferimiento del detalle."	Estructural
Shaw & Garlan (1994)	Las cuestiones estructurales incluyen: organización a grandes rasgos y estructura global de control; protocolos de comunicación, sincronización y acceso a datos; asignación de funcionalidad a elementos de diseño; distribución física; composición de elementos; escalabilidad y performance; y selección entre alternativas de diseño .	Estructural
Garlan (2000)	La AS constituye un punto entre el requerimiento y el código , ocupando el lugar que en los gráficos antiguos se reservaba para el diseño.	De razonamiento
Grady Booch	"Toda arquitectura es diseño, pero no todo diseño es arquitectura. La arquitectura representa las decisiones de diseño significativas que dan forma a un sistema, donde significativo se mide por el costo del cambio ."	Decisional
Parnas (1976, vía Reynoso)	Las decisiones tempranas de desarrollo — las que probablemente permanecerán invariantes en el desarrollo ulterior — "constituyen de hecho lo que hoy se llamarían decisiones arquitectónicas" (las que están cerca del tope del árbol de decisión de una familia de programas).	Decisional
Ralph Johnson citado por Fowler en <i>Who Needs an Architect?</i>	"La arquitectura es la comprensión compartida del diseño del sistema que tienen los desarrolladores expertos ." No hay manera objetiva de definir qué es "fundamental" o "de alto nivel"; lo que hay es ese entendimiento común. Y su corolario célebre: "la arquitectura es lo importante... lo que sea eso" .	Social

1.3 Arquitectura vs. descripción arquitectónica: la distinción de ISO 42010

Idea clave — la cosa y su representación no son lo mismo

Un tema central del estándar **ISO/IEC/IEEE 42010** (heredero de IEEE 1471) es la distinción entre la **arquitectura** — los conceptos o propiedades fundamentales de una entidad — y la **descripción arquitectónica** (AD), que es **el producto de trabajo que se usa para expresarla**: "una colección de artefactos que documentan una arquitectura". El SAIP dice lo mismo con otras palabras: **todo sistema tiene una arquitectura** (porque todo sistema tiene elementos y relaciones) pero eso no implica que alguien la conozca — quizás los diseñadores ya no están, la documentación se perdió o nunca existió, y sólo queda el binario ejecutándose. "Esto revela la diferencia entre la arquitectura de un sistema y la *representación* de esa arquitectura." De allí la importancia de documentar (y de la "arqueología arquitectónica" de Clements para recuperar la arquitectura de un sistema legacy sin documentación confiable).

Confundir ambas cosas produce dos errores frecuentes y opuestos: creer que **no hay arquitectura porque no hay documento** (falso: hay una, sólo que nadie la conoce — y probablemente sea mala), y creer que **hay arquitectura porque hay un diagrama** (falso: el diagrama puede describir algo que el código no cumple — es justamente lo que buscan las herramientas de conformidad "as-built vs. as-designed" que menciona el SAIP para las evaluaciones tardías).

1.4 Las seis propiedades de toda arquitectura

Propiedad	Consecuencia práctica
1. Es un conjunto de estructuras , no una sola	"Ninguna estructura puede reclamar en exclusiva ser <i>la</i> arquitectura". Definir la arquitectura como "la estructura global del sistema" agrega confusión , porque implica que el sistema posee una sola estructura (crítica de Clements & Northrop). Una estructura es arquitectónica si soporta el razonamiento sobre alguna propiedad importante para algún stakeholder.
2. Es una abstracción	Omite intencionalmente lo que no sirve para razonar: se ocupa del lado público de las interfaces; los detalles privados de implementación no son arquitectónicos . Esta abstracción es lo que doma la complejidad: "el punto de la arquitectura es que no haga falta tener cada detalle en la cabeza".
3. Incluye comportamiento	La conducta de cada elemento es arquitectónica en la medida en que ayude a razonar sobre el sistema y afecte su aceptabilidad (de ahí que se documente con secuencias, actividades y máquinas de estados).
4. Todo sistema tiene una	Incluso el que nadie diseñó: "la arquitectura emergerá — pero será la equivocada" (SAIP sobre el enfoque puramente emergente).
5. Ninguna es buena o mala en sí	Son " más o menos aptas para algún propósito ": una SOA de tres capas puede ser ideal para un B2B empresarial y completamente errónea para aviónica; una arquitectura afinada para modificabilidad no tiene sentido para un prototipo descartable. Evaluar sólo tiene sentido contra metas explícitas .
6. Vive dentro de otros sistemas	La arquitectura de sistema (hardware + software + humanos) y la arquitectura empresarial (procesos, flujos de información, personal, unidades organizacionales) le imponen restricciones y forman su entorno.

2. Arquitectura frente a diseño: la demarcación

Esta es la pregunta que más confusión genera en la práctica — y la que Reynoso identifica como el problema de fondo en la brecha academia-industria: "la cuestión radica más bien en el desconocimiento que la industria tiene de la arquitectura académica y **en su obstinación por llamar arquitectura a lo que sólo es diseño**". Conviene entonces tener criterios de demarcación operativos, y no uno solo.

La formulación que resume todo

"La arquitectura es diseño, pero no todo diseño es arquitectura" (SAIP; y en la versión de Booch: "toda arquitectura es diseño, pero no todo diseño es arquitectura"). Muchas decisiones de diseño quedan deliberadamente **sin ligar** por la arquitectura — es, después de todo, una abstracción — y dependen del criterio de los diseñadores e implementadores posteriores. Corolario útil: **la arquitectura es el conjunto de restricciones dentro de las cuales el diseño posterior es libre**.

2.1 Los cinco criterios de demarcación

#	Criterio	Es arquitectónico si...	Autor
1	Nivel de abstracción	...está por encima del nivel de los algoritmos y las estructuras de datos: "este diseño ocurre a un nivel más abstracto". Es el criterio más citado ("todo el mundo insiste en ello").	BCK98; Taylor & Medvidovic (postura 2, "la más cercana a la verdad")
2	Costo de reversión	...cambiar la decisión más tarde sería caro o difícil . "Significativo se mide por el costo del cambio". Test complementario del SAIP: si la decisión puede posponerse hasta el momento de tratar las cosas a bajo nivel, no es una decisión de arquitectura .	Booch; Bass, Clements & Kazman (vía Reynoso)
3	Alcance del impacto	...su cambio es no local o sistémico : afecta "la forma fundamental en que los elementos interactúan entre sí y probablemente requerirá cambios por todo el sistema" (el cambio <i>arquitectónico</i> de la taxonomía local / no local / arquitectónico).	SAIP (razón 2)
4	Tipo de entidades tratadas	...se ocupa de componentes y no de procedimientos; de las interacciones entre esos componentes y no de las interfaces; de las restricciones a ejercer sobre componentes e interacciones y no de los algoritmos, procedimientos y tipos. Composición de grano grueso (vs. fina composición procedural); protocolos de alto nivel (vs. rpc, mensajes, llamadas a rutinas).	Dewayne Perry (1997)
5	Audiencia y propósito	...sirve para razonar y negociar con stakeholders sobre atributos de calidad y riesgos, no para construir una pieza. La arquitectura es la abstracción común "que la mayoría de los participantes puede usar para crear entendimiento mutuo, formar consenso y comunicarse".	Clements & Northrop; Garlan; Ralph Johnson

◀ ARQUITECTURA

— la frontera es un continuo, y su ubicación depende del contexto del sistema —

DISEÑO (y código) ▶

¿distribuido o
monoprocesador?
¿en capas?

¿sync o async?
¿se cifra el tráfico?
¿qué protocolo?

¿microservicios o
monolito? ¿pub-sub
o cliente-servidor?

¿qué framework?
¿qué formato de
intercambio?

¿qué patrón GoF
para esta jerarquía
de clases?

¿qué algoritmo
de ordenamiento?
¿qué nombres?

Costo de cambio: ALTÍSIMO

Alcance: todo el sistema

Entidades: componentes, conectores, restricciones

Interesa a: stakeholders, gerencia, equipos

Se decide: temprano y con pocos datos (bajo riesgo asumido)

Costo de cambio: BAJO

Alcance: un módulo o una clase

Entidades: procedimientos, interfaces, tipos, algoritmos

Interesa a: el desarrollador del elemento

Se decide: tarde, con información completa

Figura 2.1 — El continuo arquitectura–diseño. Ninguna línea vertical es "la" frontera: cada proyecto la ubica según qué le resulte costoso de cambiar y sobre qué necesite razonar con sus stakeholders.

2.2 Las tres posturas de la literatura

Taylor y Medvidovic señalan que la literatura mantiene la relación en estado ambiguo, con tres posturas:

1. Arquitectura y diseño **son lo mismo**.
2. La arquitectura está en un **nivel de abstracción por encima** del diseño, o es simplemente otro paso (un artefacto) del proceso de desarrollo. ← **la que ellos consideran más cercana a la verdad**.
3. La arquitectura es **algo nuevo y en alguna medida diferente** del diseño — pero de qué manera y en qué medida se deja sin especificar.

Y agregan la observación dinámica más interesante: **a medida que la arquitectura de alto nivel se refina, sus conectores pierden prominencia**, distribuyéndose entre los elementos arquitectónicos de más bajo nivel, "resultando en la transformación de la arquitectura en diseño". La frontera, entonces, no sólo es contextual: **se mueve con el tiempo**.

2.3 La secuencia de tres pasos de Shaw & Garlan

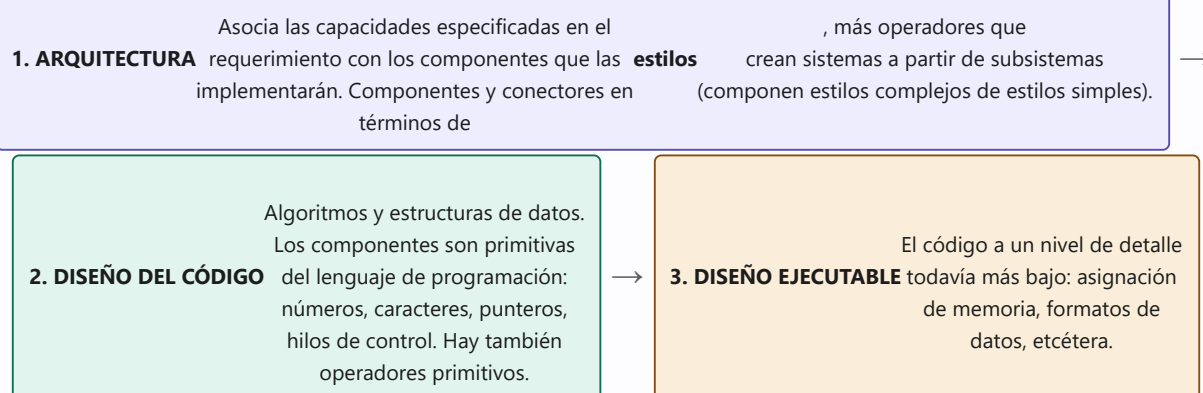


Figura 2.2 — Los tres pasos de Shaw & Garlan (1996) en la producción de un diseño de software. Nótese que **el territorio del catálogo GoF es el paso 2**, exactamente el que Perry excluye de lo arquitectónico.

2.4 Las tesis complementarias

- **Albin:** la AS **abarca** las metodologías de diseño y también las de análisis. El arquitecto contemporáneo ejecuta una combinación de roles (analista de sistemas, diseñador, ingeniero), pero "la arquitectura es más que una recolocación de funciones": esas funciones pueden seguir ejecutándose por otros, sólo que ahora caen bajo la **orquestración del *chief architect***. Y la clave: "la arquitectura es algo **más integrado** que la suma del análisis por un lado y el diseño por el otro; **la integración de metodologías y modelos** es lo que distingue a la AS de la simple yuxtaposición de técnicas".
- **Clements:** el diseño basado en arquitectura es un **paradigma de desarrollo** que difiere de las alternativas conocidas "de maneras fundamentales... en muchos sentidos, tanto como el diseño orientado a objetos difiere de sus predecesores".
- **ABD:** el diseño arquitectónico es el de más elevado nivel de abstracción, pero debe enfrentar **un requerimiento todavía difuso** y el hecho de que, en ese plano, las decisiones serán **las más críticas y las más difíciles de modificar**.
- **Reynoso,** cerrando el capítulo: el diseño arquitectónico se distingue del diseño del código en que "**viene determinado desde mucho antes y es mucho más crítico para el éxito o el fracaso de una solución**".

Típica pregunta de examen — Clasificá estas decisiones

Arquitectónicas: ¿corre en un procesador o distribuido? ¿el software será en capas y cuántas? ¿los componentes se comunican sincrónica o asincrónicamente, transfiriendo control, datos o ambos? ¿se cifra la información que fluye? ¿qué sistema operativo? ¿qué protocolo de comunicación? ¿replicamos el servicio y balanceamos? ¿el servicio es stateless? — todas del listado de "decisiones tempranas" del SAIP, y todas con "efecto de onda" que puede volverse avalancha.

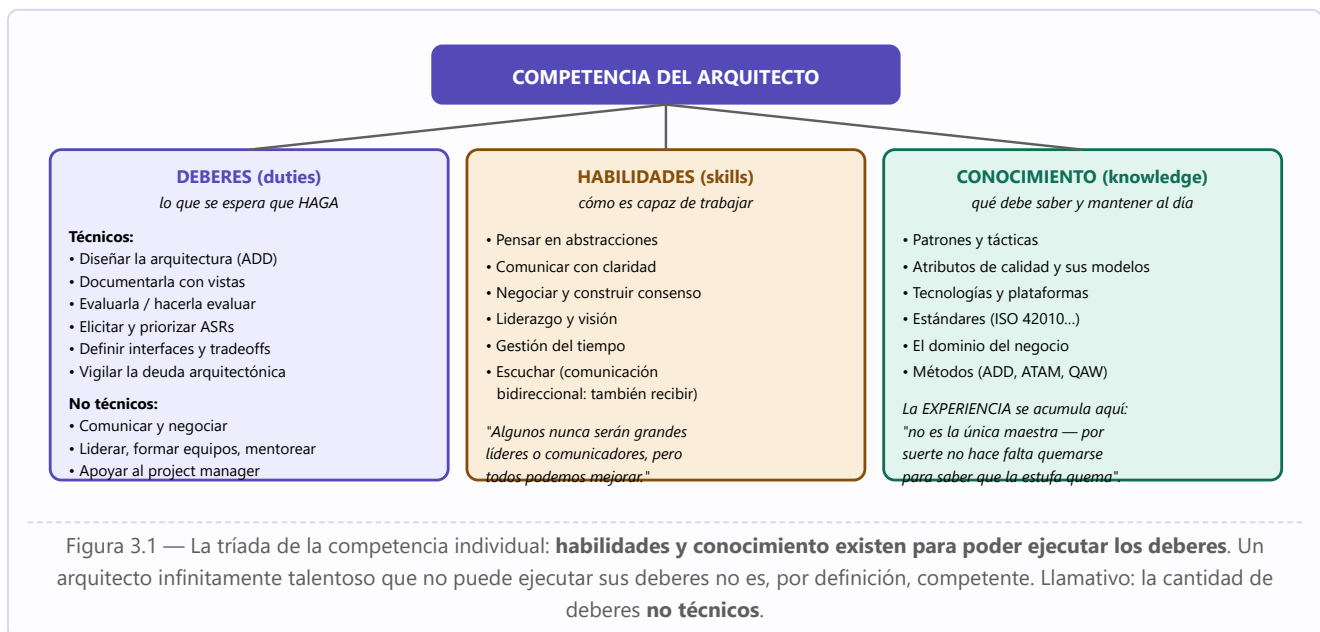
No arquitectónicas (aunque importantes): qué algoritmo de ordenamiento usa un módulo interno; cómo se nombran las variables; el layout exacto de una pantalla; qué estructura de datos privada usa una clase encapsulada. Regla mnemotécnica: **si el cambio queda dentro de un módulo y su interfaz no se mueve, no es arquitectura**.

3. El arquitecto de software

Si la arquitectura es el producto, el arquitecto es el rol responsable de producirla, comunicarla y defenderla. Tres precisiones antes que nada:

1. **Es un rol, no necesariamente un cargo.** El prefacio del SAIP lo dice con elegancia: la mayoría de los atributos de calidad son susceptibles de problemas de "eslabón más débil", y esos eslabones sólo emergen al componer sistemas; sin una mano que guíe, la composición fracasará — **"y esa mano que guía pertenece a un arquitecto, sea cual sea su título"**.
2. **Es un agente del usuario.** Brooks, en 1975, sostenía que el arquitecto "es un agente del usuario, igual que lo es quien diseña su casa"; Reynoso señala que lo mismo podría decirse de Christopher Alexander, que buscaba una arquitectura elaborada "de forma que tenga en cuenta las necesidades de los habitantes".
3. **Debe ser uno, o un grupo pequeño con líder identificado.** Primera recomendación de proceso del SAIP: la arquitectura debe ser producto de **un solo arquitecto o un grupo pequeño con un líder técnico identificado**, para darle **integridad conceptual** y consistencia técnica — y esto vale igual para proyectos ágiles y open source. Con un contrapeso explícito: debe haber **conexión fuerte con el equipo de desarrollo**, para evitar los diseños impracticables de "torre de marfil".

3.1 La tríada de la competencia: deberes, habilidades y conocimiento



3.2 Lo que el arquitecto hace en el día a día

Frente	Actividad concreta
Con los stakeholders	Elicitar y priorizar los ASRs (entrevistas, QAW, PALM); traducir metas de negocio en escenarios de calidad; explicar tradeoffs en términos de costo ("¿24/7? te muestro cuánto cuesta y decidís"); ofrecer lo que ellos no sabían que podían pedir ("puedo entregarte algo mejor de lo que tenías en mente, ¿te sirve?") — el arquitecto es la única persona en la conversación que puede decir eso.

Con el project manager	División de trabajo complementaria: el PM mira hacia afuera (presupuesto, calendario, staffing, reporte a la dirección), el arquitecto hacia adentro (la solución técnica), y cada vista debe reflejar fielmente la otra. El arquitecto aporta a casi todas las áreas del PMBOK: estimaciones bottom-up, estructura de equipos (work assignment), análisis de riesgos, plan de incrementos.
Con los desarrolladores	Restringir sin asfixiar: entregar interfaces, responsabilidades y presupuestos (de tiempo, de tráfico) sin exigir que cada implementador conozca los tradeoffs globales; mentorear; revisar; y — en áreas críticas — arremangarse a implementar y testear (el SAIP lo reconoce explícitamente, aunque aclare que no son deberes arquitectónicos <i>stricto sensu</i>).
Con la arquitectura misma	Diseñar en rondas e iteraciones (ADD); documentar incrementalmente (primero descomposición + usos + una C&C + despliegue a brocha gorda); evaluar (y aceptar ser evaluado: regla cardinal del ATAM — el arquitecto debe participar de buena gana); vigilar la erosión y la deuda; sostener la integridad conceptual ("lo mismo se hace de la misma manera en todas partes").
Con la organización	Influir para que los primeros incrementos ataquen los QAs más desafiantes (que no haya sorpresas arquitectónicas tardías); participar en la definición del producto; aconsejar sobre la estructura de los equipos; y aprovechar lo que Gregor Hohpe llama el " ascensor del arquitecto ": la capacidad singular de interactuar con gente de todos los niveles , dentro y fuera de la organización, y facilitar la comunicación entre ellos.

3.3 Cuatro mitos

- **"El arquitecto decide todo."** Falso por definición: la arquitectura es una abstracción y deja deliberadamente decisiones abiertas. Su trabajo es fijar **restricciones** y los pocos patrones de interacción que el sistema usará de manera uniforme, no dictar cada línea.
- **"El arquitecto no programa."** Ni el SAIP ni la práctica lo sostienen: hay que evitar la "torre de marfil", y los arquitectos suelen colaborar en implementación y testing de las áreas críticas. Lo que **no** puede es perder la visión de conjunto.
- **"En Agile no hace falta arquitecto."** El SAIP responde: para sistemas medianos y grandes, la visión puramente emergente "colapsó bajo el peso de la experiencia" — seguridad, performance y safety **no se atornillan después**. El punto dulce es "Iteración 0" + spikes; y SAFe reconoce explícitamente la *intentional architecture*.
- **"Arquitecto es el que dibuja el diagrama."** Confunde la arquitectura con su descripción (§1.3). El diagrama sin decisiones, rationale y evaluación no es arquitectura: es dibujo.

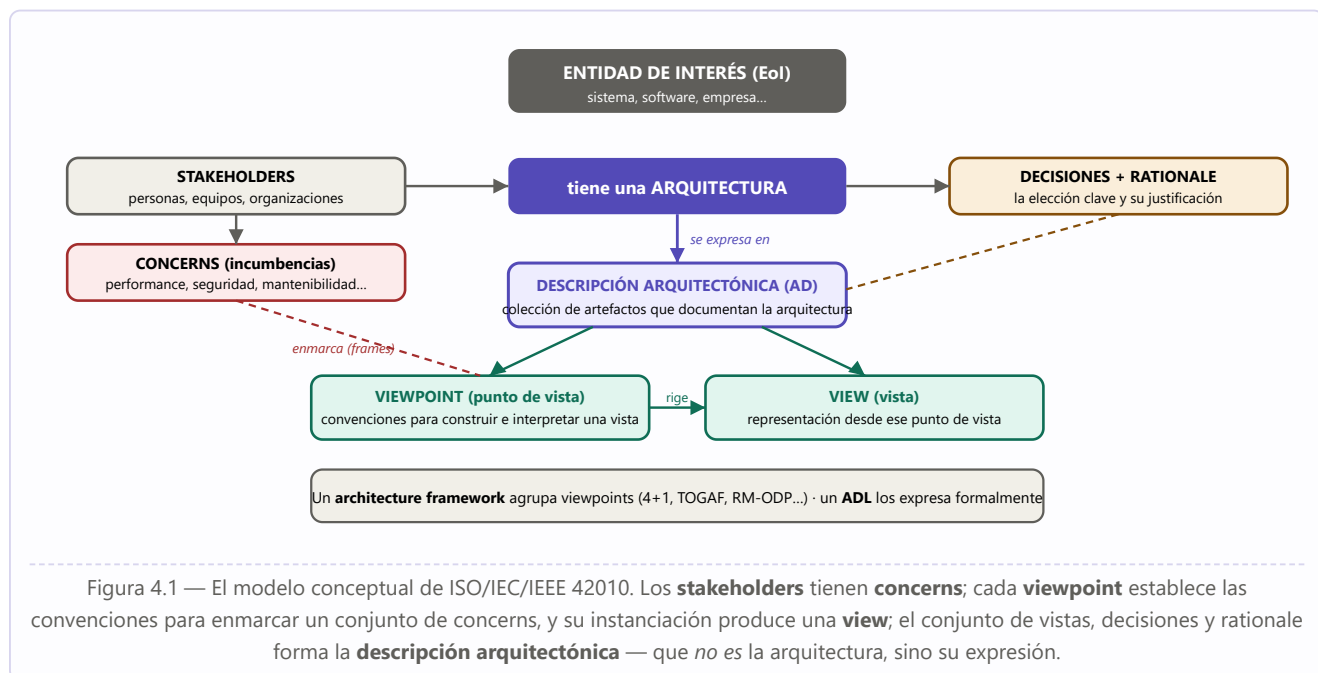
Cómo se hace un arquitecto

Tres vías, en este orden de eficacia: (1) **experiencia ejecutando los deberes** — la vía productiva es el *apprenticeship*, porque "la educación sin aplicación en el trabajo sólo aumenta el conocimiento"; (2) **mejorar las habilidades no técnicas** (liderazgo, comunicación, gestión del tiempo); (3) **dominar y mantener al día el cuerpo de conocimiento** (hace unos años la nube no era tema). Y el consejo final del SAIP: **ser mentoreado y mentorear** — porque enseñar un concepto es el test de fuego de haberlo entendido de verdad. "—¿Cómo se llega al Carnegie Hall? —*Practice, practice, practice.*"

4. Architecting: la arquitectura como verbo

En español solemos usar un solo sustantivo — "arquitectura" — para dos cosas distintas: el **producto** (las estructuras del sistema) y la **actividad** de producirlo. La literatura en inglés separa ambos con *architecture* y *architecting*, y ISO/IEC/IEEE 42010 declara explícitamente que mantiene su alcance al día "respecto de la evolución de las **actividades y prácticas de architecting** en las organizaciones". Reynoso captura la misma dualidad al describir cómo las definiciones circulantes mezclan "(1) **el trabajo dinámico de estipulación** de la arquitectura dentro del proceso de ingeniería o el diseño — su lugar en el ciclo de vida — y (2) la configuración o topología **estática** del sistema".

4.1 El vocabulario de ISO/IEC/IEEE 42010

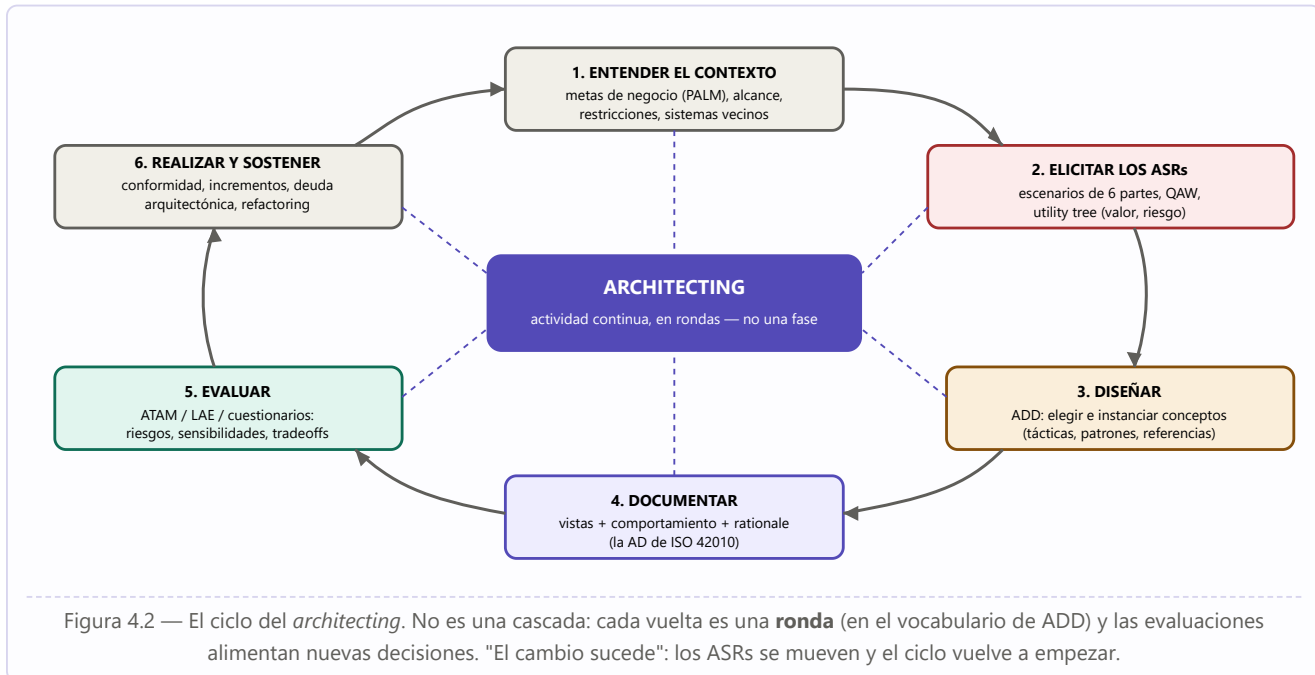


Definiciones del estándar, para fijar el vocabulario: **descripción arquitectónica (AD)**, "una colección de artefactos que documentan una arquitectura"; **entidad de interés**, "el sistema, software, empresa u otra entidad cuya arquitectura se describe"; **stakeholders**, "individuos, equipos u organizaciones con un interés en la entidad de interés"; **concerns**, "los intereses que los stakeholders tienen en la entidad de interés, tales como performance, seguridad o mantenibilidad"; **viewpoint**, "una especificación para construir una vista: define los stakeholders, los concerns y las técnicas de modelado a usar" — o, en la formulación completa, "un producto de trabajo que establece las convenciones para la construcción, interpretación y uso de vistas arquitectónicas **para enmarcar concerns específicos del sistema**"; **view**, "una representación de la arquitectura desde la perspectiva de un viewpoint particular"; **decisión arquitectónica**, "una elección clave hecha durante el diseño de la arquitectura"; **rationale**, "la justificación de las decisiones arquitectónicas, ligándolas a los concerns de los stakeholders y a otros requerimientos".

La analogía que lo fija todo

IEEE 1471 (y con él Reynoso) da la mejor regla mnemotécnica: **"un viewpoint es a una vista lo que una clase es a un objeto"**. El viewpoint es la plantilla reutilizable ("vista de despliegue": qué elementos, qué relaciones, qué notación, para qué concerns); la vista es su instancia concreta para *este* sistema.

4.2 El ciclo del architecting



4.3 Cinco rasgos del architecting como actividad

1. **Es toma de decisiones bajo incertidumbre.** "A menudo las decisiones arquitectónicas deben tomarse con conocimiento imperfecto": de allí los **prototipos descartables**, la técnica **VoI** para decidir cuánto conviene gastar en experimentar, y el principio de que **el riesgo es la guía** para saber cuándo parar de diseñar.
2. **Es cualitativa, a diferencia de la ingeniería.** El contraste que subraya Reynoso comparando IEEE 1471 con IEEE 610.12: "la noción clave de la **arquitectura es la organización** — un concepto cualitativo o estructural —, mientras que la **ingeniería** tiene fundamentalmente que ver con una **sistematicidad susceptible de cuantificarse**".
3. **Es continua e incremental.** Se libera en incrementos, al ritmo de test y release del proyecto; ADD la organiza en rondas e iteraciones con backlog y tablero Kanban ("Not Yet / Partially / Completely Addressed"). No termina en el primer release: **~80% del costo del sistema ocurre después**.
4. **Es repetible y enseñable — o no es ingeniería.** "Repetibilidad y enseñabilidad son las marcas de una disciplina de ingeniería": un método sistemático permite que la actividad "pueda ser aprendida y ejecutada competentemente por simples mortales", y no sólo por gurús con décadas de experiencia.
5. **Produce artefactos, no sólo dibujos.** Cada vuelta del ciclo deja: escenarios priorizados, decisiones **con su rationale**, vistas, riesgos identificados con sus temas, y un backlog. "Registra las decisiones en el momento en que las tomás: si lo dejás para después, no vas a recordar por qué hiciste las cosas."

Ejemplo desarrollado: el architecting deja huella incluso cuando "falla"

En las evaluaciones ATAM que relata Clements hubo casos en que el equipo llegó y **no había arquitectura que evaluar** (sólo pilas de diagramas de clases y descripciones vagas). Aun así el ejercicio produjo valor: el conjunto de atributos de calidad quedó **articulado por primera vez**, se dibujó una arquitectura "de pizarra" durante la sesión, y el proyecto salió con **obligaciones de documentación** concretas. En otro caso, un equipo de diseñadores junior — a los que el arquitecto consideraba incapaces de responder preguntas — reconstruyó entre todos, en media jornada, una vista C&C, una de procesos y una del subsistema offline: "ninguno sabía todo, pero cada uno sabía algo", y ese grupo salió transformado en un verdadero equipo de arquitectura. Moraleja: **el architecting no es sólo el documento que produce, sino el entendimiento compartido que construye** — exactamente la definición de Ralph Johnson.

PARTE II

Génesis y fundamentos de la disciplina

Basada en C. B. Reynoso, Introducción a la Arquitectura de Software (Universidad de Buenos Aires, 2004): historia, definiciones, conceptos fundamentales, corrientes y problemas abiertos.

-
- 5 Breve historia de la Arquitectura de Software
 - 6 Definiciones de la disciplina
 - 7 Conceptos fundamentales: estilos, ADLs, vistas, escenarios
 - 8 Campos de la Arquitectura de Software
 - 9 Las seis corrientes o escuelas
 - 10 Arquitectura frente a diseño
 - 11 Problemas abiertos y relevancia de la disciplina

5. Breve historia de la Arquitectura de Software

La Arquitectura de Software (en adelante, **AS**) acostumbra remontar sus antecedentes a la década de 1960, pero su historia no ha sido tan continua como la de la ingeniería de software, el campo más amplio en el que se inscribe. Tras las tempranas inspiraciones de Edsger Dijkstra, David Parnas y Fred Brooks, la AS quedó en *estado de vida latente* durante años, hasta su expansión explosiva a partir del manifiesto de Dewayne Perry y Alexander Wolf en 1992. Comprender esa trayectoria permite distinguir cuáles contribuciones son perdurables y cuáles fueron contingentes a las modas tecnológicas de cada época.

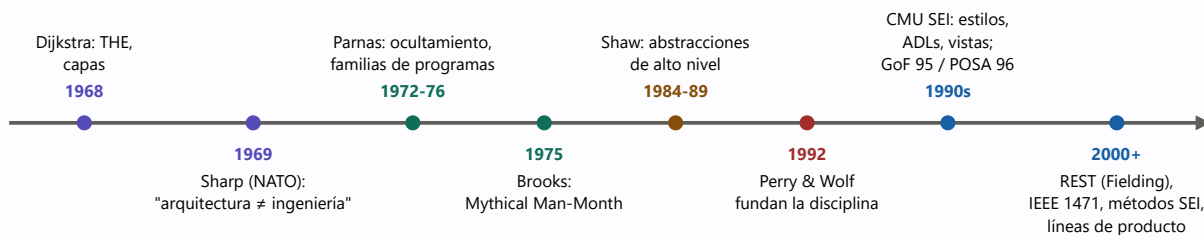


Figura 5.1 — Línea de tiempo de la Arquitectura de Software según la reseña histórica de Reynoso.

5.1 Los precursores (1968–1976)

Hacia **1968**, **Edsger Dijkstra** (Universidad Tecnológica de Eindhoven, Premio Turing 1972) propuso que se establezca una *estructuración correcta* de los sistemas de software antes de lanzarse a programar. Sostenía que las ciencias de la computación eran una rama aplicada de las matemáticas y sugería seguir pasos formales para descomponer problemas mayores. Fue uno de los introductores de la noción de **sistemas operativos organizados en capas** que se comunican sólo con las capas adyacentes, superpuestas "como capas de cebolla" (su implementación: el *THE Multiprogramming System*). Inventó o ayudó a precisar docenas de conceptos: el algoritmo del camino más corto, los *stacks*, los semáforos, los abrazos mortales. De sus ensayos arranca la tradición de los "**niveles de abstracción**". Sus ideas sentaron las bases de lo que luego expresarían Niklaus Wirth (1971) como *stepwise refinement* (refinamiento por pasos) y DeRemer y Kron (1976) como *programming-in-the-large* (programación en grande).

Cita histórica — P. I. Sharp, conferencia de la NATO (Roma, 1969), comentando a Dijkstra

"Pienso que tenemos algo, aparte de la ingeniería de software... Es la cuestión de la **arquitectura de software**. La arquitectura es diferente de la ingeniería. [...] La razón de que OS/360 sea un amontonamiento amorfo de programas es que **no tuvo arquitecto**. Su diseño fue delegado a series de grupos de ingenieros, cada uno de los cuales inventó su propia arquitectura." Y añadió: "Las especificaciones de software se consideran especificaciones funcionales. Sólo hablamos sobre lo que queremos que haga el programa. [...] Quien sea responsable de una pieza de software debe especificar más: debe especificar **el diseño, la forma**; y dentro de ese marco de referencia, los programadores e ingenieros deben crear algo."

Nadie volvió a hablar del asunto en esa conferencia. Durante años, "arquitectura" fue una metáfora usada sin precisión semántica: en 1969 Brooks e Iverson la llamaban "estructura conceptual de un sistema en la perspectiva del programador"; en 1971 C. R. Spooner tituló un ensayo *"Una arquitectura de software para los 70s"*, antecedente que la historiografía casi no registra.

David Parnas demostró en la misma época que los criterios elegidos para descomponer un sistema impactan en la estructura de los programas, y propuso principios de diseño para obtener una estructura adecuada. Sus tres aportes capitales:

- **Ocultamiento de información** (*information hiding*, 1972): la modularidad mejora la flexibilidad y el control conceptual si cada módulo se caracteriza por su conocimiento de una **decisión de diseño oculta** para los demás. "Su interfaz o definición se escoge para que revele tan poco como sea posible sobre su forma interna de trabajo". Cada módulo deviene una **caja negra** accesible a través de interfaces bien definidas y estables. La clave: basar la modularización en *decisiones de diseño* y no en etapas de procesamiento (los "niveles de abstracción" de Dijkstra eran, en cambio, técnicas de programación).
- **Estructuras de software** (1974).
- **Familias de programas** (1976): conjunto de programas (no todos construidos) que conviene considerar como grupo; se enumeran mediante el **árbol de decisiones** que se atraviesa para llegar a cada miembro. Las decisiones iniciales (cerca de la raíz) son las que probablemente permanecerán constantes entre los miembros: esas "decisiones tempranas" constituyen lo que hoy se llaman **decisiones arquitectónicas**.

Idea clave — "Structure matters"

Como escriben Clements y Northrop, en todo el desenvolvimiento ulterior de la disciplina permanece en primer plano la idea de Parnas: **la estructura es primordial**, y la elección de la estructura correcta es crítica para el éxito del desarrollo de una solución. "La elección de la estructura correcta" sintetiza, como ninguna otra expresión, el programa y la razón de ser de la AS. Aunque Dijkstra es la figura legendaria de los mitos de origen, **Parnas introdujo las nociones más esenciales y permanentes**.

Fred Brooks (diseñador de OS/360, Premio Turing 2000) utilizaba en 1975 el concepto de arquitectura para designar "la especificación completa y detallada de la interfaz de usuario", y consideraba que el arquitecto es un **agente del usuario**. Distinguía entre arquitectura (*qué hacer*) e implementación (*cómo*). Su *The Mythical Man-Month* sigue siendo el texto más leído en ingeniería de software. Se ha señalado que Dijkstra (formalismo matemático) y Brooks (variables humanas) son personalidades opuestas situadas en los orígenes de las metodologías fuertes y de las heterodoxias ágiles, respectivamente.

5.2 La fundación de la disciplina (1992)

Mientras la ingeniería de software se fundó en 1968 (F. L. Bauer, conferencia de la OTAN en Garmisch), la AS como disciplina delimitada es mucho más nueva. Mary Shaw reivindicó las abstracciones de alto nivel en 1984 y 1989 (*"Larger scale systems require higher level abstractions"*), pero el primer estudio donde aparece "arquitectura de software" en el sentido actual es el de **Dewayne Perry (AT&T Bell Labs) y Alexander Wolf (U. de Colorado), 1992** — gestado desde 1989. Proponen concebir la AS por analogía con la arquitectura de edificios (analogía de la que luego algunos abusaron y que para otros devino inaceptable), y presentan un modelo de tres componentes:

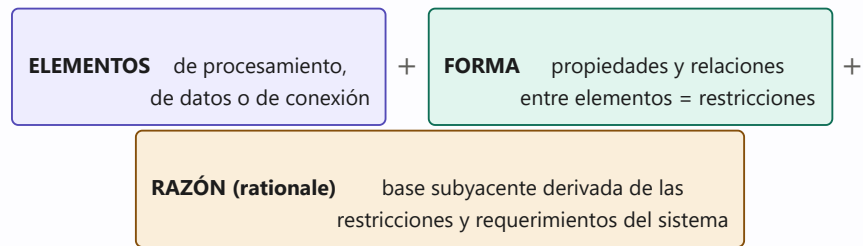


Figura 5.2 — El modelo fundacional de Perry & Wolf (1992): elementos, forma y razón.

Perry & Wolf, 1992 — la profecía

*"La década de 1990, creemos, será la década de la arquitectura de software. Usamos el término 'arquitectura' **en contraste con 'diseño'**, para evocar nociones de codificación, de abstracción, de estándares, de entrenamiento formal (de los arquitectos de software) y de estilo."*

Su llamamiento fue respondido por la **escuela estructuralista de Carnegie Mellon**: David Garlan, Mary Shaw, Paul Clements, Robert Allen. Se trata de una práctica joven que hacia 2004 tenía apenas unos doce años de trabajo constante, con una nueva ola en el SEI (Rick Kazman, Mark Klein, Len Bass), tendencias en el sur de California (Medvidovic, Rosenblum, Taylor), en el SRI (Moriconi) y en la IEEE (Hilliard), además de posturas europeas volcadas a los escenarios y al ciclo de vida.

5.3 La consolidación (años 90) y el siglo XXI

Dando cumplimiento a la profecía, los 90 fueron la década de la consolidación, con epicentro en el **CMU SEI** antes que en la industria. Hitos:

- **Programación basada en componentes**: llevó a Clements a afirmar que la AS promovía un modelo *de integración de componentes pre-programados más que de programación*.
- **Los patrones**: cristalizados en el texto de la **Banda de los Cuatro** (GoF, 1995 — expansión de la programación OO) y la serie **POSA** (1996 — marco más ligado a la AS). El originador de la idea fue **Christopher Alexander**, arquitecto *de edificios*, que en los 70 buscó un modelo constructivo y humano que tuviera en cuenta las necesidades de los habitantes; las ideas llegaron a la informática diez años más tarde. Tanto en los patrones como en la arquitectura, la idea dominante es la **reutilización**.
- Homogeneización de terminología, tipificación de **estilos**, elaboración de **ADLs** y consolidación de las **vistas** (reconocidas en todos los frameworks: 4+1, TOGAF, RM/ODP, IEEE).
- **2000**: la tesis de **Roy Fielding** presenta **REST**, que instala las tecnologías de Internet y los modelos orientados a servicios y recursos en el centro de la disciplina; se publica la recomendación **IEEE Std 1471**, que homogeneiza la nomenclatura de descripción arquitectónica y homologa los estilos.
- **Siglo XXI**: dominio de las estrategias orientadas a **líneas de productos** y redefinición de todas las metodologías del ciclo de vida en términos arquitectónicos (producción masiva de métodos, otra vez desde el SEI). La aparición de las metodologías basadas en arquitectura, junto con los métodos ágiles y XP, reordenó el campo metodológico: "después de la AS y de las tácticas radicales, las metodologías nunca volverán a ser las mismas".

6. Definiciones de la disciplina

Ninguna definición de la AS es respaldada unánimemente: circulan **cientos** (la colección del SEI es la más conocida, y cada quien puede agregar la suya). En general las definiciones entremezclan tres cosas: (1) el trabajo dinámico de estipulación de la arquitectura dentro del proceso (su lugar en el ciclo de vida), (2) la configuración o topología estática del sistema vista desde un elevado nivel de abstracción, y (3) la caracterización de la disciplina que se ocupa de esos asuntos.

Definición: Clements [Cle96a]

La AS es, a grandes rasgos, **una vista del sistema** que incluye los componentes principales del mismo, la conducta de esos componentes según se percibe desde el resto del sistema, y las formas en que los componentes interactúan y se coordinan para alcanzar la misión del sistema. La vista arquitectónica es una **vista abstracta**: aporta el más alto nivel de comprensión y la supresión o diferimiento del detalle. ("Componente" aquí no es la tecnología COM/EJB, sino el elemento propio de un estilo.)

Definición "oficial": IEEE Std 1471-2000 (adoptada también por Microsoft)

"La Arquitectura de Software es la **organización fundamental de un sistema** encarnada en sus **componentes**, las **relaciones** entre ellos y el ambiente, y los **principios que orientan su diseño y evolución**."

Es productivo contrastarla con la definición de *ingeniería* de software (IEEE 610.12-1990): "la aplicación de una estrategia sistemática, disciplinada y **cuantificable** al desarrollo, aplicación y mantenimiento del software". Obsérvese: la noción clave de la **arquitectura es la organización** (concepto cualitativo o estructural), mientras que la **ingeniería** tiene que ver con una sistematicidad **cuantificable**.

Otras definiciones influyentes: existe acuerdo en que la AS se refiere a la **estructura a grandes rasgos del sistema, consistente en componentes y relaciones entre ellos** [BCK98]; ese diseño ocurre a un nivel más abstracto que los algoritmos y las estructuras de datos. Para Shaw y Garlan [GS94] las cuestiones estructurales incluyen: organización global y estructura de control; protocolos de comunicación, sincronización y acceso a datos; asignación de funcionalidad a elementos; distribución física; composición; escalabilidad y performance; y selección entre alternativas de diseño. Para Garlan [Gar00], la AS es **un puente entre el requerimiento y el código**.

6.1 Clasificación de modelos (Shaw & Garlan)

Ante la variedad de definiciones, Shaw y Garlan explicaron las diferencias en función de distintas clases de modelos:

Modelo	Qué sostiene	Trabajo característico
1. Estructurales	La AS se compone de componentes, conexiones entre ellos y otros aspectos: configuración, estilo, restricciones, semántica, análisis, propiedades, racionalizaciones, requerimientos.	El desarrollo de ADLs (lenguajes de descripción arquitectónica).
2. De framework	Similar al estructural, pero con énfasis en la estructura coherente del sistema completo (usualmente una sola), orientado a dominios o clases de problemas específicos.	Arquitecturas específicas de dominio; CORBA; repositorios como PRISM.

3. Dinámicos	Enfatizan la cualidad conductual: cambios en la configuración del sistema o dinámica del cómputo (valores cambiantes).	Sistemas dinámicos y reconfigurables.
4. De proceso	La arquitectura es el resultado de seguir un guión (<i>script</i>) de proceso; foco en los pasos de su construcción.	Programación de procesos para derivar arquitecturas.
5. Funcionales	(Minoritario) La arquitectura como conjunto de componentes funcionales organizados en capas que proporcionan servicios hacia arriba.	Un framework particular.

Ninguna de estas vistas excluye a las otras: representan un **espectro de énfasis** (partes constituyentes, totalidad, conducta, proceso de construcción) que, tomado en conjunto, destaca un consenso. Común a todos los autores: la arquitectura como punto de vista que concierne a un **alto nivel de abstracción**.

7. Conceptos fundamentales

Más allá de los numerosos conceptos del plano técnico detallado, la AS se articula alrededor de unos pocos conceptos y principios esenciales.

7.1 Estilos

Definición: Estilo arquitectónico

Un estilo es un **concepto descriptivo que define una forma de articulación u organización arquitectónica**. El conjunto de los estilos cataloga las formas básicas posibles de estructuras de software; las formas complejas se articulan mediante **composición** de los estilos fundamentales. Los estilos conjugan **componentes, conectores, configuraciones y restricciones**.

Al estipular los **conectores como elemento de juicio de primera clase**, el concepto de estilo se sitúa en un orden de discurso que el modelado OO en general y UML en particular **no cubren satisfactoriamente**. A diferencia de los patrones de diseño (que son centenares), los estilos se ordenan en **seis o siete clases fundamentales y unos veinte ejemplares como máximo** — es notable el empeño por subsumir todas las aplicaciones existentes en tan pocas dimensiones. Estilos típicos: arquitecturas basadas en **flujo de datos**, **peer-to-peer**, de **invocación implícita**, **jerárquicas**, **centradas en datos**, de **intérprete / máquina virtual**. La mejor forma de describir un estilo es un ADL o un lenguaje formal de especificación. El estilo es, además, la **encarnación principal del principio de reusabilidad** en el plano arquitectónico, y su identificación se hace "a grandes rasgos": en un estilo, **menos es más**.

Los estilos, uno por uno → Parte IX

El documento de Reynoso **enumera** los estilos pero no los describe: su inventario y descripción son el capítulo 2 de su serie, publicado aparte y no incluido en la carpeta de la materia. Por eso este catálogo se desarrolla en la **Parte IX** de esta guía (caps. 48–54), con una ficha por estilo — componentes, conectores, restricciones, cuándo usarlo, qué atributos favorece y cuáles sacrifica — más la tabla comparativa y el caso KWIC.

7.2 Lenguajes de descripción arquitectónica (ADLs)

Los ADLs ocupan una parte importante del trabajo arquitectónico desde la fundación de la disciplina: propuestas de variado rigor, casi todas académicas, surgidas desde comienzos de los 90 en contemporaneidad con la unificación de UML. Permiten **modelar una arquitectura mucho antes de programar** las aplicaciones, analizar su adecuación, determinar puntos críticos y hasta simular su comportamiento. Difieren sustancialmente de UML, que en su versión 1.x se estima **inadecuado para expresar conectores** y limitado en su modelo semántico para las clases de descripción y análisis requeridas. Hay unos **veinte ADLs de primera magnitud** (Acme/Armani, ADML, Aesop, ArTek, C2 SADL, CHAM, Darwin, Jacal, LILEANNA, MetaH/AADL, Rapide, UniCon, Wright, xArch/xADL...) y quizás cuarenta o cincuenta más que no resistieron el paso del tiempo. Son bien conocidos en la universidad, pero **muy pocos arquitectos de industria los conocen y menos aún los usan**. A futuro, ningún ADL será viable si no se coordina con **UML 2.0** por un lado y con **XML** por el otro.

7.3 Frameworks y vistas

Definición: Vista

Una vista es **un subconjunto resultante de practicar una selección o abstracción sobre una realidad, desde un punto de vista determinado** [Hilliard]. La motivación para introducir múltiples vistas es la **separación de incumbencias** (*separation of concerns*). La idea no es invención de Kruchten: se origina al menos en los 70, en el trabajo de Douglas Ross sobre análisis estructurado (1977).

Los organismos de estándares (ISO, CEN, IEEE, OMG) han codificado aspectos de la AS (RM-ODP, RUP, MDA, MOF, XMI, IEEE 1471...). Los marcos y sus vistas principales:

Marco	Vistas / niveles	Observaciones
Zachman (1987)	36 "celdas": 6 niveles (scope, empresa, sistema lógico, tecnología, representación, funcionamiento) × 6 aspectos (datos, función, red, gente, tiempo, motivación)	Estimado excesivamente rígido y sobre-articulado; consenso sobre su posible obsolescencia; pertenece más al <i>Information Management</i> . Tres de sus vistas (conceptual, lógica, física) anticipan los marcos posteriores.
RM-ODP (ISO/ITU)	5 puntos de vista: empresa, información, computación, ingeniería, tecnología	Para grandes sistemas distribuidos; los viewpoints no corresponden a etapas del proceso. CORBA no es 100% conforme. Hoy la interoperabilidad se garantiza más por XML/SOAP/BPEL4WS/WS-I que por conformidad con estas recomendaciones.
C4ISR (DoD)	Operacional, de Sistemas, Técnica	Su definición de arquitectura (v2, 1997) es exactamente la que luego canonizó IEEE 1471. Inspiró a TOGAF.
TOGAF (Open Group)	Negocios, Datos/Información, Aplicación, Tecnológica	Propone el método de descripción ADM, independiente de las técnicas de modelado.
"4+1" (Kruchten, 1995; RUP)	Lógica, de Proceso, Física, de Desarrollo + Casos de uso	Lo académicamente arquitectónico concierne a las dos primeras. Hoy se percibe como reformulación de una arquitectura estructural en términos de objetos y UML; aun así, sus vistas son repertorio estándar.
UML/Tres Amigos [BRJ99]	Casos de uso, Diseño, Procesos, Implementación, Despliegue	La AS como "conjunto de decisiones significativas" sobre organización, elementos e interfaces, comportamiento, composición y estilo. Yuxtaposición informal más que integración rigurosa.
POSA [BMR+96]	Arquitecturas: conceptual / de módulos / de código / de ejecución. Vistas: lógica, proceso, física, desarrollo	La segunda lista coincide con 4+1 sin el énfasis en el quinto elemento.
Bass, Clements & Kazman (1998)	9 vistas: módulo, lógica/conceptual, procesos, física, uso, llamados, flujo de datos, flujo de control, clases	Sesgada hacia el diseño concreto y la implementación; no coincide con ninguna otra taxonomía.

IEEE Std 1471-2000	arquitectura / vista / punto de vista	Un punto de vista (viewpoint) define un patrón o plantilla para representar un conjunto de incumbencias (concerns); una vista es la representación concreta de un sistema desde una perspectiva unitaria. Viewpoint es a vista como clase es a objeto . No delimita el número de vistas.
Microsoft [Platt02]	Arquitecturas: Negocios, Aplicación, Información, Tecnología; vistas Conceptual, Lógica, Física	En consonancia con las conceptualizaciones más generalizadas (herencia de OMT).

Cuidado / típica pregunta de examen

Las "múltiples vistas" son el Santo Grial de la ingeniería, pero su estatus en la AS es oscuro: **no existe fundamentación coherente** para su uso, y muchos las consideran problemáticas porque introducen **problemas de integración y consistencia entre vistas**. Los arquitectos las usan de todos modos porque **simplifican la visualización de sistemas complejos**. Recordar además que la AS clásica (Garlan y Shaw) **no habla de vistas en absoluto**: se funda en una vista singular e implícita, de carácter estructural.

7.4 Procesos y metodologías

En los marcos, las vistas estáticas se corresponden con las perspectivas de los **stakeholders**, y las dinámicas con las etapas del proceso (requerimiento, análisis, diseño, implementación, integración/prueba). Durante años la AS no elaboró metodologías propias, asumiéndose compatible con las prácticas establecidas (RUP, RAD, CMM...). Desde **1998** el SEI y otros organismos elaboran métodos que sistematizan el rol de la arquitectura en todo el proceso: **ABD** (Architecture Based Design), **SAAM**, **QAW** (Quality Attribute Workshops), **QASAR**, **ADD** (Attribute-Driven Design), **ATAM** (Architecture Tradeoff Analysis Method), **ARID**, **CBAM** (Cost-Benefits Analysis Method), **FAAM**, **ALMA**, **SACAM**. La comunidad metodológica, por su parte, sigue dividida por el "**gran debate metodológico**": métodos pesados/rigurosos tipo SEI/CMM (De Marco, Yourdon, Lister) contra métodos ágiles (XP, SCRUM, Crystal, FDD, DSDM, Lean, Adaptive, Agile Modeling — Orr, Highsmith, Fowler, Jackson), con Kruchten y RUP intentando establecerse en ambos terrenos. Ambos bandos operan en el contexto de la "crisis del software", acusándose mutuamente de haberla ocasionado.

7.5 Abstracción

El concepto de abstracción (a veces el proceso de abstraer, a veces la entidad abstracta) tiene un núcleo común: para la IEEE, Rumbaugh o Shaw, consiste en **extraer las propiedades esenciales**, identificar los aspectos importantes o examinar selectivamente ciertos aspectos de un problema, **posponiendo o ignorando los detalles** menos sustanciales o irrelevantes. Booch, en cambio, la identifica con el encapsulamiento de la tecnología de objetos — las diferencias de uso ayudan a identificar las corrientes dentro de la AS.

Idea clave — El test de la decisión arquitectónica

Para Bass, Clements y Kazman: **si una decisión puede posponerse hasta el momento de tratar las cosas a bajo nivel, no es una decisión de arquitectura**. Y Clements & Northrop, sobre el trabajo de Garlan y Shaw: aunque los programas pueden combinarse de maneras casi infinitas, **hay mucho que ganar restringiéndose a un conjunto relativamente pequeño de elecciones** — mejor reutilización, mejores análisis, menor tiempo de selección, mayor interoperabilidad. "Menos es más".

7.6 Escenarios

Noción en la base de muchos métodos (ALMA, SAAM, ATAM). Precaución terminológica: lo que se traduce como "escenario" debería ser más bien **"guión" o "libreto"** (en el sentido teatral o cinematográfico). Desde Kruchten se reconocen dos categorías: **casos de uso** (secuencias de responsabilidades) y **casos de cambio** (modificaciones propuestas al sistema). Son técnicas de **elicitación de requerimientos** (típicamente con sesiones de *brainstorming*), también usadas para comparar alternativas de diseño; describen una utilización anticipada o deseada del sistema y típicamente se expresan **en una frase** (Kazman, Abowd, Bass y Clements proponen llamarlas "viñetas"). Sirven para analizar una vista determinada o para mostrar cómo los elementos de múltiples vistas se relacionan entre sí. Los expertos mundiales en casos de uso (Anderson, Fowler, Cockburn) recomiendan desarrollarlos **en texto, no en diagramas**.

8. Campos de la Arquitectura de Software

La AS es hoy un conjunto inmenso y heterogéneo de áreas de investigación teórica y formulación práctica. Comenzó siendo una abstracción descriptiva puntual, sin investigar sistemáticamente ni su relación con los requerimientos previos ni los pasos metodológicos posteriores; con el tiempo tiende a **redefinir todos los aspectos de la ingeniería de software**, a mayor nivel de abstracción y agregando una dimensión reflexiva sobre la fundamentación formal del proceso.

Garlan y Perry (introducción al volumen de abril de 1995 de *IEEE Transactions on Software Engineering*) enumeran las áreas de investigación más promisorias:

- Lenguajes de descripción de arquitecturas (ADLs);
- Fundamentos formales de la AS (bases matemáticas, caracterizaciones formales de propiedades extra-funcionales, teorías de la interconexión);
- Técnicas de análisis arquitectónico;
- Métodos de desarrollo basados en arquitectura;
- Recuperación y reutilización de arquitectura;
- Codificación y guía arquitectónica;
- Herramientas y ambientes de diseño;
- Estudios de casos.

Clements [Cle96b] define cinco temas fundamentales de la disciplina:

Tema	Problema que aborda
Diseño o selección	Cómo crear o seleccionar una arquitectura sobre la base de requerimientos funcionales, de performance o de calidad.
Representación	Cómo comunicar una arquitectura (recursos lingüísticos + selección de la información a comunicar).
Evaluación y análisis	Cómo analizar una arquitectura para predecir cualidades del sistema; cómo comparar y escoger entre arquitecturas en competencia.
Desarrollo y evolución	Cómo construir y mantener un sistema dada una representación que se cree que es la arquitectura que resolverá el problema.
Recuperación	Cómo hacer evolucionar un sistema <i>legacy</i> cuando los cambios afectan su estructura; si no hay documentación confiable, se requiere primero una " arqueología arquitectónica " que extraiga su arquitectura.

Mary Shaw (2001) considera que los campos más promisorios siguen siendo el tratamiento sistemático de los estilos, los ADLs, la formulación de metodologías y el trabajo con patrones; se requieren aún modelos precisos para razonar sobre propiedades, verificar consistencia y completitud, y automatizar el análisis, el diseño y la síntesis. Estima que la AS está en su fase de desarrollo y extensión, pero que **ni las ideas ni las herramientas están maduras**. Fundamental en la concepción de Clements y Northrop es el criterio de **reusabilidad**: el estudio actual de la AS puede verse como un esfuerzo *ex post facto* por proporcionar un almacén estructurado de información reutilizable de diseño de alto nivel propio de una **familia** (en el sentido de Parnas), de modo que las decisiones de alto nivel no deban ser re-inventadas, re-validadas y re-descriptas.

9. Las seis corrientes o escuelas

En los 90 la AS se establece definitivamente como un dominio separado — todavía de manera confusa — de la ingeniería (el marco global) y del diseño (la práctica puntual). No hay un discurso autoconsciente sobre "escuelas", pero pueden distinguirse a grandes rasgos **seis corrientes**. Fuera de la metodología, el único factor reconocible de discordia ha sido la preeminencia de **UML y el diseño orientado a objetos**; todo lo demás parece negociable.

Corriente	Referentes	Rasgos distintivos
1. Arquitectura como etapa de la ingeniería y el diseño OO	Rumbaugh, Jacobson, Booch ("Los Tres Amigos"), Larman; mundo UML/Rational	La arquitectura se restringe a las fases iniciales y a los niveles más altos de abstracción, pero sin ligazón sistemática con el requerimiento previo ni la composición del diseño posterior; se confunde con el modelado y el diseño; a cualquier topología se la llama arquitectura; predilección por modelado denso y profusión de diagramas (CMM/UPM); los estilos se confunden con patrones; jamás se mencionan los ADLs . Definición típica (Booch): "la estructura lógica y física de un sistema, forjada por todas las decisiones estratégicas y tácticas que se aplican durante el desarrollo" — arquitectura isomorfa a las piezas de código .
2. Arquitectura estructural	CMU: Shaw, Clements, Garlan, Allen, Abowd, Ockerbloom	Corriente fundacional y clásica ; modelo estático de estilos, ADLs y vistas; dominante en la academia, mal conocida en la práctica industrial. Tres modalidades según formalización: descripciones verbales/diagramas de cajas ("boxología"), ADLs, y lenguajes formales (CHAM, Z — grupo de Moriconi, SRI). El diseño arquitectónico no tiene por qué coincidir con la configuración explícita de las aplicaciones : rara vez se habla de lenguajes, clases u objetos. División interna: unos excluyen el modelo de datos (Shaw, Garlan), otros lo reivindican (Medvidovic, Taylor). Todo estructuralismo es estático.
3. Estructuralismo arquitectónico radical	Desprendimiento europeo de la anterior	Actitud confrontativa con UML: una tendencia excluye de plano la relevancia del modelado OO para la AS; otra procura definir nuevos metamodelos y estereotipos de UML como correctivos.
4. Arquitectura basada en patrones	Comunidad POSA (Buschmann et al.); secundariamente GoF	Surge hacia 1996; reconoce el modelo OO pero sin rigidez UML/CMM. Nótese la diferencia entre sus dos "textos sagrados": en POSA "Software Architecture" figura en el título; el GoF trata la arquitectura mínimamente. Tolerancia hacia procesos tácticos; simpatía por Fowler y XP. El diseño consiste en identificar y articular patrones preexistentes , definidos de forma parecida a los estilos.
5. Arquitectura procesual	SEI, siglo XXI: Kazman, Bass, Clements, Bachmann, Barbacci, Carrière, Weinstock	Establece modelos de ciclo de vida y técnicas específicas para la arquitectura: elicitación de requerimientos, brainstorming, diseño, análisis, selección de alternativas, validación, comparación, estimación de calidad y justificación económica. Su documentación no se mezcla jamás con la de CMM, a la que redefine de punta a punta.

6. Arquitectura basada en escenarios	Movimiento europeo con centro en Holanda: Eindhoven, Vrije, Groningen, Philips Research (Bosch, van Vliet, Obbink, Ionita, Hammer, de Bruin, Folmer, van Gurp)	La corriente más nueva ; recupera el nexo de la AS con los requerimientos y la funcionalidad, borroso en la arquitectura estructural clásica; especialización de la procesual. Usa casos de uso UML como herramienta informal (los casos de uso no están orientados a objetos). La profusión de holandeses es significativa: Eindhoven es el lugar donde surgió lo que Sharp proponía llamar "la escuela arquitectónica de Dijkstra".
---	--	---

Ha habido intentos de fundar otras variedades (arquitectura adaptativa inspirada en programación genética/caos/fractales; centrada en la acción con IA heideggeriana; epistemológicamente reflexiva con Popper o Kuhn), omitidas por falta de masa crítica. Pero hay un movimiento cismático digno de tomarse en serio: una **anti-arquitectura** que, en nombre de los métodos heterodoxos, se opone tanto al modelado OO sobredocumentado como a la abstracción académica. El **Manifiesto por la Programación Ágil** valoriza:

- Los **individuos y las interacciones** por encima de los procesos y las herramientas;
- El **software que funciona** por encima de la documentación exhaustiva;
- La **colaboración con el cliente** por encima de la negociación contractual;
- La **respuesta al cambio** por encima del seguimiento de un plan.

10. Arquitectura frente a diseño

Una vez reconocida la diferencia entre diseño e implementación, ¿es la AS solamente otra palabra para designar el diseño? La comunidad académica sostiene que difiere sustancialmente. **Taylor y Medvidovic** señalan que la literatura alberga tres posturas:

1. Arquitectura y diseño **son lo mismo**.
2. La arquitectura está **en un nivel de abstracción por encima del diseño**, o es simplemente otro paso (un artefacto) del proceso de desarrollo. *(Para Taylor y Medvidovic, la más cercana a la verdad.)*
3. La arquitectura es **algo nuevo y en alguna medida diferente** del diseño (sin especificar cómo ni cuánto).

El foco de la AS en la **estructura del sistema y las interconexiones** la distingue del diseño tradicional (p. ej., OO), concentrado en abstracciones de más bajo nivel (algoritmos y tipos de datos). A medida que la arquitectura de alto nivel se refina, **sus conectores pierden prominencia**, distribuyéndose entre elementos de más bajo nivel: la arquitectura se transforma en diseño.

Autor	Posición sobre la diferencia
Albin [Alb03]	La AS abarca las metodologías de diseño y de análisis; el arquitecto ejecuta una combinación de roles (analista, diseñador, ingeniero) bajo la orquestación del <i>chief architect</i> . La integración de metodologías y modelos — no la mera yuxtaposición de análisis + diseño — es lo que distingue a la AS.
Shaw & Garlan [SG96]	La AS es el primer paso de una secuencia de tres: (1) Arquitectura — asocia capacidades del requerimiento con componentes que las implementarán; descripción con componentes y conectores en términos de estilos, y operadores que componen sistemas de subsistemas; (2) Diseño del código — algoritmos y estructuras de datos; primitivas del lenguaje (números, caracteres, punteros, hilos); (3) Diseño ejecutable — nivel aún más bajo: asignación de memoria, formatos de datos.
Clements [CBK+96]	El diseño basado en arquitectura es un paradigma de desarrollo que difiere de las alternativas conocidas tanto como el diseño OO difería de sus predecesores.
Perry (1997)	La arquitectura concierne a un nivel de abstracción más elevado: se ocupa de componentes (no procedimientos), interacciones (no interfaces), restricciones (no algoritmos, procedimientos ni tipos). Composición de grano grueso (vs. fina composición procedural); interacciones como protocolo de alto nivel (vs. rpc, mensajes, llamadas a rutinas).
ABD [BBC+00]	El diseño arquitectónico es el de más elevado nivel de abstracción, enfrentando un requerimiento aún difuso, donde las decisiones son las más críticas y difíciles de modificar. Tres fundamentos: (1) descomposición de la función (acoplamiento y cohesión), (2) realización de los requerimientos de calidad y negocio a través de los estilos , (3) plantillas de software (patrones de interacción con servicios compartidos e infraestructura).

11. Problemas abiertos y relevancia de la disciplina

11.1 Problemas abiertos

Aunque la evaluación dominante es positiva, a comienzos de siglo se perciben desacuerdos y frustraciones: la definición original del proyecto, académica, guarda poca relación con la imagen de la AS en la industria; muchas presentaciones son posturas teóricas interesadas, más programáticas que instrumentales; y abundan las quejas por no haber consensuado una definición universal. El inventario de aspectos negativos de **Clements & Northrop**:

- **Los abogados de las posturas traen sus sesgos consigo:** las definiciones coinciden en su núcleo pero difieren seriamente en los bordes (¿racionalización?, ¿pasos de construcción?, ¿funcionalidad de los componentes o basta la topología?). Al leer textos de AS es esencial conocer la motivación de sus responsables.
- **El estudio de la AS sigue a la práctica, no la lidera:** evolucionó de observar los principios de diseño y las acciones de los diseñadores en sistemas reales; se redefine constantemente en función de lo que hacen los programadores.
- **El estudio es sumamente nuevo** (avalancha reciente de libros, conferencias y talleres).
- **Las fundamentaciones han sido imprecisas:** proliferan términos inciertos; p. ej., definir la arquitectura como "la estructura global del sistema" agrega confusión, porque implica que un sistema posee **una sola** estructura.
- **El término se usó en exceso:** arquitectura específica de dominio, de megaprogramación, de sistemas, de redes, de infraestructura, de aplicaciones, técnica, de framework, conceptual, de referencia, empresarial, de factoría, de manufactura... El significado se diluye porque "parece estar de moda".

Hacia el 2000, Taylor y Medvidovic advertían que la AS por momentos parecía **una moda que prometía más de lo realizable**, y les preocupaba que terminara confundiendo con los ADLs y con el diseño: la arquitectura seguía siendo una noción académica, con poca tecnología transferida a la industria y mal comunicada. **Baragry** cuestionó la metáfora de la arquitectura de edificios como columna vertebral de la disciplina: ha ocasionado la falta de acuerdo sobre cantidad y calidad de vistas y la carencia de un mecanismo formal de coordinación entre ellas.

La tabla de contrastes de Bosch: academia vs. industria

Academia	Industria
La arquitectura se define explícitamente	Prevalece una comprensión conceptual; definiciones explícitas mínimas, eventualmente mediante notaciones
La arquitectura consiste en componentes y conectores de primera clase	No hay conectores explícitos de primera clase (a veces soluciones <i>ad hoc</i> de binding en tiempo de ejecución)
Los ADLs describen la arquitectura explícitamente y a veces la generan	Se utilizan lenguajes de programación
Los componentes reutilizables son entidades de caja negra	Los componentes son grandes piezas de software de estructura interna compleja, no necesariamente encapsulados
Los componentes tienen interfaces con un solo punto de acceso	Las interfaces se proporcionan mediante entidades (clases), sin diferencia explícita con las que no son de interfaz

Prioridad a la **funcionalidad y la verificación formal**

La funcionalidad y los **atributos de calidad** (performance, robustez, tamaño, reusabilidad, mantenibilidad) tienen igual importancia

No conviene sobredimensionar el contraste: la mayor parte de las herramientas académicas se elaboró en proyectos industriales y de gobierno de misión crítica, y el número de problemas de consistencia detectados a tiempo gracias a la guía arquitectónica es colosal. La cuestión radica más bien en el desconocimiento que la industria tiene de la arquitectura académica y **en su obstinación por llamar arquitectura a lo que sólo es diseño.**

11.2 Relevancia de la AS

Barry Boehm (creador de COCOMO y del desarrollo en espiral)

*"Si un proyecto no ha logrado una arquitectura del sistema, incluyendo su justificación, **el proyecto no debe empezar el desarrollo en gran escala.** Si se especifica la arquitectura como un elemento a entregar, se la puede usar a lo largo de los procesos de desarrollo y mantenimiento."*

Síntesis de las virtudes de la AS articulando las opiniones convergentes de los expertos [CN96] [Garoo] — *estas siete virtudes anticipan las "trece razones" del capítulo 11:*

1. **Comunicación mutua:** alto nivel de abstracción común que la mayoría de los participantes puede usar para crear entendimiento, formar consenso y comunicarse; expone las restricciones de alto nivel y la justificación de las decisiones fundamentales.
2. **Decisiones tempranas de diseño:** la AS encarna las decisiones más tempranas, cuyo peso es desproporcionado respecto del desarrollo restante; funciona como la "puerta de peaje" del método SAAM: el desarrollo no puede proseguir hasta que los participantes aprueben su diseño.
3. **Restricciones constructivas:** proporciona *blueprints* parciales, indicando componentes y dependencias (p. ej., una vista en capas documenta los límites de abstracción e interfaces principales).
4. **Reutilización (abstracción transferible):** un modelo pequeño e intelectualmente tratable, transferible a sistemas con requerimientos parecidos; promueve la reutilización en gran escala de componentes y frameworks.
5. **Evolución:** expone las dimensiones a lo largo de las cuales evolucionará el sistema; esas "paredes perdurables" permiten estimar costos de modificación y manejar requerimientos cambiantes de interoperabilidad, prototipado y reutilización.
6. **Análisis:** nuevas oportunidades — consistencia del sistema, conformidad con restricciones de estilo y atributos de calidad, análisis de dependencias, análisis específicos de dominio y negocio.
7. **Administración:** los proyectos exitosos consideran una arquitectura viable como logro clave; su evaluación crítica conduce a una comprensión más clara de requerimientos, estrategias de implementación y riesgos potenciales.

Ejemplo desarrollado: el "tour de force" de Shaw & Garlan

Una de las demostraciones más elocuentes de la AS como herramienta de decisión: comparar **una misma solución de indexación de palabras clave en cuatro estilos diferentes** — datos compartidos, tubería y filtro, tipos abstractos de datos e invocación implícita. El estudio estima relaciones de dependencia, modularidad, refinamiento y reutilización, y las ventajas y desventajas de cada arquitectura, **antes de escribir una sola línea de código**; demuestra cómo los estilos definen tácticas de descomposición funcional y pautan el desarrollo, cerrando con tablas de comparación de atributos. Si hubiera que singularizar un trabajo representativo del estado del arte, sería esta demostración cabal — a treinta años de distancia — de que **Dijkstra tenía razón**.

La AS se encuentra reconocidamente en una etapa formativa: sus mejores teóricos consideran que la disciplina es **tentativa y en estado de flujo**. Pero con lo hecho ya existe un enorme repertorio de ideas, experiencias e instrumentos que ayudan a mejorar las prácticas aquí y ahora.

PARTE III

La arquitectura como conjunto de estructuras

Basada en Bass, Clements y Kazman, Software Architecture in Practice, 4.^a ed., capítulos 1–3: definición moderna, estructuras y vistas, las trece razones y el aparato conceptual de atributos de calidad, escenarios, tácticas y patrones.

-
- 12 Qué es (y qué no es) la arquitectura de software
 - 13 Estructuras arquitectónicas y vistas
 - 14 Qué hace "buena" a una arquitectura
 - 15 Las trece razones por las que la arquitectura importa
 - 16 Atributos de calidad, escenarios, tácticas y patrones
-

12. Qué es (y qué no es) la arquitectura de software

Definición central (SAIP, 4.ª ed.)

"La arquitectura de software de un sistema es **el conjunto de estructuras necesarias para razonar sobre el sistema**. Estas estructuras comprenden **elementos de software, relaciones entre ellos y propiedades de ambos**."

Esta definición contrasta deliberadamente con las que hablan de decisiones "tempranas", "mayores" o "importantes": muchas decisiones arquitectónicas **no** se toman temprano (sobre todo en proyectos ágiles o en espiral), muchas decisiones tempranas no son arquitectónicas, y es difícil saber si una decisión es "mayor" (a veces sólo el tiempo lo dice). Las **estructuras**, en cambio, son fáciles de identificar y constituyen una herramienta poderosa de diseño y análisis. En síntesis: **la arquitectura trata de estructuras que habilitan el razonamiento**.

12.1 Implicaciones de la definición

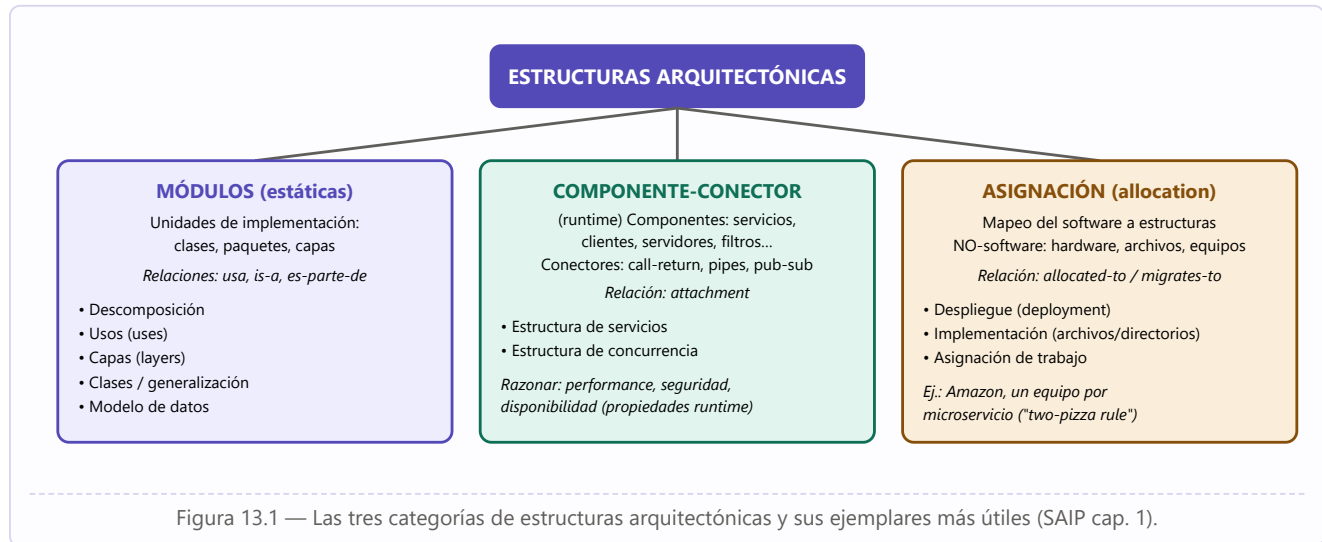
1. **La arquitectura es un conjunto de estructuras de software.** Una estructura es simplemente un conjunto de elementos unidos por una relación. Los sistemas tienen muchas estructuras y **ninguna puede reclamar en exclusiva ser "la" arquitectura**. No todas las estructuras son arquitectónicas (ejemplo del libro: "las líneas de código que contienen la letra z, ordenadas por longitud" es una estructura, pero no interesante): una estructura **es arquitectónica si soporta el razonamiento sobre el sistema y sus propiedades** importantes para algún stakeholder. El conjunto de estructuras arquitectónicas no es fijo ni limitado: depende de qué sea útil razonar en tu contexto.
2. **La arquitectura es una abstracción.** Omite intencionalmente la información de los elementos que no es útil para razonar: se ocupa del **lado público** de las interfaces; los detalles privados — los de pura implementación interna — **no son arquitectónicos**. Esta abstracción es esencial para domar la complejidad: no podemos ni queremos lidiar con toda la complejidad todo el tiempo. El punto de la arquitectura es que **no haga falta tener cada detalle en la cabeza**.
3. **Arquitectura es diseño, pero no todo diseño es arquitectura.** Muchas decisiones quedan sin ligar por la arquitectura y corresponden al criterio de los diseñadores e implementadores posteriores.
4. **Todo sistema tiene una arquitectura**, porque todo sistema tiene elementos y relaciones — pero eso no implica que alguien la conozca. De ahí la diferencia entre la arquitectura y su **representación**, y la importancia de la documentación.
5. **No toda arquitectura es buena.** La definición es indiferente a ello: una arquitectura puede favorecer o estorbar los requerimientos. **No hay arquitecturas intrínsecamente buenas o malas: son más o menos aptas para algún propósito** (una SOA de tres capas puede ser ideal para un B2B empresarial y completamente errónea para aviónica). De ahí la importancia del diseño (cap. 33) y la evaluación (cap. 34).
6. **La arquitectura incluye comportamiento:** la conducta de cada elemento es parte de la arquitectura en la medida en que ayude a razonar sobre el sistema.

Dos disciplinas relacionadas imponen restricciones a la AS: la **arquitectura de sistema** (totalidad de hardware, software y humanos; mapeo de funcionalidad a componentes hw/sw) y la **arquitectura empresarial** (estructura y conducta de procesos, flujos de información, personal y unidades organizacionales;

cómo los sistemas de software soportan los objetivos del negocio). Ambas comparten con la AS que pueden diseñarse, evaluarse y documentarse; responden a requerimientos; satisfacen stakeholders; y consisten en estructuras de elementos y relaciones.

13. Estructuras arquitectónicas y vistas

Las estructuras tienen contrapartes en la naturaleza: el neurólogo, el ortopedista y el hematólogo ven distintas estructuras del cuerpo humano — distintas pero inherentemente relacionadas: juntas describen la arquitectura del cuerpo. Igual con el software. Las estructuras se dividen en **tres categorías** según la naturaleza de sus elementos y el razonamiento que soportan:



13.1 Estructuras de módulos

Particionan el sistema en **unidades de implementación** ("módulos": clases, paquetes, capas o meras divisiones de funcionalidad) con responsabilidades computacionales asignadas — son la base de las asignaciones de trabajo a los equipos. Visión **estática**. Las más útiles:

- **Descomposición:** relación "es-submódulo-de"; los módulos se descomponen recursivamente hasta que sean fáciles de entender. Punto de partida común del diseño; determina en gran medida la **modificabilidad** (¿los cambios caen en pocos módulos, preferiblemente pequeños?); base de la organización del proyecto (documentación, integración, planes de prueba).
- **Usos (uses):** importante y a menudo pasada por alto. Una unidad **usa** otra si su corrección requiere la presencia de una versión correcta (no un stub) de la segunda. Sirve para construir **subconjuntos funcionales y extensiones** — habilita el desarrollo incremental. También es la base para medir la "**deuda social**": cuánta comunicación real (versus la debida) hay entre equipos.
- **Capas:** cada capa es una "**máquina virtual**" que ofrece un conjunto cohesivo de servicios a través de una interfaz gestionada; en sistemas estrictos una capa sólo usa a la inmediata inferior. Otorga **portabilidad** (cambiar la máquina virtual subyacente).
- **Clases / generalización:** relación "hereda-de" / "es-instancia-de"; razona sobre reutilización y adición incremental de funcionalidad.
- **Modelo de datos:** entidades y relaciones (Cuenta, Cliente, Préstamo; un cliente tiene una o más cuentas...).

13.2 Estructuras componente-y-conector (C&C)

Muestran una vista **de tiempo de ejecución**: los módulos ya compilados en ejecutables. Los **componentes** son las unidades principales de cómputo (servicios, peers, clientes, servidores, filtros); los **conectores** son los vehículos de comunicación (call-return, sincronización de procesos, pipes). La relación es **attachment** (qué conectores enganchan qué componentes). Responden: ¿cuáles son los principales componentes en ejecución y cómo interactúan? ¿cuáles son los almacenes de datos compartidos? ¿qué partes están replicadas? ¿cómo progresan los datos? ¿qué puede correr en paralelo? ¿puede cambiar la estructura durante la ejecución? Son cruciales para las **propiedades runtime**: performance, seguridad, disponibilidad. Ejemplos útiles: estructura de **servicios** (interoperan por mecanismos de coordinación) y de **concurrency** ("hilos lógicos" para hallar oportunidades de paralelismo y contención de recursos).

Cuidado: módulos ≠ componentes

Las estructuras C&C son **ortogonales** a las de módulos: una unidad de código puede compilarse en un servicio replicado miles de veces en ejecución, y mil módulos pueden enlazarse en un único ejecutable. En general los mapeos entre estructuras son **muchos-a-muchos**. Ejemplo del libro: un sistema cliente-servidor con **2 módulos** (software de cliente y de servidor) pero, en runtime, **11 componentes** (10 clientes + 1 servidor) y 10 conectores; la vista C&C sirve para análisis de performance y cuellos de botella, imposibles con la vista de módulos. En map-reduce: un módulo para todo el sistema, un componente por nodo (miles).

13.3 Estructuras de asignación

Mapean las estructuras de software sobre entornos no-software: **despliegue** (software sobre procesadores y vías de comunicación; relaciones "allocated-to" y "migrates-to" si es dinámica; clave para performance, integridad, seguridad, disponibilidad y deployabilidad), **implementación** (elementos sobre la estructura de archivos de desarrollo/integración/configuración; crítica para la gestión y los builds) y **asignación de trabajo** (responsabilidad de implementar e integrar módulos por equipos; tiene implicaciones arquitectónicas además de gerenciales — el arquitecto sabrá la pericia requerida por equipo).

13.4 Elegir estructuras y relacionarlas

Consejos del libro: (1) los proyectos suelen considerar **dominante** una estructura (a menudo la descomposición de módulos, porque engendra la estructura del proyecto — espeja la organización de equipos); (2) **menos es más**: no todos los sistemas ameritan muchas estructuras; cuanto más grande el sistema, más marcadas las diferencias entre estructuras; diseñar y documentar una estructura sólo si trae retorno positivo (menores costos de desarrollo o mantenimiento); (3) elegir las estructuras que den **insight y palanca sobre los atributos de calidad más importantes**.

Cuando los elementos se componen de maneras que resuelven problemas recurrentes y probados en muchos dominios, esas composiciones documentadas y diseminadas se llaman **patrones arquitectónicos** (empaquetan estrategias listas para problemas conocidos).

14. Qué hace "buena" a una arquitectura

No existe una arquitectura inherentemente buena o mala: son **más o menos aptas para algún propósito** — y la evaluación sólo tiene sentido en presencia de **metas explícitas**. Aun así, hay reglas generales, aplicables proactivamente (greenfield) o como heurísticas de análisis (sistemas existentes). Fallar en alguna no es fatal, pero es una señal de advertencia a investigar.

14.1 Recomendaciones de proceso

1. La arquitectura debe ser producto de **un único arquitecto o un grupo pequeño con un líder técnico identificado** — para dar **integridad conceptual** y consistencia técnica (vale también para proyectos ágiles y open source). Conexión fuerte con el equipo de desarrollo para evitar diseños de "torre de marfil".
2. El arquitecto debe basar la arquitectura, de forma continua, en una **lista priorizada de requerimientos de atributos de calidad bien especificados**, que informarán los tradeoffs. **La funcionalidad importa menos**.
3. Documentar la arquitectura **con vistas** (vista = representación de una o más estructuras) que atiendan las incumbencias de los stakeholders más importantes, al ritmo del proyecto (mínima al principio, elaborada después).
4. **Evaluar** la arquitectura por su capacidad de entregar los atributos de calidad importantes — **temprano** (cuando más beneficio devuelve) y repetidamente.
5. Que se preste a la **implementación incremental**, evitando la integración big-bang ("casi nunca funciona"): crear un **sistema esquelético** donde las vías de comunicación estén ejercitadas con funcionalidad mínima e ir "creciendo" el sistema, refactorizando según sea necesario.

14.2 Recomendaciones estructurales

1. Módulos bien definidos según **ocultamiento de información y separación de incumbencias**; encapsular lo que probablemente cambie; interfaces que permitan a los equipos trabajar con independencia.
2. Salvo requerimientos sin precedentes, lograr los atributos de calidad usando **patrones y tácticas bien conocidos** específicos de cada atributo.
3. **Nunca depender de una versión particular** de un producto o herramienta comercial; y si es inevitable, estructurar el sistema para que cambiar de versión sea sencillo y barato.
4. Separar los módulos que **producen** datos de los que los **consumen** (los cambios suelen confinarse a un solo lado; si se agrega un dato nuevo, la separación permite una actualización escalonada).
5. **No esperar correspondencia uno-a-uno entre módulos y componentes** (conurrencia: múltiples instancias de un componente construido del mismo módulo; un hilo puede usar servicios de varios componentes...).
6. Escribir cada proceso de modo que su **asignación a un procesador específico pueda cambiarse fácilmente**, incluso en runtime (fuerza motriz de la virtualización y la nube).
7. Usar un **número pequeño de patrones de interacción simples**: que el sistema haga las mismas cosas de la misma manera en todas partes — comprensibilidad, menos tiempo de desarrollo, fiabilidad, modificabilidad.

8. Contener un conjunto **específico y pequeño de áreas de contención de recursos**, con resolución claramente especificada y mantenida (presupuestos de tráfico de red o de tiempo por equipo, aplicados y verificados).

15. Las trece razones por las que la arquitectura importa

Trece razones técnicas y no técnicas — también son trece **usos** de la arquitectura en un proyecto, o trece formas de justificar el esfuerzo invertido en ella:

#	Razón	Desarrollo
1	Inhibe o habilita los atributos de calidad	"Si no recordás nada más de este libro, recordá esto." Performance → gestionar tiempo, recursos compartidos y comunicación entre elementos; modificabilidad → asignar responsabilidades y limitar acoplamiento para que cada cambio afecte pocos elementos (idealmente uno); seguridad → gestionar y proteger la comunicación, controlar accesos, elementos especializados (autorización, perímetro); escalabilidad → localizar recursos y no "hardcodear" supuestos; subconjuntos incrementales → gestionar el uso inter-componentes; reutilización → restringir el acoplamiento para que al extraer un elemento no salga con demasiados "apegos". Pero la arquitectura sola no garantiza nada: <i>"lo que la arquitectura da, la implementación puede quitarlo"</i> .
2	Permite razonar y gestionar el cambio	~80% del costo total ocurre después del despliegue inicial. Toda arquitectura particiona los cambios posibles en tres: locales (modificar un solo elemento — los deseables), no locales (varios elementos, sin tocar el enfoque — al menos escalonables) y arquitectónicos (cambian la forma en que los elementos interactúan; p. ej., pasar de mono a multihilo — repercuten en todo). La arquitectura efectiva es aquella donde los cambios más frecuentes son locales. Sin atención a la integridad conceptual → deuda arquitectónica (cap. 36).
3	Predice cualidades del sistema	Es posible hacer predicciones de calidad evaluando sólo la arquitectura , sin construir: si sabemos que ciertas decisiones producen ciertas cualidades, podemos tomarlas y esperar el resultado — y al examinar una arquitectura, verificar si fueron tomadas. Aun sin modelado cuantitativo, este principio detecta problemas temprano.
4	Facilita la comunicación entre stakeholders	Abstracción común que casi todos pueden usar para entendimiento mutuo, negociación y consenso: usuario (rápido, confiable), cliente (plazo y presupuesto), gerente (equipos independientes, interacción disciplinada), arquitecto (todo lo anterior). Anécdota del libro ("¿qué pasa cuando aprieto el botón de modo?"): en una revisión, la primera lámina de arquitectura disparó 45 minutos de debate <i>de requerimientos</i> que nunca se había dado — ver el sistema de una manera nueva hace aflorar preguntas nuevas; para los usuarios, la arquitectura es esa manera nueva.
5	Porta las decisiones más tempranas	Las decisiones iniciales cargan un peso desproporcionado: constriñen todo lo que sigue (como las primeras decisiones de un cuadro). ¿Correrá en un procesador o distribuido? ¿Capas? ¿cuántas y qué hace cada una? ¿Comunicación síncrona o asíncrona? ¿Se cifra la información? ¿Qué SO? ¿Qué protocolo? — imaginar la pesadilla de cambiar cualquiera de ellas: el "ripple" puede volverse avalancha.
6	Define restricciones a la implementación	La implementación conforme debe tener los elementos prescritos, interactuando como se prescribió, cumpliendo cada uno su responsabilidad. Ejemplo clásico: presupuestos de performance — si cada unidad respeta el suyo, la transacción global cumple su requerimiento; los implementadores pueden ignorar el presupuesto global. Recíprocamente, el arquitecto no necesita ser experto en todos los algoritmos, pero es responsable de establecer, analizar y hacer cumplir las decisiones y tradeoffs.

7	Dicta la estructura organizacional (o viceversa)	La work-breakdown structure suele basarse en la descomposición del sistema, y a su vez dicta planificación, presupuesto, canales de comunicación, control de configuración, planes de integración y prueba... hasta quién se sienta con quién en el picnic. Efecto colateral: la WBS congela aspectos de la arquitectura (los grupos resisten redistribuir responsabilidades; con contratos, cambiar es caro o litigioso) — un argumento más para analizar la arquitectura de un sistema grande antes de fijarla.
8	Habilita el desarrollo incremental	Sistema esquelético : la infraestructura (inicialización, comunicación, datos compartidos, acceso a recursos, errores, logging) presente, la funcionalidad después — "diseña y construí un poco de infraestructura para un poco de funcionalidad de punta a punta; repetí hasta terminar". Ejemplos extensibles por plug-ins: R, VS Code, browsers. Es el "walking skeleton" de Cockburn y el espíritu del MVP; reduce riesgo y permite detectar problemas de performance temprano.
9	Base de estimaciones de costo y calendario	Las mejores estimaciones surgen del consenso entre las top-down (arquitecto y PM) y las bottom-up (desarrolladores, sobre los elementos que les tocan — más precisas y con más ownership). Requerimientos revisados y validados mejoran todo.
10	Modelo transferible y reutilizable	Cuanto más temprano en el ciclo de vida se reutiliza, mayor el beneficio; la reutilización de arquitecturas apalanca sistemas con requerimientos similares y transfiere todas las consecuencias de las decisiones tempranas. Corazón de las líneas de producto (conjunto de sistemas construidos con activos compartidos): la arquitectura define lo fijo y lo variable; pagos de orden de magnitud en time-to-market, costo, productividad y calidad.
11	Permite incorporar elementos desarrollados independientemente	Del "progreso por líneas de código" a componer/ensamblar elementos separados (COTS, open source, apps, servicios): la arquitectura define qué puede enchufarse (cómo recibe y cede control, qué datos consume/produce, protocolos). Beneficios: menor time-to-market, más fiabilidad, menor costo, flexibilidad. Sistema abierto : estándares de comportamiento e interacción que evitan el <i>vendor lock-in</i> . Industria de herramientas: Ant, Maven, MSBuild, Jenkins.
12	Restringe el vocabulario de alternativas de diseño	Aunque los elementos pueden combinarse de infinitas maneras, conviene restringirse voluntariamente a pocas elecciones : el ingeniero no es un <i>artiste</i> ; la disciplina viene, en parte, de restringir el vocabulario a soluciones probadas (tácticas y patrones). Beneficios: reutilización, diseños regulares y comprensibles, análisis con confianza, menor tiempo de selección, interoperabilidad. Lo no precedente es riesgoso; innovar sólo cuando lo existente es insuficiente.
13	Base para la formación	Primera introducción del sistema a un nuevo miembro: vistas de módulos (quién hace qué, qué equipo tiene qué), C&C (cómo funciona y cumple su trabajo), asignación (dónde encaja mi parte en el desarrollo o despliegue).

16. Atributos de calidad, escenarios, tácticas y patrones

16.1 Funcionalidad y responsabilidades

La funcionalidad no determina la arquitectura: dado un conjunto de funciones requeridas, hay infinitas arquitecturas que las satisfacen — de hecho, si sólo importara la funcionalidad, un **blob monolítico sin estructura interna** bastaría. Estructuramos los sistemas (módulos, capas, servicios, hilos, tiers...) para hacerlos comprensibles y para servir "otros propósitos": los atributos de calidad. Los sistemas se rediseñan **no por deficiencia funcional** (los reemplazos suelen ser funcionalmente idénticos) sino porque son difíciles de mantener, portar o escalar, o lentos, o fueron vulnerados. La funcionalidad se logra **asignando responsabilidades** a los elementos (de allí la estructura de descomposición); el interés del arquitecto en la funcionalidad es cómo interactúa con las demás cualidades y las restringe. El libro prefiere el término **"responsabilidad"**: preguntas como "¿qué restricciones de tiempo tiene este conjunto de responsabilidades?" son accionables, mientras que "requerimiento no funcional" y aun "funcional" son categorías resbaladizas.

16.2 Atributos de calidad (QA)

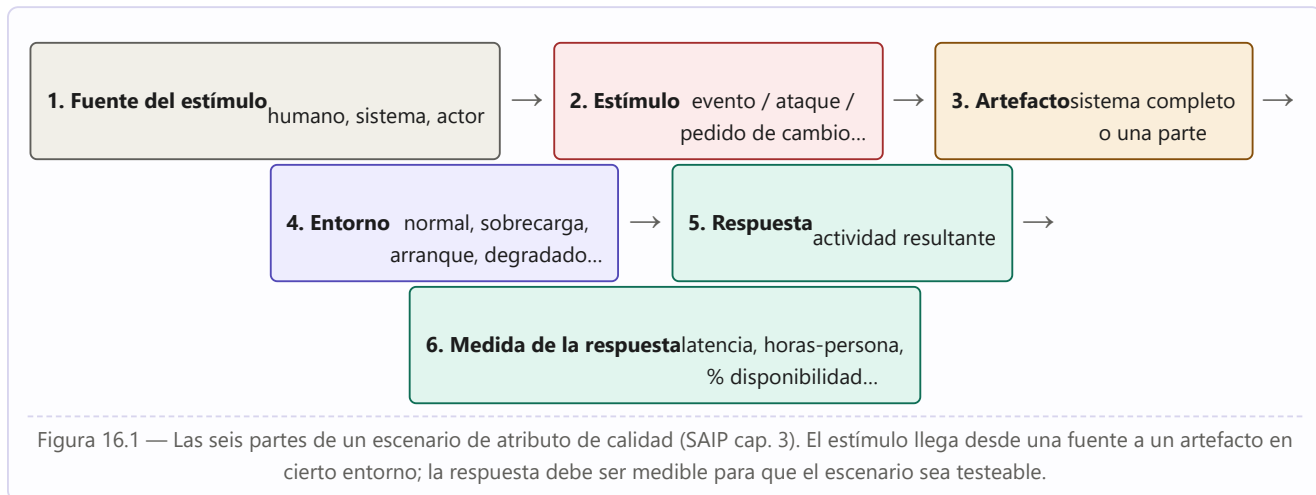
Definición: Atributo de calidad

Un atributo de calidad (QA) es una **propiedad medible o testeable de un sistema, que indica cuán bien satisface las necesidades de sus stakeholders más allá de la función básica** — la "utilidad" del producto en alguna dimensión de interés.

Tres problemas clásicos de las discusiones de QAs: (1) **definiciones no testeables** ("el sistema será modificable" no significa nada: todo sistema es modificable frente a un conjunto de cambios y no frente a otro); (2) **disputas de pertenencia** (un ataque DoS ¿es disponibilidad, performance, seguridad o usabilidad? — todas las comunidades lo reclaman y el debate no ayuda a diseñar); (3) **vocabularios propios** (a la performance le "llegan eventos"; a la seguridad, "ataques"; a la disponibilidad, "fallas"; a la usabilidad, "input del usuario" — pueden ser la misma ocurrencia). La solución a (1) y (2) son los **escenarios**; a (3), presentar los conceptos de cada comunidad en una forma común.

Dos categorías: QAs que describen al **sistema en ejecución** (disponibilidad, performance, usabilidad, seguridad, safety, energía) y QAs del **desarrollo del sistema** (modificabilidad, testabilidad, deployabilidad, integrabilidad). **Ningún QA se logra en aislamiento:** lograr uno afecta — a veces positiva, a veces negativamente — a los otros (casi todos afectan negativamente a la performance; ej. portabilidad → aislar dependencias → overhead). Diseñar es, en parte, hacer los tradeoffs apropiados.

16.3 El escenario de atributo de calidad (6 partes)



El **escenario** es la forma común de especificar todos los requerimientos de QA: testeable, sin ambigüedad, insensible a los caprichos de categorización. Se distinguen **escenarios generales** (independientes del sistema, sirven a cualquier sistema — el libro trae uno por cada QA de los caps. 4–13, para facilitar el brainstorming) y **escenarios concretos** (específicos del sistema en consideración): es mucho más fácil que un stakeholder adapte un escenario general a su sistema a que lo genere de la nada. Es común omitir partes al principio, pero saber que las seis existen obliga a considerar si cada una es relevante. El **entorno** puede referir incluso a estados donde el sistema no corre (desarrollo, testing, recarga); la quinta falla consecutiva de un componente puede tratarse distinto que la primera.

Ejemplos de escenarios concretos (uno por tipo)

Disponibilidad: "Un servidor de una granja falla en operación normal; el sistema informa al operador y sigue operando sin downtime". **Performance:** "500 usuarios inician 2.000 requests en 30 segundos en operación normal; el sistema los procesa con latencia promedio de 2 segundos". **Modificabilidad:** "El desarrollador quiere cambiar la UI en código en tiempo de diseño; el cambio se hace y prueba en menos de 3 horas sin efectos colaterales". **Deployabilidad:** "Una nueva versión de un servicio de autenticación se prueba y despliega a producción en 40 horas de tiempo transcurrido y no más de 120 horas-persona, sin defectos ni violación de SLA".

Anécdota: "Not My Problem" (Livermore Labs)

En un análisis para el Lawrence Livermore National Laboratory (cuyo sitio repite "security" por doquier), los stakeholders enumeraron performance, modificabilidad, evolucionabilidad, interoperabilidad... y **jamás mencionaron seguridad**. Su respuesta: "No nos importa: nuestros sistemas no están conectados a ninguna red externa y tenemos cercas de púas y guardias con ametralladoras". Lección: **el arquitecto de software puede no ser responsable de todos los requerimientos de QA**.

16.4 Tácticas y patrones arquitectónicos

Definiciones: Táctica y Patrón

Una **táctica** es una decisión de diseño que influye en el logro de la respuesta de un atributo de calidad — afecta directamente la respuesta del sistema a un estímulo. Un **patrón arquitectónico** describe un problema de diseño recurrente que surge en contextos específicos y presenta una solución arquitectónica bien probada, especificando roles, responsabilidades, relaciones y colaboraciones de sus elementos. Los patrones **"empaquetan"** tácticas y por eso suelen implicar tradeoffs entre varios QAs a la vez; la táctica, en cambio, se enfoca en una sola respuesta de un solo QA.

¿Por qué enfocarse en las tácticas? (1) A veces **ningún patrón resuelve el problema completo** (se necesita el broker de alta disponibilidad y alta seguridad, no el de libro): las tácticas dan un medio sistemático de **aumentar un patrón** para llenar los huecos. (2) Si no existe patrón, las tácticas permiten construir un fragmento de diseño desde "primeros principios". (3) Hacen el diseño y el análisis más sistemáticos. Las tácticas son **primitivas de diseño** y por eso reaparecen por todas partes: hay "súper-tácticas" tan fundamentales que merecen mención — **encapsular, restringir dependencias, usar un intermediario, abstraer servicios comunes** (de modificabilidad, presentes en casi todos los patrones), el **scheduling** (un load balancer es un intermediario que hace scheduling) y el **monitoreo** (aparece en energía, performance, disponibilidad y safety). Cada táctica se **refina** al aplicarse ("schedule resources" → shortest-job-first o round-robin; "usar intermediario" → capa, broker, proxy o tier) y su aplicación depende del contexto ("manage sampling rate" vale en ciertos sistemas de tiempo real, jamás en una base de datos o trading donde perder un evento es grave).

Deterioro y deuda: la "muerte por mil cortes"

Las arquitecturas suelen emerger y evolucionar por muchas pequeñas decisiones y fuerzas de negocio: un sistema tolerablemente modificable se deteriora con el tiempo por la acción bienintencionada de programadores enfocados en su tarea inmediata y no en preservar la integridad arquitectónica. Ese deterioro es una forma de deuda técnica llamada **deuda arquitectónica** (cap. 36); se revierte con **refactoring** (por seguridad, por performance, por modificabilidad — p. ej., factorizar la funcionalidad duplicada de dos módulos que siempre cambian juntos). Y ojo: lograr QAs también involucra decisiones **de proceso** — la mejor arquitectura de seguridad es inútil si los empleados caen en phishing o usan contraseñas débiles.

16.5 Cuestionarios basados en tácticas

Herramienta de análisis liviana: para cada QA, un cuestionario donde cada táctica se vuelve pregunta ("¿el sistema soporta la detección de intrusiones?"). Para cada una el analista registra: si está **soportada** (S/N); las **decisiones de diseño concretas y su ubicación** en el código (útil para auditoría y reconstrucción); el **riesgo** de su uso/no uso (escala H/M/L); y el **rationale** y los supuestos (incluida la decisión de NO usarla, con sus implicaciones de costo, calendario y evolución). Puede sonar simplista, pero es poderoso: obliga a dar un paso atrás y mirar el panorama; un cuestionario típico de un solo QA toma **entre 30 y 90 minutos**.

PARTE IV

Los diez atributos de calidad

SAIP 4.^a ed., capítulos 4–14: para cada atributo — concepto y definiciones, escenario general, catálogo de tácticas y patrones asociados, con sus beneficios y tradeoffs.

17	Disponibilidad
18	Deployabilidad
19	Eficiencia energética
20	Integrabilidad
21	Modificabilidad
22	Performance
23	Safety (inocuidad)
24	Seguridad
25	Testabilidad
26	Usabilidad
27	Trabajar con otros atributos de calidad

17. Disponibilidad (Availability)

Definición: Disponibilidad

Propiedad del software de **estar allí y listo para realizar su tarea cuando se lo necesita**. Engloba la fiabilidad (reliability) y le agrega la noción de **recuperación**: cuando el sistema se rompe, se repara a sí mismo. También abarca la capacidad de **enmascarar o reparar faltas** de modo que el período acumulado de interrupción del servicio no exceda un valor requerido en un intervalo de tiempo dado.

17.1 Conceptos: falta, error, falla

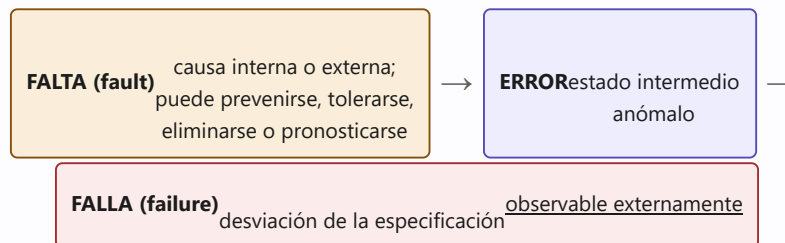


Figura 17.1 — La cadena falta → error → falla. Si el sistema se recupera de la falta sin desviación observable, **no hubo falla**; distinguirlas permite hablar de estrategias de reparación.

Una **falla** requiere un observador externo: la noción de "observabilidad" es crítica (si una falla pudo haberse observado, es una falla, se haya observado o no). El tiempo de reparación es el tiempo hasta que la falla deja de ser observable. La **disponibilidad estacionaria** se calcula (herencia del hardware):

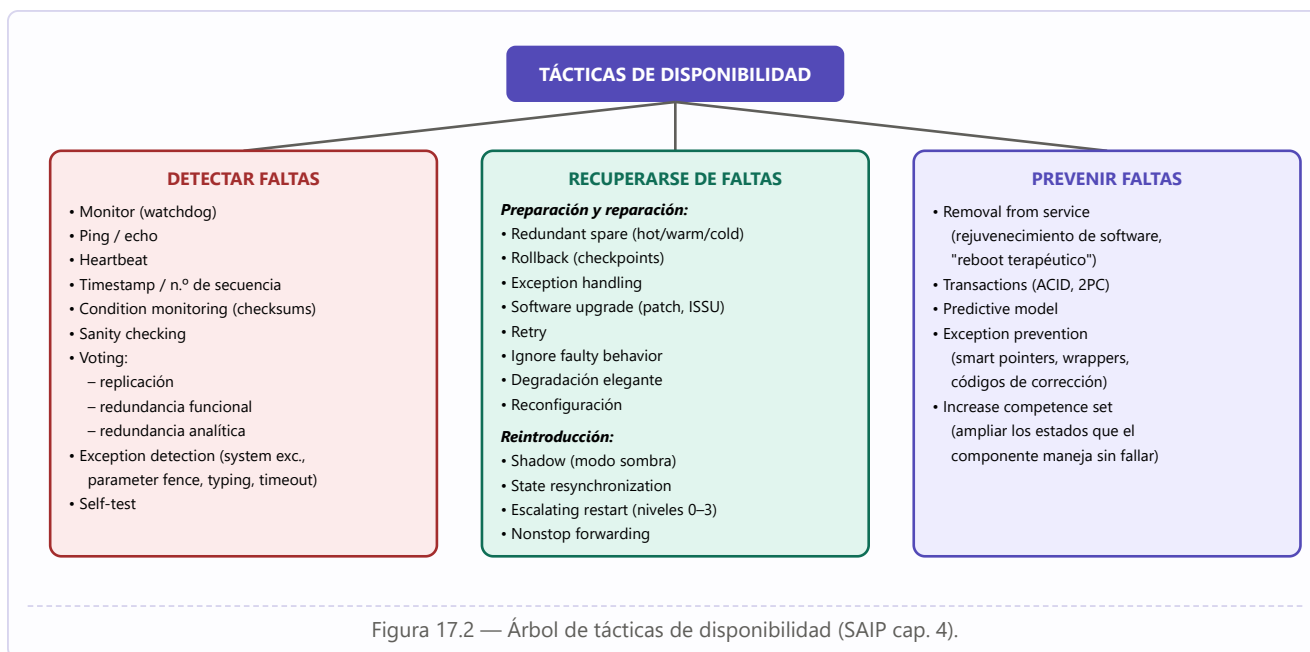
Fórmula clave

Disponibilidad = $MTBF / (MTBF + MTTR)$ — donde MTBF es el tiempo medio entre fallas y MTTR el tiempo medio de reparación. De aquí surgen afirmaciones como "99,999% de disponibilidad" (los célebres "**5 nueves**", que implican ≈ 5 minutos de baja por año). Los downtime **programados** no cuentan (el sistema "no se necesita" entonces) — sujeto al SLA. Ej. de SLA: AWS EC2 garantiza "Monthly Uptime Percentage of at least 99.99%" con créditos de servicio como penalidad.

Las faltas detectadas se categorizan por **severidad** (crítica, mayor, menor) e **impacto de servicio** (afecta o no afecta el servicio), lo que orienta la estrategia de reparación. La disponibilidad se relaciona estrechamente con la **seguridad** (un ataque DoS busca precisamente hacerla fallar), con la **performance** (es difícil distinguir "falló" de "responde lentísimo") y con **safety**.

17.2 Tácticas de disponibilidad

Tienen tres propósitos: **detectar** faltas, **recuperarse** de ellas y **prevenir**las. Muchas las provee la infraestructura (middleware): el trabajo del arquitecto suele ser **elegir y evaluar** (más que implementar) las tácticas correctas.



Detalle de las tácticas de detección

- **Monitor:** componente que vigila la salud de otras partes (procesadores, procesos, E/S, memoria); puede detectar congestión o DoS y orquesta las demás tácticas de detección. Si se implementa con un contador/temporizador que se reinicia periódicamente, se llama **watchdog** (y el proceso vigilado que lo reinicia "acaricia al watchdog").
- **Ping/echo:** par asíncrono de mensajes petición/respuesta para determinar alcanzabilidad y retardo de ida y vuelta; requiere umbral de tiempo (timeout).
- **Heartbeat:** mensaje periódico del componente monitoreado al monitor. Diferencia con ping/echo: **quién inicia el chequeo** (heartbeat: el componente; ping: el monitor). Puede "colgarse" (piggyback) de otros mensajes de control para reducir overhead.
- **Timestamp:** detecta secuencias incorrectas de eventos en sistemas distribuidos de paso de mensajes (o números de secuencia, ya que los relojes distribuidos pueden ser inconsistentes).
- **Condition monitoring / sanity checking:** verificar condiciones y suposiciones (checksums) / verificar validez o razonabilidad de salidas, típicamente en interfaces. El monitor debe ser simple (idealmente demostrable) para no introducir nuevos errores.
- **Voting:** comparar resultados de múltiples fuentes que deberían coincidir; la lógica de votación debe ser un singleton simple y rigurosamente probado. Tres esquemas: **replicación** (clones exactos — protege de fallas aleatorias de hardware, NO de errores de diseño, pues no hay diversidad); **redundancia funcional** (diversidad de diseño contra fallas de modo común — mismas salidas ante mismas entradas; más caro de desarrollar y verificar; vulnerable a errores de especificación); **redundancia analítica** (diversidad también de entradas y salidas; tolera errores de especificación; ej. aviónica: altitud calculada por presión barométrica, radioaltímetro y geometría — el votante debe ser sofisticado: entender qué sensores son confiables y hasta producir un valor de mayor fidelidad mezclando y suavizando).
- **Exception detection:** excepciones de sistema (división por cero, faltas de bus); **parameter fence** (patrón conocido, p. ej. 0xDEADBEEF, tras los parámetros de longitud variable para detectar sobrescritura); **parameter typing** (mensajes TLV: emisor y receptor acuerdan tipo y longitud); **timeout**.

Detalle de recuperación

- **Redundant spare**: uno o más duplicados listos para tomar el trabajo — corazón de los patrones hot/warm/cold spare.
- **Rollback**: revertir a un estado bueno conocido ("línea de rollback") usando un **checkpoint** previo; suele combinarse con transacciones y con spares (tras el rollback se promueve el standby).
- **Software upgrade en servicio**: **function patch** (parche incremental en código procedural), **class patch** (agregar datos/funciones miembro en runtime OO) — ambos para bug fixes; **hitless ISSU** (In-Service Software Upgrade sobre redundant spare) para features.
- **Retry** (asume falta transitoria; limitar reintentos), **ignorar conducta defectuosa** (mensajes espurios de un sensor moribundo), **degradación elegante** (mantener las funciones críticas, soltar las demás), **reconfiguración** (reasignar responsabilidades a los recursos que quedan).
- **Reintroducción**: **shadow** (operar el componente rehabilitado "en las sombras" un tiempo antes de devolverle el rol activo); **resincronización de estado** (con redundancia activa es orgánica; con pasiva, por checkpoints; comparación por checksum o hash); **escalating restart** (reiniciar con granularidad variable: nivel 0 mata/recrea hilos hijos; nivel 1 libera memoria no protegida; nivel 2 toda la memoria; nivel 3 recarga la imagen ejecutable completa); **nonstop forwarding** (de los routers: el plano de datos sigue reenviando paquetes por rutas conocidas mientras el plano de control se recupera y reconstruye incrementalmente — "graceful restart").

17.3 Patrones de disponibilidad

Patrón	Mecánica	Tradeoff clave
Redundancia activa (hot spare)	Todos los nodos del grupo de protección reciben y procesan las mismas entradas en paralelo ; el spare mantiene estado sincrónico.	Conmutación en milisegundos ; el más caro en cómputo. Caso 1+1 = "one-plus-one redundancy".
Redundancia pasiva (warm spare)	Sólo los activos procesan tráfico y envían actualizaciones periódicas de estado a los spares.	Equilibrio disponibilidad/costo; recuperación más lenta que hot.
Cold spare	Los spares permanecen fuera de servicio hasta el failover (power-on-reset previo al servicio).	MTTR alto: inadecuado para alta disponibilidad ; el más barato.

18. Deployabilidad (Deployability)

Definición: Deployabilidad

Propiedad del software que indica que puede ser **desplegado** — **asignado a un entorno de ejecución** — **dentro de un tiempo y esfuerzo predecibles y aceptables**; y que, si el despliegue no cumple sus especificaciones, puede ser **revertido (rollback)** también en tiempo y esfuerzo predecibles y aceptables.

Contexto: la **velocidad de release** como ventaja competitiva. Si el proceso de codificar→producción es totalmente automatizado se llama **despliegue continuo** (continuous deployment); si requiere intervención humana para el paso final (por regulación o política), **entrega continua** (continuous delivery). **DevOps** ("development"+"operations") es el movimiento asociado: *"conjunto de prácticas destinadas a reducir el tiempo entre la confirmación (commit) de un cambio y su puesta en producción normal, asegurando alta calidad"*. DevSecOps incorpora seguridad a todo el proceso (popular en aeroespacial/defensa).

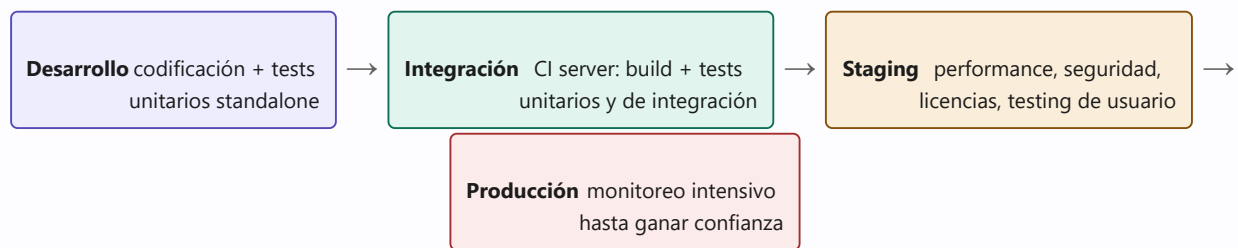


Figura 18.1 — El pipeline de despliegue y sus entornos (SAIP cap. 5). Con virtualización se logra "paridad de entornos", evitando el clásico "en mi máquina pasaba los tests".

Tres medidas de calidad del pipeline: **cycle time** (ritmo de avance — organizaciones que despliegan cientos de veces por día no pueden esperar coordinación entre equipos), **trazabilidad** (recuperar todos los artefactos que llevaron a un elemento: código, dependencias, test cases, herramientas — en una base de artefactos), **repetibilidad** (mismo resultado con los mismos artefactos: no es trivial — ej.: el build toma "la última" versión de una librería, o un test modifica la base de datos).

El arquitecto debe considerar: ¿cómo llega la actualización (push/pull)? ¿puede integrarse mientras el sistema corre? ¿medio y empaquetado? ¿eficiencia y controlabilidad del proceso? Interesan despliegues **granulares** (de elementos, no todo el sistema), **controlables** (desplegar a distinta granularidad, monitorear y revertir) y **eficientes**.

18.1 Tácticas

Categoría	Tácticas
Gestionar el pipeline de despliegue	Scale rollouts (desplegar gradualmente a subconjuntos controlados de usuarios, monitoreando y revirtiendo si es necesario; requiere mecanismo de ruteo por identidad de usuario); roll back (revertir despliegues multi-servicio de forma automatizada, rastreando todas las actualizaciones coordinadas); script deployment commands (despliegues complejos orquestados por scripts que se tratan <i>como código</i> : documentados, revisados, testeados, versionados).

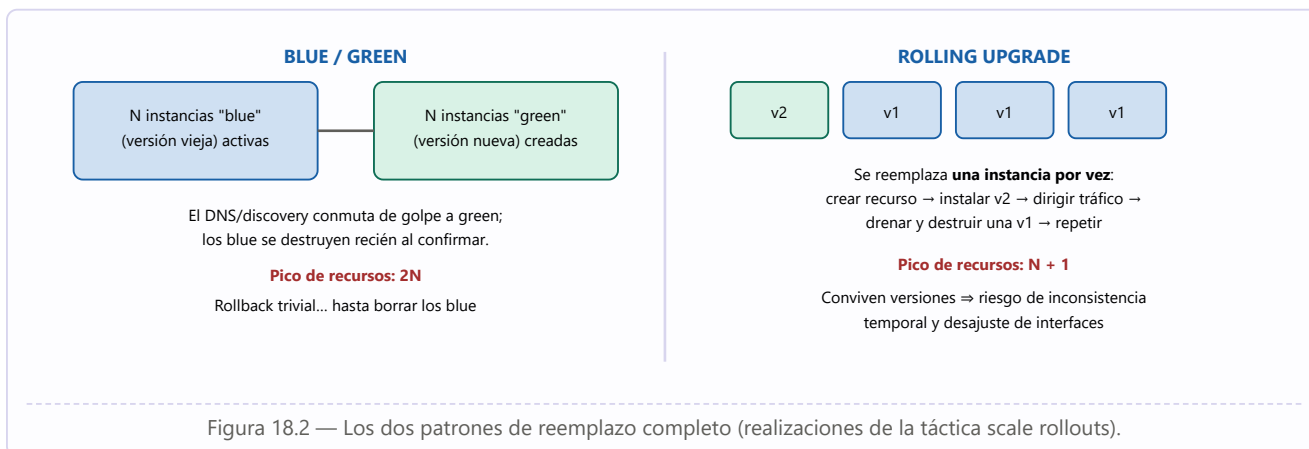
Gestionar el sistema desplegado	Manage service interactions (ejecutar simultáneamente múltiples versiones de un servicio, mediando interacciones para evitar incompatibilidades); package dependencies (empaquetar el elemento junto con sus dependencias — librerías, versiones de SO, contenedores utilitarios — para que se muevan consistentes de desarrollo a producción: contenedores, pods o VMs); feature toggle ("kill switch" que desactiva features en runtime sin redespargar).
--	--

18.2 Patrones

Microservicios (estructura de los servicios)

El sistema como colección de **servicios independientemente desplegables que se comunican únicamente por mensajes a través de interfaces de servicio**: sin enlace directo, sin leer el almacén de datos de otro equipo, sin memoria compartida, sin puertas traseras. Los servicios suelen ser stateless, desarrollados por un equipo pequeño (regla de Amazon de las "dos pizzas"), con dependencias acíclicas y un **servicio de descubrimiento** para el ruteo. **Beneficios**: time-to-market reducido (cada servicio se despliega sin coordinar con otros equipos); libertad tecnológica por equipo (elimina errores de incompatibilidad de integración); escalado sencillo por servicio. **Tradeoffs**: overhead de comunicación por red (mitigable con service mesh); inadecuados para transacciones complejas (sincronizar actividades distribuidas es difícil); la organización debe mantener todas las tecnologías elegidas; el control intelectual del sistema total es difícil (catálogos de interfaces); diseñar la granularidad correcta es un problema formidable. Usuarios intensivos: Google, Netflix, PayPal, Twitter, Facebook, Amazon.

Reemplazo total de servicios



Con **blue/green**, en cada instante hay una sola versión activa para los clientes; el problema es que al descubrir un error tardío quizá ya se borraron las instancias viejas. Con **rolling upgrade** conviven ambas versiones: aparecen la **inconsistencia temporal** (requests consecutivos servidos por versiones distintas — prevenible con *manage service interactions*) y el **desajuste de interfaces** (prevenible extendiendo la interfaz sin modificar la existente, con un mediador que traduzca).

Reemplazo parcial

Canary testing: probar la nueva versión en producción con un conjunto pequeño de usuarios "canarios" (power users que ejercitan casos límite, o empleados propios — en Google los empleados actúan como testers de los releases venideros); si el foco es medir aceptación de features, la variante se llama **dark launch**. **A/B testing**:

variantes simultáneas para distintos grupos, como experimento de mercado. (El nombre viene de los canarios de las minas de carbón del s. XIX, centinelas tempranos de gases tóxicos.)

19. Eficiencia energética

La energía dejó de ser "gratis e ilimitada": los data centers consumían ya en 2016 más energía que todo el Reino Unido ($\approx 3\%$ mundial, estimaciones recientes hasta 10%), y en el otro extremo móviles e IoT viven de baterías. Razones para tratarla como QA de primera clase: (1) sin técnicas *de todo el sistema* para monitorear y gestionar energía, los desarrolladores inventan soluciones ad hoc imposibles de mantener o medir; (2) la mayoría de arquitectos y desarrolladores desconoce el atributo (no se enseña); (3) faltan modelos, patrones y catálogos maduros. En la nube el problema es la asignación óptima de recursos (escalar hacia arriba y abajo); en móvil/IoT, el balance con performance y usabilidad (a veces derivando cómputo a la nube).

19.1 Tácticas

Categoría	Tácticas
Monitorear recursos <i>"No se puede gestionar lo que no se mide"</i>	Metering (recolectar consumo en tiempo casi real vía sensores: medidor del data center, PDUs por rack, ASICs; en baterías, el battery management system); static classification (cuando no hay datos en vivo: catalogar los recursos y sus consumos conocidos de benchmarks/especificaciones); dynamic classification (modelos dependientes de la carga: tabla, regresión o simulación).
Asignar recursos	Reduce usage (bajar refresco/brillo del display, apagar CPUs o servidores ociosos, bajar el reloj, consolidar VMs en menos servidores físicos, derivar cómputo a la nube si comunicar cuesta menos que computar); discovery (elegir proveedor de servicio según sus características energéticas anotadas — un "directorio verde"); schedule resources (asignación de tareas consciente de la energía, respetando restricciones y prioridades).
Reducir la demanda	Categoría remitida al cap. de performance: manage event arrival, limit event response, prioritize events (dejar sin servicio los de baja prioridad), reduce overhead, bound execution times, increase efficiency. Complementaria de <i>reduce usage</i> : aquí se gestiona (reduce) la demanda misma.

19.2 Patrones

- **Sensor fusion**: usar datos de un sensor de bajo consumo para inferir si vale la pena leer el de alto consumo (ej. móvil: acelerómetro decide si actualizar el GPS). Riesgo: si la inferencia casi siempre termina leyendo el sensor caro, el consumo total puede EMPEORAR.
- **Kill abnormal tasks**: monitorear apps de proveniencia desconocida y matar operaciones devoradoras de energía (ej. app que vibra y suena sin que el usuario responda $\rightarrow$ timeout $\rightarrow$ kill). Tradeoff con usabilidad.
- **Power monitor**: deshabilitar automáticamente dispositivos e interfaces que la aplicación no usa activamente (herencia de los circuitos integrados). Tradeoff: latencia y hasta pico de energía al reencender.

20. Integrabilidad

Más que "hacer cooperar componentes desarrollados por separado", la integrabilidad concierne a los **costos y riesgos técnicos de las tareas de integración futuras**, anticipadas o no. Formalización: integrar un conjunto de unidades $\{C_i\}$ en un sistema S ; la dificultad es función de:

Idea clave

Dificultad de integración = $f(\text{tamaño}, \text{distancia})$ — donde **tamaño** = número de dependencias potenciales entre $\{C_i\}$ y S , y **distancia** = dificultad de resolver las diferencias en cada dependencia. Las dependencias no son sólo sintácticas: hay acoplamiento **temporal** y **semántico** (recursos compartidos, protocolos, formatos, unidades de medida) — rara vez comprendido, reconocido o documentado; el conocimiento faltante o implícito siempre eleva costos y riesgos. La orientación a servicios reduce sólo la dependencia sintáctica.

20.1 Las cinco distancias

Distancia	Qué debe acordarse	Ejemplo
Sintáctica	Número y tipo de los datos compartidos.	Uno envía entero, el otro espera punto flotante; bit masks interpretadas distinto.
Semántica de datos	Interpretación de los valores aunque el tipo coincida.	Altitud en metros vs. pies (¡Mars Climate Orbiter!). Difícil de observar; ayudan los metadatos.
Semántica de comportamiento	Conducta respecto de estados y modos del sistema; quién inicia el control.	Un dato se interpreta distinto en arranque/apagado/recuperación; ambos esperan que el otro inicie.
Temporal	Supuestos sobre el tiempo.	Emitir a 10 Hz cuando el otro espera 60 Hz; "A sigue a B" vs. "A sigue a B con ≤ 50 ms de latencia".
De recursos	Supuestos sobre recursos compartidos.	Acceso exclusivo vs. compartido a un dispositivo; 12+10 GB requeridos con 16 GB físicos; 3 productores de 3 Mbps sobre un canal de 5 Mbps.

Analogía del libro: enchufar un aparato norteamericano en un tomacorriente británico requiere adaptador (sintaxis); si es de 110 V en una red de 220 V, transformador (semántica); y si espera 60 Hz donde hay 70 Hz, puede "enchufar bien" y aun así no funcionar (temporal).

20.2 Tácticas

Categoría	Tácticas y mecánica
-----------	---------------------

Limitar dependencias	Encapsular (la base de todas: interfaz explícita + todo acceso pasa por ella; reduce dependencias y distancias — rara vez se usa sola); usar un intermediario (pub-sub bus, repositorio compartido, discovery: eliminan conocer la identidad del otro; traductores de datos/protocolos resuelven distancias sintácticas y semánticas); restringir vías de comunicación (limitar con quién puede hablar cada elemento; en SOA se desalientan las conexiones punto a punto forzando el paso por un enterprise service bus); adherirse a estándares (habilitador primario de integrabilidad e interoperabilidad; su eficacia depende del alcance del estándar y de la probabilidad de que los futuros proveedores lo cumplan); abstraer servicios comunes (dos elementos similares tras una abstracción general — se integra una vez contra la abstracción; normaliza variaciones; ej.: sensores de distintos fabricantes tras una interfaz común, plug-ins de un browser).
Adaptar	Discover (catálogo de direcciones registradas estática o dinámicamente, con des-registro y health checks; entradas con atributos consultables); tailor interface (agregar u ocultar capacidades sin cambiar API ni implementación: buffering, validación de datos —contra SQL injection—, traducción de formatos; filtros interceptores, AOP); configure behavior (componentes configurables en build, inicialización o runtime — ej. soportar varias versiones de un estándar).
Coordinar	Orchestrate (un mecanismo de control invoca y coordina servicios que no se conocen entre sí: workflow engines, BPM, BPEL; centraliza las dependencias en el orquestador); manage resources (un gestor media todo acceso a recursos de cómputo — hilos, memoria — preservando invariantes y equidad: el SO, las transacciones de BD, thread pools, ARINC 653 para particionamiento espacio-temporal en sistemas críticos).

20.3 Patrones

- **Wrapper:** encapsula un componente en una abstracción alternativa; es el **único** autorizado a usarlo; traduce datos/control (ej. medidas imperiales ↔ métricas); puede traducir, ocultar o preservar cada elemento de la interfaz.
- **Bridge:** traduce los supuestos "requires" de un componente arbitrario a los "provides" de otro; a diferencia del wrapper es **independiente de un componente particular**, lo invoca un agente externo, suele ser **transitorio** y su traducción se define en el momento de su construcción; cuanto más supuestos aborda, a menos componentes aplica.
- **Mediator:** propiedades de bridge y wrapper; incorpora **planificación que determina la traducción en runtime**; tiene suficiente complejidad semántica y autonomía para ser un componente de primera clase de la arquitectura. Los tres permiten acceder a un elemento sin cambiarlo; todos agregan algo de trabajo previo y overhead.
- **SOA:** componentes distribuidos que proveen y consumen servicios, con lenguajes y plataformas distintos, desplegados independientemente, a menudo de **organizaciones distintas**; interfaces que describen lo provisto y lo requerido; SLAs a veces legalmente vinculantes; WSDL/SOAP. Diferencia con microservicios: éstos componen **un** sistema gestionado por **una** organización. Beneficio: servicios genéricos, independientes, heterogéneos; costo: complejidad y overhead de la maquinaria de interoperabilidad.
- **Dynamic discovery:** descubrimiento de proveedores en runtime → binding en runtime; exige publicidad clara de servicios y datos consultables mecánicamente (versiones de interfaz); elegir servicios al arranque o en ejecución por precio o disponibilidad; requiere automatizar registro y des-registro.

21. Modificabilidad

"El cambio sucede" — para agregar o retirar features, corregir defectos, endurecer seguridad, mejorar performance o experiencia, abrazar nueva tecnología... Estudio tras estudio: **la mayor parte del costo del sistema típico ocurre después del primer release**. La modificabilidad busca bajar el **costo y riesgo** de cambiar; planificarla exige responder cuatro preguntas:

1. **¿Qué puede cambiar?** Funciones, plataforma, entorno, cualidades, capacidad... cualquier aspecto.
2. **¿Cuál es la probabilidad del cambio?** No se puede planificar para todo (el sistema no terminaría nunca o costaría demasiado): decidir qué cambios probables se soportarán.
3. **¿Cuándo se hace el cambio y quién lo hace?** En el código fuente, en compilación, en el build, en configuración o en ejecución; por un desarrollador, un administrador o el usuario final (cambiar el protector de pantalla vs. cambiar el motor de BD). Los sistemas que aprenden borran la frontera: el agente del cambio es el propio sistema.
4. **¿Cuál es el costo?** Dos componentes: crear el **mecanismo** de cambio + **usar** el mecanismo. Extremos: no hay mecanismo (cambiar el fuente y revalidar — mecanismo gratis, uso caro) → generadores de aplicaciones (UI builder: mecanismo caro, uso barato) → sistemas que se auto-modifican (uso gratis, mecanismo carísimo).

Fórmula de justificación del mecanismo

Para N cambios similares, conviene crear el mecanismo si: $N \times \text{costo del cambio sin mecanismo} > \text{costo de crear el mecanismo} + N \times \text{costo del cambio con mecanismo}$. Pero N es una predicción (si llegan menos cambios, el mecanismo caro no se amortiza), hay costo de oportunidad, y el tiempo cuenta. Eso sí: apilar cambio sobre cambio sin mecanismo → **deuda técnica**.

Sabores con nombre propio: **escalabilidad** (horizontal/scale out: más unidades — otro servidor al cluster; vertical/scale up: más recursos en la unidad — más memoria; en la nube, la horizontal se llama **elasticidad**); **variabilidad** (soportar la producción planificada de variantes — esencial en líneas de producto); **portabilidad** (mover de plataforma: minimizar y aislar dependencias, correr sobre una "máquina virtual" como la JVM); **independencia de ubicación** (la ubicación de las piezas distribuidas se descubre o cambia en runtime con impacto mínimo).

21.1 El modelo: cohesión, acoplamiento, tamaño, binding time

Parámetro	Definición	Meta
Acoplamiento	Probabilidad de que una modificación a un módulo se propague a otro.	Reducirlo (enemigo de la modificabilidad) — con intermediarios entre módulos.
Cohesión	Cuán relacionadas están las responsabilidades de un módulo ("unidad de propósito"): probabilidad de que un escenario de cambio que afecta una responsabilidad afecte también otras distintas.	Aumentarla — quitar responsabilidades no afectadas por los cambios anticipados.
Tamaño del módulo	A igual todo lo demás, los módulos grandes son más difíciles y caros de cambiar, y más propensos a bugs.	Reducirlo.

Binding time	Momento del ciclo de vida en que se liga el valor/decisión.	Ligar lo más tarde posible... siempre que el mecanismo que lo permite sea costo-efectivo.
---------------------	---	--

21.2 Tácticas

- **Aumentar cohesión: split module** (refactorizar un módulo no cohesivo en submódulos cohesivos — no partir por mitades de líneas de código); **redistribute responsibilities** (juntar responsabilidades similares dispersas; método: hipotetizar cambios probables como escenarios — si tocan sólo una parte del módulo, lo demás quizá sobre; si tocan varios módulos, agrupar lo afectado).
- **Reducir acoplamiento: encapsular; usar intermediario; abstraer servicios comunes** (las tres compartidas con integrabilidad); **restrict dependencies** (restringir con qué módulos interactúa uno: visibilidad y autorización; se ve en capas — usar sólo capas inferiores — y en wrappers — sólo se ve el wrapper).
- **Defer binding** (diferir la ligadura) — "el trabajo de las personas es más caro y propenso a error que el de las computadoras": en **compilación/build**: reemplazo de componentes (build scripts), parametrización en compilación, aspectos; en **despliegue/arranque/inicialización**: configuración, resource files; en **runtime**: discovery, interpretación de parámetros, repositorios compartidos, polimorfismo. Separar la construcción del mecanismo de su uso permite que otro stakeholder (administrador, instalador) haga el cambio: **"externalizar el cambio"**.

21.3 Patrones

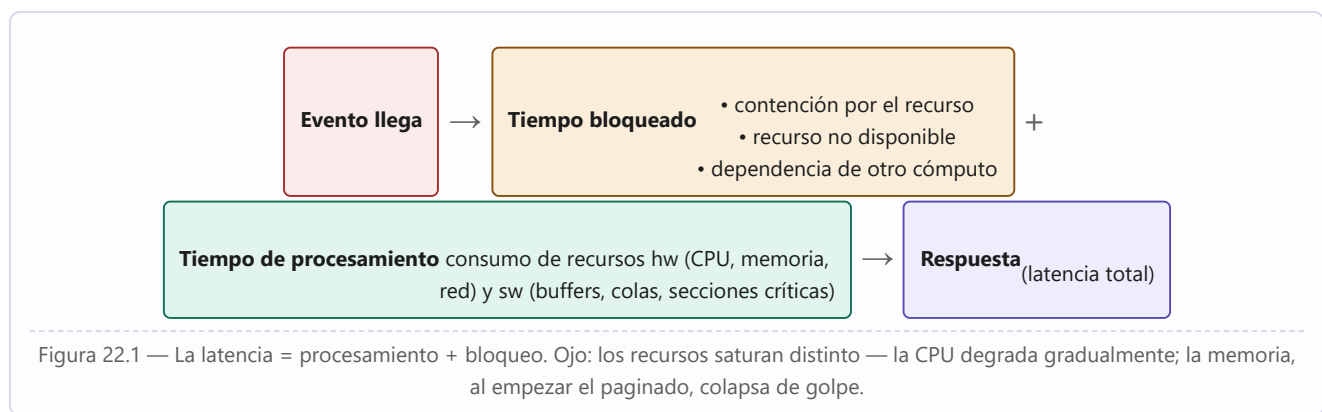
Patrón	Mecánica	Beneficios / tradeoffs
Client-server	Servidor con servicios simultáneos a múltiples clientes distribuidos; descubrimiento + interacción por protocolo acordado; posibles instancias múltiples; si es con estado: identificar cliente, fin de sesión, timeouts.	+ acoplamiento bajo servidor-cliente (conexión dinámica, sin conocimiento a priori), sin acoplamiento entre clientes, escala, evolución independiente, servicios comunes compartidos, UI aislada en el cliente. – performance impredecible por red; seguridad/integridad en redes compartidas.
Plug-in (microkernel)	Núcleo de funcionalidad + variantes especializadas (plug-ins) ligadas por interfaces fijas, en build o después. Ej.: micro-kernel de SO (el núcleo da mecanismos: espacios de direcciones, hilos, IPC; los plug-ins dan la funcionalidad: drivers, gestión de tareas).	+ extensión controlada; dos mercados (núcleo y plug-ins); evolución independiente. – plug-ins de terceros = vulnerabilidades de seguridad y privacidad más fáciles de introducir.
Layers (capas)	Partición completa del software en grupos cohesivos de módulos (capas) con interfaz pública, y relación de uso unidireccional y estrictamente ordenada (idealmente sólo la capa inmediata inferior). Uso de capas no adyacentes = layer bridging .	+ cambiar capas inferiores sin afectar superiores (si la interfaz no cambia); reutilizar capas bajas (portabilidad, a menudo comerciales: SO, comunicaciones); menos interfaces que entender por equipo. – una capa mal diseñada estorba (no da las abstracciones que arriba necesitan); penalidad de performance (atravesar capas); mucho bridging destruye las metas de portabilidad/modificabilidad.

Publish-subscribe	Componentes que se comunican por mensajes asíncronos (eventos/topics): publicadores que no conocen suscriptores, suscriptores que sólo conocen tipos de mensaje; un event bus gestiona suscripciones y despacho (invocación implícita).	+ desacoplamiento total (agregar/quitar suscriptores sin tocar publicadores); cambiar conducta del sistema cambiando qué se publica; logging de eventos → record/playback para reproducir errores. – impacto en latencia y dificultad para razonar performance; menor determinismo (orden de invocación variable); menor testabilidad (un cambio chico en el bus impacta mucho); cifrado de extremo a extremo limitado (publicador y suscriptor no se conocen; requeriría clave compartida por todos).
--------------------------	---	--

22. Performance

"It's about time": la performance es la capacidad del sistema de **cumplir sus requerimientos de tiempo**. Cuando ocurren eventos — interrupciones, mensajes, requests de usuarios o de otros sistemas, eventos de reloj — el sistema debe responder **a tiempo**. Caracterizar los eventos que pueden ocurrir (y cuándo) y la respuesta temporal a ellos es la esencia de la discusión. Órdenes de magnitud a tener presentes: cómputo $\approx$ miles de nanosegundos; acceso a disco $\approx$ decenas de milisegundos; red: cientos de microsegundos dentro del data center, hasta 100 ms intercontinental. Durante décadas la performance dominó la arquitectura (y comprometió a todas las demás cualidades); hoy compete con otras, pero **todo sistema tiene requerimientos de performance**, aunque no estén expresados.

22.1 Modelo: de dónde sale la latencia



Concurrencia y race conditions (recuadro de L. Bass)

Concurrencia = operaciones en paralelo — uno de los conceptos más importantes y menos enseñados. Si dos hilos ejecutan `x = 1; x++;` el resultado puede ser **2 o 3** (condición de carrera por estado compartido). Prevención: **locks** (acceso secuencial forzado) o **particionar el estado** (dos instancias de x). Las race conditions están entre los bugs más difíciles: esporádicos, dependientes del timing (a Bass le tomó **más de un año** reproducir uno en un SO). Aun así, usar concurrencia — con secciones críticas identificadas — es esencial para la performance.

22.2 Tácticas

Dos categorías: **controlar la demanda de recursos** (procesar menos) o **gestionar los recursos** (atender mejor la demanda).

Controlar la demanda	Detalle
Manage work requests	Manage event arrival: un SLA acota la tasa que se acepta ("procesamos X eventos/unidad de tiempo con respuesta Y" — restringe al sistema y al cliente); manage sampling rate: bajar la frecuencia de muestreo (menos cuadros de video, otro códec) sacrificando fidelidad — cuando "suficientemente bueno" alcanza y se prefiere latencia predecible.
Limit event response	Procesar hasta una tasa máxima (encolando el resto) o descartar eventos con una política definida (¿se loguean? ¿se avisa?); disparado por tamaño de cola o utilización del procesador.

Prioritize events	Atender por importancia; los de baja prioridad pueden esperar o ignorarse (alarma de incendio >> alarma de "sala fría").
Reduce computational overhead	Reducir indirección (los intermediarios de la modificabilidad cuestan latencia — el tradeoff clásico; a veces la optimización del código o los brokers con comunicación directa post-descubrimiento dan lo mejor de ambos mundos); co-localizar recursos comunicantes (mismo procesador, mismo componente, mismo rack); limpieza periódica (recalcular hash tables, reboot periódico).
Bound execution times	Limitar iteraciones de algoritmos dependientes de datos; costo: exactitud ("¿es suficientemente bueno?"). Suele emparejarse con manage sampling rate.
Increase efficiency	Mejorar los algoritmos de las áreas críticas ("tune up" — la táctica favorita y única de muchos programadores; es sólo una entre muchas).

Gestionar recursos	Detalle
Increase resources	Procesadores más rápidos o adicionales, más memoria, redes más veloces — a menudo la mejora inmediata más barata.
Introduce concurrency	Procesar en paralelo distintos streams o actividades en distintos hilos; reduce el tiempo bloqueado; luego se eligen políticas de scheduling.
Multiple copies of computations	Réplicas (servicios replicados en microservicios, pools de web servers) reducen la contención de la instancia única; un load balancer asigna el trabajo nuevo.
Multiple copies of data	Replicación de datos (copias completas — carga de consistencia y sincronización) y caching (copias en almacenamiento de distinta velocidad; elegir qué cachear, incluso predecir y precomputar/prefetch).
Bound queue sizes	Limitar arribos encolados; requiere política de overflow. Se empareja con limit event response.
Schedule resources	Toda contención requiere scheduling (procesadores, buffers, redes): entender cada recurso y elegir la estrategia compatible.

Políticas de scheduling

Conceptualmente: **asignación de prioridad + despacho** (el recurso se asigna sólo si está disponible, a veces con apropiación — en cualquier momento, en puntos definidos, o nunca). Políticas comunes: **FIFO** (todos iguales — riesgo de quedar atrapado tras un pedido largo); **prioridad fija**: por importancia semántica, **deadline monotonic** (prioridad mayor a menor deadline — streams de tiempo real) o **rate monotonic** (a menor período mayor prioridad; caso especial mejor soportado por los SO); **prioridad dinámica**: **round-robin** (turnos cíclicos; el *cyclic executive* asigna en intervalos fijos), **earliest-deadline-first** y **least-slack-first** (mayor prioridad al trabajo con menor "holgura" = tiempo de ejecución restante – tiempo al deadline) — estas dos son **óptimas** para monoprocesador apropiativo; **scheduling estático** (offline, sin overhead de runtime).

Ejemplo desarrollado: tácticas de performance en la vía pública (R. Kazman)

Los ingenieros de tráfico usan exactamente estas tácticas: semáforos en las rampas de acceso = **manage event rate**; sirenas y carriles VAO/HOV = **prioritize events**; agregar carriles o rutas paralelas = **maintain multiple copies**; comprarse una Ferrari = **increase resources**; buscar una ruta más corta = **increase efficiency**; ir pegado al de adelante o compartir auto = **reduce computational overhead**. Moraleja: "performance is performance is performance" — son estrategias probadas por siglos de ingeniería.

22.3 Patrones

Patrón	Mecánica	Tradeoffs
Service mesh	En microservicios: un sidecar (proxy) acompaña a cada microservicio y concentra los asuntos transversales — comunicación entre servicios, monitoreo, seguridad, descubrimiento, balanceo, cifrado, observabilidad. Servicio y sidecar se co-despliegan (pods), recortando latencia de red.	+ la lógica de negocio se separa de lo transversal (equipo especialista o compra); comunicación dependiente de contexto (facilita canary/A-B). – más procesos ejecutando (overhead); el sidecar carga funciones que no todo servicio usa.
Load balancer	Intermediario que es el único punto de contacto (una IP) y reparte los mensajes entrantes a un pool de proveedores; implementa <i>schedule resources</i> (round-robin, menor carga, menor cola...).	+ la falla de un servidor es invisible al cliente; latencia baja y predecible; agregar recursos sin que el cliente lo sepa. – el algoritmo debe ser rapidísimo (o él mismo es el problema); potencial cuello de botella / punto único de falla (se lo replica y balancea a él mismo).
Throttling	Empaqueta <i>manage work requests</i> : un "throttler" monitorea los requests y decide si se atienden; mantiene el servicio en su "sweet spot".	+ maneja con gracia las variaciones de demanda. – lógica ultrarrápida obligatoria; si la demanda excede la capacidad de forma crónica, buffers enormes o pérdida de requests; difícil de agregar a sistemas fuertemente acoplados.
Map-reduce	Sort y análisis distribuido y paralelo de datasets enormes, con programación simple: infraestructura especializada + función map (hash a buckets + filtrado; miles de instancias paralelas sin comunicación entre sí) + shuffle + función reduce (el análisis pesado; tantas instancias como buckets; salida mucho menor que la entrada). Comunicación map→reduce sólo por pares <clave, valor>.	+ paralelismo masivo sobre datos no ordenados; la falla de una instancia impacta poco. – injustificado sin datasets grandes; si los datos no se parten en subconjuntos parejos se pierde el paralelismo; múltiples reduces = orquestación compleja. Usuarios: Google, Facebook, Yahoo, Netflix.

23. Safety (inocuidad)

Definición: Safety

Capacidad del sistema de **evitar caer en estados que causen o conduzcan a daño, lesión o pérdida de vidas** de actores de su entorno, y de **detectar y recuperarse** de esos estados inseguros para prevenir o minimizar el daño. "Don't kill anyone" debería estar en la misión de todo arquitecto.

El software daña a través de aquello a lo que está conectado: **actuadores** (puente entre los bits y el mundo físico) o incluso una simple pantalla que informa mal a operadores humanos. Casos citados: la turbina de **Shushenskaya** (2009: keystrokes remotos accidentales activaron una turbina fuera de servicio → decenas de muertos); el **Therac-25** (sobredosis letales de radiación); **Ariane 5**; el satélite soviético de 1983 (reportó 5 misiles estadounidenses en vuelo; el teniente coronel **Stanislav Petrov** decidió ignorar a las computadoras — era un reflejo solar); **Air France 447** (2009: tubos pitot congelados → autopiloto desconectado → los pilotos, confundidos por un aviso de pérdida (stall) que se *apagaba* cuando actuaban bien y *sonaba* cuando actuaban mal — el software desactivaba la alarma con lecturas "inválidas" de baja velocidad —, mantuvieron la nariz arriba durante toda la caída; 228 muertos); **Boeing 737 MAX** (el MCAS empujaba la nariz hacia abajo con datos de **un solo sensor** defectuoso, nunca probado ante falla de sensor, con tripulaciones que ignoraban su existencia; 346 muertos en dos accidentes).

Taxonomía de causas de estados inseguros: omisión (un evento no ocurre); comisión (evento espurio o indeseable en ese estado); timing temprano o tardío; problemas de valores (incorrectos *gruesos* = detectables; *sutiles* = indetectables); omisión/comisión de secuencia (falta un evento en la secuencia o se inserta uno inesperado); fuera de secuencia (llegan todos pero desordenados).

Idea clave — Primero identificar, después diseñar

Arquitectar para safety **comienza identificando las funciones críticas** con técnicas como **FMEA** (failure mode and effects analysis / análisis de riesgos, hazard analysis) y **FTA** (fault tree analysis — enfoque deductivo top-down para hallar las fallas que llevarían al estado inseguro). Recién entonces se diseñan los mecanismos. Y debe repetirse el análisis sobre la arquitectura diseñada: **las propias decisiones de diseño pueden introducir vulnerabilidades nuevas**. Ante el estado inseguro, las respuestas son: recuperarse y continuar (o modo seguro), apagarse (**fail safe**), o pasar a operación manual — siempre reportando/logueando.

23.1 Tácticas

Categoría	Tácticas
Evitar el estado inseguro	Substitution: reemplazar features de software riesgosas por mecanismos de protección — a menudo de hardware — (watchdogs, monitores, interlocks; un dispositivo aparte controla sus propios recursos y no puede ser "muerto de hambre"); beneficioso sólo para funciones relativamente simples. Predictive model: predecir el estado de salud y dar aviso temprano (el crucero adaptativo calcula la tasa de acercamiento y avisa antes de que sea tarde); combinado con condition monitoring.

Detectar el estado inseguro	Timeout (restricciones temporales incumplidas → detecta timing tardío y omisión; pariente de monitor/heartbeat/ping-echo); timestamp (secuencias incorrectas en sistemas distribuidos); condition monitoring (chequear condiciones/aserciones; el monitor, simple y demostrable); sanity checking (razonabilidad de resultados, en interfaces); comparison (comparar salidas de elementos replicados sincronizados; con ≥ 3 réplicas, además de detectar se identifica al culpable; a diferencia del voting puede simplemente apagarse ante discrepancia).
Contener el daño — redundancia	Replication (clones — sólo fallas aleatorias de hardware); functional redundancy (diversidad de diseño contra fallas de modo común; salidas iguales ante entradas iguales); analytic redundancy (diversidad hasta en entradas/salidas; tolera errores de especificación; partición alta-garantía — simple y confiable — vs. alta-performance — compleja, más precisa, menos estable).
Contener el daño — limitar consecuencias	Abort (abortar la operación insegura antes de que dañe — la base del "fail safe"); degradation (mantener lo crítico soltando o reemplazando funcionalidad de forma planificada y segura — navegación por dead reckoning en el túnel); masking (enmascarar la falta votando entre redundantes; el votante, simple y ultra confiable).
Contener el daño — barreras	Firewall (caso particular de limit access: restringir acceso a procesadores, memoria, conexiones); interlock (protección contra secuencias incorrectas: controla TODO el acceso a los componentes protegidos y el orden correcto de los eventos).
Recuperación	Rollback (estado bueno previo + checkpointing/transacciones; luego retry o degradación); repair state (reparar el estado erróneo y seguir — el lane keep assist devuelve el auto al carril; inadecuado para faltas <i>no anticipadas</i>); reconfiguration (remapear la arquitectura lógica sobre los recursos que quedan; si no alcanza, funcionalidad parcial + degradación).

Patrones: los de disponibilidad aplican todos; agregado propio: **sensores redundantes** (si un dato sensado decide entre seguro/inseguro, replicar el sensor y monitorear cada uno con software independiente — exactamente lo que faltó en el 737 MAX).

24. Seguridad (Security)

Definición: Seguridad y la tríada CIA

Medida de la capacidad del sistema de **proteger los datos y la información del acceso no autorizado, sin dejar de brindar acceso a las personas y sistemas autorizados**. Se caracteriza por tres propiedades (CIA): **Confidencialidad** (los datos/servicios protegidos de acceso no autorizado — nadie lee tu declaración de impuestos), **Integridad** (no sujetos a manipulación no autorizada — tu nota no cambió desde que la puso el docente), **Disponibilidad** (el sistema accesible para el uso legítimo — el DoS no impide comprar el libro).

Un **ataque** es una acción contra el sistema con intención de dañar: acceso o modificación no autorizados, o denegación de servicio a usuarios legítimos. Herramienta de análisis: **threat modeling** con **árboles de ataque** (raíz = ataque exitoso; nodos = causas directas; hojas = el estímulo del escenario). Tema hermano: **privacidad** — proteger la **PII** (información personalmente identificable, según NIST: cualquier información que permita distinguir o rastrear la identidad — nombre, SSN, biometría — o vinculada a un individuo — médica, educativa, financiera, laboral); regulada (GDPR); típicamente los testers no deben ver PII real.

24.1 Tácticas

Mnemotecnia física: una instalación segura **limita el acceso** (cercas, checkpoints), **detecta intrusos** (credenciales, cámaras), **disuade** (guardias), **reacciona** (puertas que se traban) y **se recupera** (backups fuera del sitio). De ahí las cuatro categorías:

Categoría	Tácticas
Detectar ataques	Detect intrusion (comparar tráfico/patrones de requests contra firmas de conducta maliciosa: protocolo, payload, direcciones, puertos); detect service denial (comparar el patrón de tráfico entrante contra perfiles históricos de ataques DoS conocidos); verify message integrity (checksums y hashes sobre mensajes, archivos de recursos, de despliegue y de configuración — un cambio ínfimo altera radicalmente el hash); detect message delivery anomalies (detectar man-in-the-middle por tiempos de entrega anómalos o números anormales de conexiones/desconexiones).
Resistir ataques	Identify actors (identificar la fuente de todo input externo: IDs de usuario, códigos, IP, protocolos, puertos); authenticate actors (garantizar que es quien dice ser: contraseñas y one-time passwords, certificados digitales, 2FA, biometría, CAPTCHA; reautenticación periódica); authorize actors (que el autenticado tenga derecho: control de acceso por actor, clase o rol); limit access (restringir puntos de acceso y tipos de tráfico — minimizar la superficie de ataque ; la DMZ entre Internet y la intranet, protegida por un par de firewalls); limit exposure (defensa pasiva: minimizar los datos/servicios comprometibles en un solo punto de acceso); encrypt data (cifrado simétrico o asimétrico; único amparo en enlaces públicos sin control de autorización); separate entities (separación física, VMs, air gap ; datos sensibles aparte de los no sensibles); validate input (limpiar y verificar entradas: filtrado, canonicalización, sanitización — defensa principal contra SQL injection y XSS); change credential settings (forzar el cambio de configuraciones por defecto y el recambio periódico de contraseñas).
Reaccionar	Revoke access (limitar severamente el acceso a recursos sensibles durante un ataque, incluso a usuarios normalmente legítimos); restrict login (bloqueo tras intentos fallidos repetidos — a veces duplicando el tiempo de bloqueo en cada fallo); inform actors (notificar a operadores, personal y sistemas cooperantes).

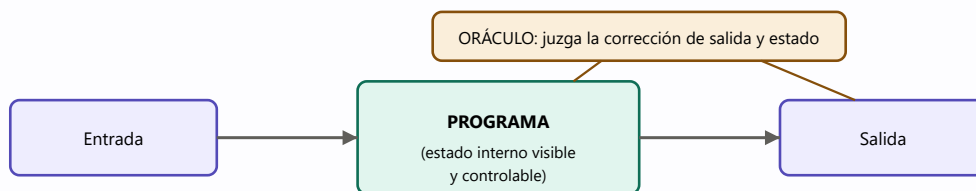
Recuperarse	Las tácticas de disponibilidad (un ataque exitoso es una clase de falla) + audit (registro de acciones y efectos para rastrear e identificar al atacante, procesarlo o mejorar defensas) + nonrepudiation (garantizar que el emisor no pueda negar haber enviado ni el receptor haber recibido — firmas digitales, terceros de confianza).
--------------------	---

Patrones: **intercepting validator** (wrapper entre la fuente y el destino de los mensajes — sobre todo externos — que implementa verify message integrity y puede sumar detección de intrusión/DoS/anomalías: gran cobertura de la categoría "detectar" en un solo paquete, al costo de latencia) e **intrusion prevention system**.

25. Testabilidad

Definición: Testabilidad

Facilidad con la que el software puede ser inducido a **demostrar sus faltas mediante testing** (típicamente basado en ejecución). Formalmente: la probabilidad de que, **asumiendo que tiene al menos un defecto, falle en su próxima ejecución de test**. Un sistema es testeable si "revela" sus faltas fácilmente; además, la arquitectura mejora la testabilidad si facilita **reproducir** el bug y **acotar sus causas** — porque revelarlo no basta: hay que encontrarlo y arreglarlo. El testing consume una porción sustancial del costo (los proyectos suelen gastar ~la mitad del esfuerzo en testing): si la arquitectura puede reducirlo, el pago es grande.



Para testear: poder CONTROLAR entradas y estado, y OBSERVAR salidas y estado (test harness).

Figura 25.1 — Modelo de testing de SAIP: programa + oráculo; el control y la observación del estado interno son el corazón de la testabilidad.

Ejemplo desarrollado: la Simian Army de Netflix

Netflix ($\approx 15\%$ del tráfico global de Internet en 2018) corre en EC2 y usó como parte de su testing un "ejército de simios": el **Chaos Monkey** mataba procesos al azar en el sistema en ejecución para verificar que la falla de un proceso no degradara el servicio; el **Latency Monkey** inducía retardos artificiales para simular degradación; el **Conformity Monkey** apagaba instancias que no seguían las buenas prácticas (p. ej., fuera de un auto-scaling group); el **Doctor Monkey** vigilaba los health checks; el **Janitor Monkey** eliminaba recursos sin uso; el **Security Monkey** terminaba instancias con violaciones de seguridad y verificaba certificados SSL/DRM; el **10-18 Monkey** (L10n-i18n) detectaba problemas de localización/internacionalización. Lección: algunos sistemas son demasiado complejos y adaptativos para testearse por completo — sus conductas son **emergentes**; el logging operacional pasa a ser parte del testing.

25.1 Tácticas

Categoría	Tácticas
-----------	----------

Controlar y observar el estado del sistema <i>(control y observación van de la mano: no tiene sentido controlar lo que no se puede observar)</i>	Specialized interfaces (get/set de variables importantes, método report del estado completo, reset a un estado dado, modo verbose/niveles de logging/instrumentación — separados de las interfaces funcionales para poder quitarlos; ojo en sistemas críticos: lo probado debe ser lo desplegado); record/playback (capturar la información que cruza una interfaz para "reproducir" el estado que causó la falta); localize state storage (estado en un solo lugar — idealmente una máquina de estados — para arrancar el sistema en cualquier estado y externalizarlo); abstract data sources (abstraer las interfaces de datos para apuntar a BDs de prueba sin tocar el código funcional); sandbox (aislar el sistema del mundo real para experimentar sin consecuencias — o con rollback; forma común: virtualizar recursos , p. ej. un reloj virtual para probar el pasaje al horario de verano sin esperar; stubs, mocks y dependency injection son virtualizaciones simples y efectivas); executable assertions (aserciones codificadas a mano en lugares deseados — pre/postcondiciones e invariantes de clase — que marcan cuándo y dónde el programa está en estado defectuoso; equivalen a incrustar el oráculo en el código).
Limitar la complejidad	Limit structural complexity (evitar/resolver dependencias cíclicas, encapsular dependencias del entorno, reducir acoplamiento; en OO: limitar profundidad de herencia, hijos por clase, polimorfismo y llamadas dinámicas; métrica correlacionada: la response of a class — métodos propios + métodos ajenos invocados; alta cohesión, bajo acoplamiento y separación de incumbencias ayudan — son tácticas de modificabilidad; el patrón de capas permite testear de abajo hacia arriba con confianza); limit nondeterminism (el no determinismo es la forma perniciosa de la complejidad conductual: cazar las fuentes — paralelismo sin restricciones — y eliminarlas en lo posible; para lo inevitable, record/playback).

Técnicas de reemplazo con versiones "testeables": component replacement (en el build), macros de preprocesador (expanden a código de reporte de estado), aspectos (manejan el reporte como cross-cutting concern).

Patrones: **dependency injection** (cliente/servicio/interfaz/injector; **inversión de control:** las dependencias se inyectan desde fuera, el cliente se escribe sin saber cómo será testeado; se inyectan instancias de test que gestionan y monitorean el estado — base de los frameworks modernos de testing; tradeoff: performance runtime menos predecible); **strategy** (la clase elige el algoritmo en runtime; se sustituyen versiones de test con salidas extra y sanity checks); **intercepting filter** (filtros en el flujo de entrada/salida).

26. Usabilidad

Definición: Usabilidad

Cuán fácil es para el usuario **lograr la tarea deseada** y qué **soporte** le brinda el sistema. De las formas más baratas de mejorar la calidad percibida y la satisfacción. Cinco áreas: **aprender** las características del sistema; **usarlo eficientemente**; **minimizar el impacto de los errores** del usuario; **adaptar** el sistema a las necesidades del usuario; **aumentar la confianza y satisfacción** (feedback de progreso).

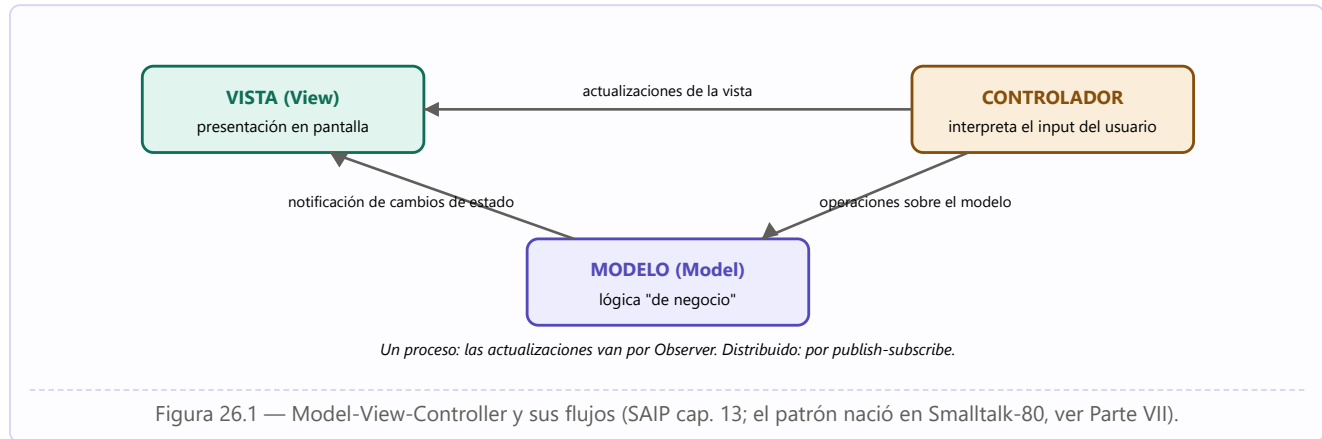
Concepto organizador: **iniciativa** — quién toma la acción: el usuario (*user initiative*), el sistema (*system initiative*) o mixta (cancelar es iniciativa del usuario; la barra de progreso durante la cancelación, del sistema). Conexión fuerte con la **modificabilidad**: el diseño de UI es iterativo (diseñar → probar con usuarios → corregir), así que la arquitectura debe hacer indolora la iteración — de ahí los patrones de separación (MVC).

26.1 Tácticas

Soportar la iniciativa del usuario	Detalle
Cancel	Un listener constante (no bloqueado por la actividad cancelada); terminar la actividad; liberar sus recursos; informar a los colaboradores para que también actúen.
Undo	Mantener suficiente información de estado (snapshots/checkpoints u operaciones reversibles). No todo es reversible (cambiar todas las "a" por "b" no se invierte cambiando las "b" por "a"; no se puede "des-enviar" un paquete ni "des-disparar" un misil). Sabores: single undo, multiple undo (hasta un límite o hasta la apertura de la app).
Pause/resume	Para operaciones largas (descargas): liberar recursos temporalmente y reasignarlos.
Aggregate	Agrupar objetos de operaciones repetitivas y aplicar la operación al grupo (seleccionar todos los objetos de una diapositiva y ponerles fuente de 14 pt) — evita el tedio y los errores.

Soportar la iniciativa del sistema	Detalle — el sistema mantiene un modelo
Maintain task model	Modelo de la tarea en curso para dar contexto y asistir (autocompletado predictivo, corrección ortográfica).
Maintain user model	Modelo del conocimiento y conducta del usuario (tiempo de respuesta esperado; apps de idiomas que refuerzan donde el usuario se equivoca); caso especial: la customización de la UI por el propio usuario.
Maintain system model	Modelo explícito de sí mismo para predecir su conducta y dar feedback apropiado (la barra de progreso que estima el tiempo restante).

26.2 Patrones



- **MVC** y variantes (**MVP**, **MVVM**, **MVA**): separar el modelo de sus vistas. Beneficios: cambios de layout sin tocar modelo ni controlador; trabajo y testing en paralelo de las tres partes; un modelo con muchas vistas y viceversa. Tradeoffs: engorroso en UIs complejas (información desparramada); complejidad inicial injustificada en UIs simples; algo de latencia extra.
- **Observer**: un *subject* con uno o más *observers* registrados; al cambiar el estado del sujeto, se notifica a los observadores (así se implementa MVC en un proceso). Tradeoffs: overkill sin múltiples vistas; si los observadores no se des-registran → **fuga de memoria** y trabajo desperdiciado; "impedance mismatch" (el sensor reporta centésimas de grado y la vista muestra grados → procesamiento repetido inútil).
- **Memento**: implementa **undo** — el *originator* procesa eventos que cambian su estado; el *caretaker*, antes de provocar un cambio, le pide un *memento* (snapshot opaco del estado) y puede restaurarlo devolviéndoselo — sin saber nada de cómo se gestiona el estado (preserva la abstracción). Tradeoff: memoria arbitrariamente grande (cortar y pegar secciones enormes y deshacerlo todo enlentece al procesador de texto); en algunos lenguajes cuesta imponer la opacidad.

27. Trabajar con otros atributos de calidad

Los diez QAs anteriores son la "lista A", pero hay más. Categorías adicionales:

- **QAs de la arquitectura misma** (no del sistema ni del proyecto): **buildability** (cuán bien se presta la arquitectura a un desarrollo rápido y eficiente, medida en costo/tiempo para volverla producto); **integridad conceptual** (consistencia del diseño: *lo mismo se hace de la misma manera en todas partes* — pocas formas de comunicar, reportar errores, loguear, sanitizar; "menos es más"); **marketability** (la percepción de la arquitectura puede importar tanto como sus cualidades reales: organizaciones que se sienten obligadas a "ser cloud/microservicios" sea o no la elección técnica correcta).
- **Development distributability**: diseñar para el desarrollo distribuido globalmente — minimizar la coordinación necesaria entre equipos (bajo acoplamiento de los subsistemas Y del modelo de datos); la estructura arquitectónica y la social/de negocio del proyecto deben estar razonablemente alineadas.
- **QAs del sistema físico** (peso, tamaño, consumo, calor...): el software los afecta (software ineficiente → más memoria/procesador/batería → más peso y costo) y el hardware limita al software (no corres el modelo meteorológico global en la laptop del abuelo; la seguridad física supera a la del software). Las técnicas de escenarios sirven igual.

27.1 Listas estándar de QAs: usarlas con criterio

ISO/IEC 25010 divide en "quality in use" y "product quality"; sus ocho características de producto: functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, portability — con casi **cinco docenas de sub-características** ("modifiability" y "testability" dentro de "maintainability"; "availability" dentro de "reliability"; y **scalability ni aparece**). Veredicto del libro: útiles como **checklist** para no olvidar necesidades (y como base de tu checklist propio), pero: (1) ninguna lista es completa (siempre habrá un concern imprevisto — el ejemplo de la **"Iowability"**: una arquitectura diseñada para retener talento en el Midwest con tecnología de punta y libertad creativa); (2) generan más controversia que comprensión (discutir si portability es un tipo de modifiability es esfuerzo mal gastado); (3) pretenden ser taxonomías pero los QAs son escurridizos (el DoS pertenece a cuatro). **Los nombres de QAs, por sí solos, son casi inútiles: son invitaciones a empezar una conversación; la especificación real son los escenarios.**

27.2 Incorporar un QA nuevo ("X-ability")

Receta para un QA sin cuerpo de conocimiento propio: (1) **capturar escenarios** con los stakeholders cuyas incumbencias lo motivan (refinar el QA en sub-atributos; construir escenarios concretos; generalizarlos); (2) **modelar el QA** (conjunto de parámetros a los que es sensible + características arquitectónicas que influyen en esos parámetros; ej.: el modelo de colas de performance tiene exactamente 7 parámetros: tasa de arribos, disciplina de cola, algoritmo de scheduling, tiempo de servicio, topología, ancho de banda, algoritmo de ruteo — y cada uno es afectable por decisiones arquitectónicas); (3) **ensamblar design approaches**: por cada parámetro, enumerar los mecanismos que lo afectan (revisar los mecanismos conocidos, buscar diseños que hayan lidiado con el QA, literatura, expertos). El resultado es una lista finita y razonablemente pequeña de mecanismos: el problema de diseño se vuelve tratable.

PARTE V

Interfaces e infraestructura moderna

SAIP 4.^a ed., capítulos 15–18: el diseño y documentación de interfaces, la virtualización (VMs, contenedores, pods, serverless), la nube y la computación distribuida, y los sistemas móviles.

-
- 28** Interfaces de software
 - 29** Virtualización: VMs, contenedores, pods, serverless
 - 30** La nube y la computación distribuida
 - 31** Sistemas móviles
-

28. Interfaces de software

El costo de una interfaz mal acordada

"La NASA perdió su Mars Climate Orbiter de 125 millones de dólares porque los ingenieros no convirtieron de medidas inglesas a métricas al intercambiar datos vitales antes del lanzamiento... El equipo de navegación usaba milímetros y metros; la empresa que construyó la nave entregó los datos cruciales de aceleración en pulgadas, pies y libras. La nave se perdió, en cierto sentido, en la traducción." — Los Angeles Times, 1/10/1999.

Definición: Interfaz

Una interfaz es una **frontera a través de la cual los elementos se encuentran e interactúan, se comunican y se coordinan**; controla el acceso a los internos del elemento. Los **actores** de un elemento son los otros elementos, usuarios o sistemas con los que interactúa; su colección es el **entorno** del elemento. "Interactuar" es **cualquier cosa que un elemento haga que pueda impactar el procesamiento de otro** — incluso hechos indirectos (que usar el recurso X de A deje a B en cierto estado es parte de la interfaz entre A y su entorno).

Tres principios: (1) **todos los elementos tienen interfaz** (todos interactúan con actores — si no, ¿para qué existen?); (2) **las interfaces son de dos vías**: no sólo lo que el elemento *provee*, también lo que *requiere* de su entorno para funcionar; (3) un elemento puede interactuar con **más de un actor por la misma interfaz** (límites de conexiones simultáneas). Además, un elemento puede tener **múltiples interfaces** — separación de incumbencias, distintos derechos de acceso (lectura/escritura), niveles (pública anónima de sólo lectura vs. privada autenticada), interfaces separadas de debugging, monitoreo de performance o administración.

Los **recursos** de una interfaz tienen **sintaxis** (la firma: nombre, argumentos y tipos) y **semántica** (qué produce invocarlo: asignación de valores, supuestos sobre los valores, cambios de estado — incluyendo condiciones excepcionales —, eventos o mensajes emitidos, efectos sobre otros recursos futuros, resultados humanamente observables, atomicidad). Los recursos de las interfaces provistas se componen de **operaciones** (transfieren control y datos, devuelven resultado, pueden fallar), **eventos** (normalmente asíncronos: llegada de mensajes, arribos de stream; los elementos activos producen eventos salientes para notificar) y **propiedades** (metadatos: derechos de acceso, unidades de medida, supuestos de formato).

28.1 Evolución de interfaces

Todo software evoluciona; la interfaz es un **contrato** entre elemento y actores, y se cambia con cuidado. Tres técnicas: **deprecation** (retirar la interfaz — avisar con muchísima anticipación; en la práctica muchos actores lo descubren recién cuando desaparece; técnica: un código de error "se deprecia el (fecha)"); **versioning** (mantener la vieja y agregar la nueva; el actor especifica cuál usa); **extension** (dejar la original intacta y agregar recursos; si la extensión introduce incompatibilidades — ej. el número de departamento pasa de ir embebido en la dirección a parámetro separado — se agrega una **interfaz interna** y un **mediador** que traduce entre la interfaz externa original y la interna).

28.2 Diseñar una interfaz

- **Principio de la menor sorpresa:** comportarse consistentemente con las expectativas del actor (los nombres importan).
- **Principio de interfaces pequeñas:** si dos elementos deben interactuar, que intercambien **la menor información posible**.
- **Principio del acceso uniforme:** no filtrar detalles de implementación; el recurso se accede igual sin importar cómo esté implementado (el actor no debería notar si el valor viene de una cache, de un cómputo o de una consulta externa fresca).
- **Don't repeat yourself:** primitivas componibles, no muchas maneras redundantes de lograr lo mismo.
- **Consistencia:** convenciones de nombres, orden de parámetros y manejo de errores en toda la arquitectura; seguir los idiomas de la plataforma. Minimiza errores por malentendidos.

Un patrón para acotar y mediar el acceso: el **gateway** (de mensajes), que traduce los requests de los actores a requests sobre los recursos del elemento — útil cuando la granularidad no coincide, cuando hay que restringir el acceso a subconjuntos, o cuando los detalles (número, protocolo, tipo, ubicación) cambian con el tiempo y el gateway da una interfaz estable. La interacción exitosa exige acuerdo en cuatro cosas: **alcance de la interfaz, estilo de interacción, representación y estructura de los datos intercambiados, y manejo de errores**.

28.3 Estilos de interacción: RPC/gRPC y REST

Estilo	Descripción
RPC / gRPC	Modelado sobre la llamada a procedimiento de los lenguajes imperativos, pero el procedimiento vive en otro nodo: la llamada se traduce en un mensaje, se invoca remoto, y el resultado vuelve como mensaje. Data de los 80 (versiones tempranas síncronas y con parámetros de texto). La versión moderna, gRPC : parámetros binarios, asíncrono , con autenticación, streaming bidireccional, control de flujo, bindings bloqueantes o no, cancelación y timeouts; transporta sobre HTTP 2.0 .
REST	Protocolo para servicios web surgido del protocolo original de la WWW. Seis restricciones: (1) interfaz uniforme (todas las interacciones con la misma forma — típicamente HTTP; recursos identificados por URLs; menor sorpresa); (2) cliente-servidor ; (3) stateless (el cliente no asume que el servidor retuvo nada de su último request; la autorización viaja como token en cada request); (4) cacheable ; (5) arquitectura en tiers (el "servidor" puede partirse en elementos independientes desplegados por separado — lógica y BD); (6) code-on-demand (opcional: el servidor envía código ejecutable al cliente, ej. JavaScript). Comandos HTTP → CRUD: POST=create, GET=read, PUT=update, DELETE=delete (+PATCH).

28.4 Representación de los datos intercambiados

Toda interfaz abstrae la representación interna (tipos del lenguaje) en una externa apta para viajar — la conversión se llama **serialización / marshaling / traducción**. Criterios de elección: **expresividad** (¿serializa estructuras arbitrarias? ¿texto multilenguaje?), **interoperabilidad** (¿los actores saben parsearlo? ¿es estándar?), **performance** (uso del ancho de banda; costo de parsing; tiempo de preparación; costo monetario), **acoplamiento implícito** (supuestos compartidos que pueden causar pérdida de datos al decodificar), **transparencia** (¿se puede interceptar y leer? — arma de doble filo: ayuda a debuggear... y a espiar).

Formato	Características	Tradeoff
---------	-----------------	----------

XML (W3C, 1998)	Anotaciones textuales (tags con atributos); es un meta-lenguaje : se define un lenguaje propio mediante un schema (a su vez XML). Usos: SOAP, XHTML, SVG, DOCX, WSDL, archivos de configuración.	+ documentos validables contra el schema (elimina chequeos de error en el código). – parsing y validación caros (leer completo, múltiples pasadas); verboso: performance y ancho de banda pueden ser inaceptables.
JSON (estandarizado en 2013)	Pares nombre/valor anidados y arreglos; nació de JavaScript pero es independiente; tiene schema propio; tipos cercanos a los de los lenguajes modernos.	+ mucho menos verboso que XML (los nombres aparecen una vez); parseable mientras se lee ; serialización eficiente. Caso de uso original: browser↔server.
Protocol Buffers (Google, open source 2008)	Formato binario con schema (archivo <code>.proto</code> compilado por un compilador específico del lenguaje que genera los procedimientos de serialización); elementos requeridos, opcionales y anidados; heredero espiritual de ASN.1 (años 80).	+ extremadamente compacto; memoria y red muy eficientes; la especificación sirve de documentación consultable. Uso principal: junto a gRPC. – no legible sin herramientas.

28.5 Manejo de errores y documentación

El mundo real está lejos del caso nominal: la interfaz debe definir qué pasa con parámetros inválidos, memoria insuficiente, operaciones que no retornan, sensores que responden basura. Estrategias para que el actor sepa si la interacción fue exitosa: **excepciones**, **códigos de estado**, **propiedades** con el resultado de la última operación, **eventos de error** (timeout) o **log de errores**. Fuentes comunes de error a manejar con gracia: información incorrecta/ilegal enviada a la interfaz; el elemento en el **estado equivocado** para atender el pedido (por acción u omisión previa — invocar antes de que termine la inicialización); error de hardware/software (procesador, red, memoria); configuración incorrecta. Indicar la **fuentes** del error orienta la recuperación: transitorio + idempotente → esperar y reintentar; input inválido → corregir y reenviar; dependencia faltante → reinstalar; bug → caso de test nuevo.

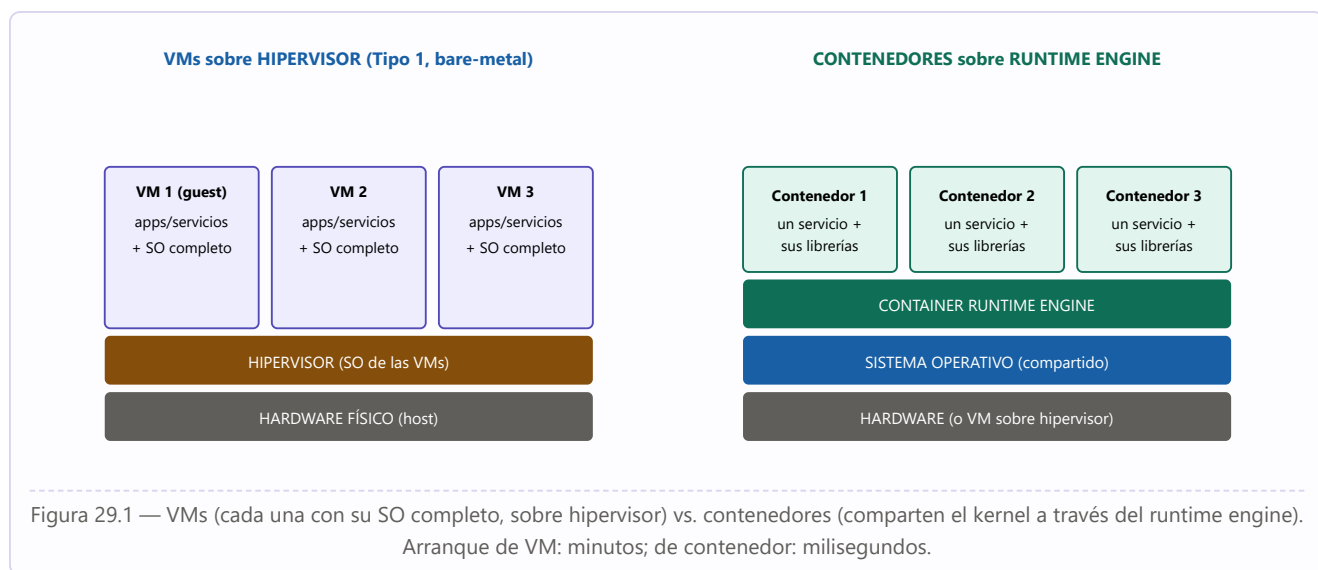
Idea clave — Interfaz ≠ documentación de la interfaz

Todo lo observable de un elemento es parte de su interfaz (hasta cuánto tarda una operación). La **documentación** es el subconjunto que decidimos **prometer**: el contrato del que los actores pueden depender. Escribir todo aspecto de toda interacción no es práctico ni deseable: exponer **sólo lo que los actores necesitan saber**. Un desarrollador puede observar propiedades no documentadas, pero las usa **a su propio riesgo**. La **ley de Hyrum**: "con suficientes usuarios de una interfaz, no importa qué prometas en el contrato: **todas las conductas observables de tu sistema serán dependidas por alguien**". Stakeholders de la documentación: el desarrollador del elemento, el mantenedor, el desarrollador que la usa, el integrador/tester, el analista (SLAs), el arquitecto que busca activos para reutilizar.

29. Virtualización

Problema de los 60: compartir recursos (memoria, disco, E/S) de una máquina física — que costaba millones — entre aplicaciones independientes que usaban ~10% de los recursos. Las máquinas virtuales y luego los contenedores nacieron para **aislar** una aplicación de otra **compartiendo** recursos: el aislamiento permite escribir la aplicación como si la computadora fuera propia; el compartir permite que muchas corran a la vez. Recursos a compartir con aislamiento: **CPU** (compartida por el *scheduler* de hilos: nadie toma el procesador sin pasar por él), **memoria** (memoria virtual paginada con aislamiento de espacios de direcciones por hardware), **disco** (controladores que secuencian los streams + etiquetas de usuario/grupo para visibilidad y acceso), **red** (identificación de mensajes por dirección IP de cada VM/contenedor + puertos por servicio).

29.1 Máquinas virtuales



El **hipervisor** es el "sistema operativo de las VMs". **Tipo 1 / bare-metal:** corre directo sobre el hardware (data centers y nube). **Tipo 2 / hosted:** corre como servicio sobre un SO anfitrión (desktops: correr Linux sobre Windows, replicar el entorno de producción en la máquina de desarrollo). Funciones: (1) **interceptar y ejecutar** en nombre de la VM el código que se comunica con el exterior (disco/red virtualizados), etiquetando los pedidos para rutear las interrupciones asíncronas de respuesta a la VM correcta; (2) **gestionar las VMs:** crearlas/destruirlas (a pedido de la infraestructura de nube), monitorear salud y uso, defender el perímetro de seguridad, y **hacer cumplir los límites de recursos** de cada VM (CPU, memoria, ancho de banda de disco y red). La VM bootea como una máquina física (boot loader desde el disco que conecta el hipervisor). Requisito: las VMs usan el **mismo instruction set** que la CPU física — para otra arquitectura se necesita un **emulador** (ej. QEMU simula un PC completo, BIOS, x86, placas). Overhead de virtualización: depende del perfil de E/S; Microsoft reporta ~10% en Hyper-V. Dos implicaciones para el arquitecto: costo de performance, y **separación de incumbencias** (los recursos de ejecución como commodity — el aprovisionamiento se difiere a otra persona u organización).

Imágenes de VM: los bits del disco de arranque (SO + servicios + boot loader). Tres formas de crearlas: snapshot de una máquina andando; partir de una imagen existente y agregar software; desde cero con los medios de instalación. Riesgos de imágenes ajenas: no controlás versiones; puede haber vulnerabilidades, mala

configuración o directamente malware. Problemas: son **enormes** (transferirlas es lento), van con todas sus dependencias. Práctica habitual: imágenes mínimas (SO + esenciales) y agregar los servicios tras el boot (**configuración**) para evitar una imagen única por cada versión de cada servicio.

29.2 Contenedores, pods y serverless

Una imagen de VM de 8 GB tarda >3 minutos reales en moverse por una red de 1 Gbps (que rinde ~35%), más el boot del SO. Los **contenedores** conservan las ventajas de la virtualización recortando transferencia y arranque: comparten el SO vía el **container runtime engine** (que actúa como "SO virtualizado") — no hay que transferir el SO — y las imágenes se construyen en **capas**: al actualizar la capa superior (ej. subir la versión de PHP en un stack LAMP construido capa por capa: Linux → Apache → MySQL → PHP), el gestor **mueve sólo la capa cambiada**. Cargar una nueva versión: de minutos (VM) a **milisegundos** (contenedor). Los scripts de construcción de imágenes se versionan y tratan **como código**. La interfaz contenedor ↔ engine está estandarizada (**Open Container Initiative**): un contenedor creado con Docker corre en containerd o Mesos → **portabilidad** dev→producción.

VM	Contenedor
Virtualiza el hardware ; corre cualquier SO y casi cualquier programa (legacy); puede alojar múltiples servicios acoplados que comparten datos; el hipervisor arranca/monitorea/reinicia el SO .	Virtualiza el SO (limitado a Linux/Windows/iOS); el engine arranca/monitorea/reinicia el servicio ; típicamente un solo servicio por contenedor (imagen chica, arranque rápido); si el programa termina, el contenedor muere.
Persisten más allá de los servicios que corren dentro.	No persisten ; restricciones de puertos.

Pods (Kubernetes — software de orquestación para desplegar, gestionar y escalar contenedores): grupo de contenedores relacionados que **comparten IP y espacio de puertos**, pueden comunicarse por IPC (semáforos, memoria compartida), comparten volúmenes efímeros y **tienen el mismo ciclo de vida** (se asignan y desasignan juntos). Propósito: abaratar la comunicación entre contenedores íntimamente relacionados (ej.: el service mesh se empaqueta como pod). Jerarquía: nodos ⊃ pods ⊃ contenedores.

Arquitectura serverless (FaaS): como cargar un contenedor tarda milisegundos, no hace falta dejarlo corriendo — se **asigna un contenedor nuevo por cada request** y al terminar se desasigna. Hay servidores, claro, pero su asignación es dinámica y responsabilidad de la infraestructura del proveedor (*function-as-a-service*). Consecuencia: los contenedores efímeros **no pueden mantener estado** — todo estado de coordinación va en servicios de infraestructura del proveedor o como parámetros. Limitaciones prácticas: selección restringida de imágenes base (lenguajes/librerías), **cold start** de varios segundos la primera vez (luego el cache del nodo responde casi instantáneo), y **tiempo máximo de ejecución** por request (o el proveedor lo termina). Hay diseñadores que gastan mucha energía en esquivar esto (pre-arrancar servicios, requests dummy para mantener el cache, encadenar requests).

30. La nube y la computación distribuida

Leslie Lamport

"Un sistema distribuido es aquel en el que la falla de una computadora cuya existencia ignorabas puede dejar tu propia computadora inutilizable."

Cloud computing = disponibilidad de recursos **a demanda**: los recursos son de otro (que se ocupa del capital, mantenimiento y backups), accesibles por Internet, se paga sólo lo usado, el servicio es **elástico** (crece y se achica con la necesidad) y **auto-aprovisionado** (creas la cuenta y usás). Modelos: nube **pública** (del proveedor, para quien pague), **privada** (de la organización, por control/seguridad/costo) e **híbrida** (cargas repartidas; migraciones; datos legalmente restringidos). Técnicamente, para el arquitecto es casi lo mismo.

Un data center típico: **~100.000 dispositivos físicos** (el límite: la energía que se puede traer y el calor que se puede sacar); racks de 25+ computadoras conectados por switches de alta velocidad. El proveedor organiza los data centers en **regiones** (cercanía física a los usuarios → menor delay; cumplimiento regulatorio, ej. GDPR y fronteras de datos) y dentro de ellas en **availability zones** — grupos de data centers con energía y conectividad independientes tales que **la probabilidad de que dos zonas fallen a la vez es bajísima**. Elegir región y zona es una decisión de diseño (disponibilidad, continuidad de negocio). Acceso por dos puertas: el **management gateway** (pide una VM con región + tipo de instancia + ID de imagen; el gateway encuentra un hipervisor con capacidad, crea la VM y devuelve **IP + hostname en el DNS**; también factura, monitorea y destruye; accesible por API — scriptable — o por la web del proveedor) y el **message gateway** (los mensajes de negocio).

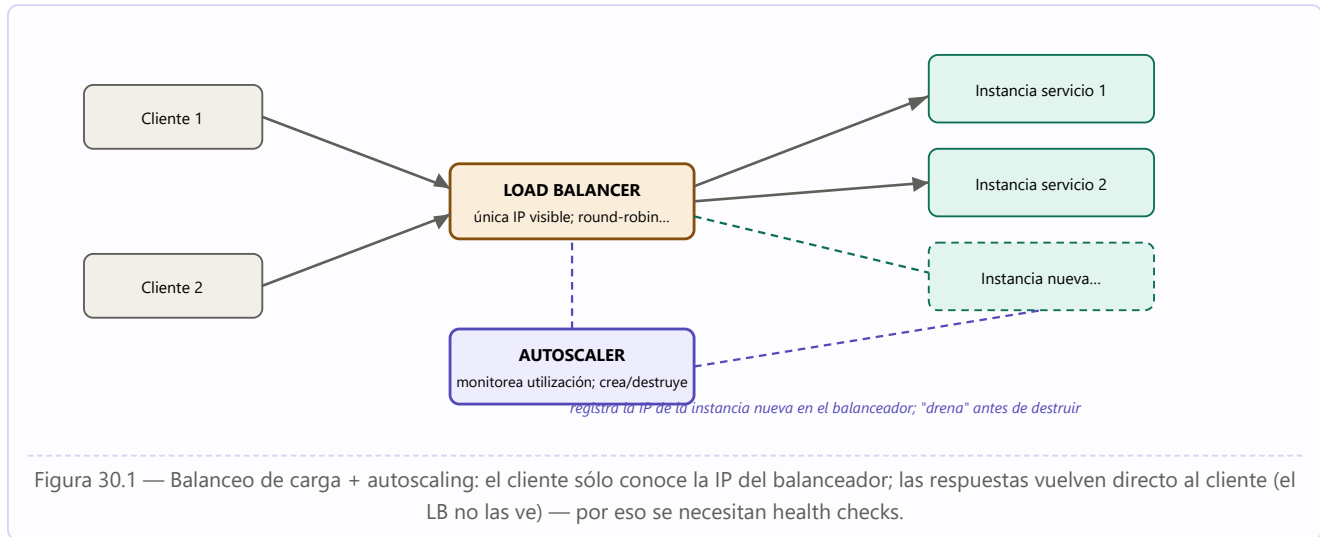
30.1 La falla como estado normal

Cuidado — Los números de la falla

Amazon reporta que en un data center de **64.000 computadoras** (con dos discos giratorios cada una) fallan por día **≈5 computadoras y 17 discos**. Google reporta cifras similares. También fallan switches, el centro puede sobrecalentarse, o un desastre natural puede tumbarlo. Si la disponibilidad te importa, **asumí que la máquina física donde corre tu VM va a fallar** y diseñá para ello.

- **Timeouts**: detectan la falla, pero **no distinguen** computadora caída / red rota / respuesta lenta, ni dicen **dónde** ocurrió; con cadenas de servicios, latencias individuales "casi normales" pueden sumar un falso positivo. Hay costo de recuperación (arrancar otra VM tarda minutos; re-establecer sesión afecta usabilidad) → parametrizar: **intervalo de timeout + número de respuestas perdidas en una ventana** (ej.: timeout de 200 ms, recuperación tras 3 pérdidas en 1 segundo); agresivo dentro del data center, laxo sobre WAN/celular/satélite.
- **Long tail latency**: histograma real de 1.000 pedidos "launch instance" a AWS: pico en 22 s, media 28 s, mediana 23 s... y el **percentil 95 en 57 s** — hasta 5% de tus requests puede tardar **5–10 veces el promedio**, por congestión o falla en el camino (colas de servidores, scheduling del hipervisor — fuera de tu control). Mitigaciones: **hedged requests** (pedir de más y cancelar los sobrantes: para lanzar 10 instancias, pedir 11 y matar la que no llegó) y **alternative requests** (pedir 10; cuando completaron 8, pedir 2 más; cancelar las 2 que sobren).

30.2 Escalado, balanceo y estado



- **Escalado: vertical**/scale up (tipo de instancia más grande — la táctica *increase resources*; simple, pero hay techo) y **horizontal**/scale out (múltiples copias + load balancer — *maintain multiple copies*).
- **Load balancer**: debe ser eficientísimo (está en el camino de cada mensaje); algoritmos: round-robin y variantes por carga real. El cliente ve la IP del LB (resiliencia al cambio — un intermediario); si el LB se satura, se balancean los balanceadores (**global load balancing**, jerarquías). **Health checks**: como las respuestas van directo del servicio al cliente, el LB no sabe si la instancia procesa; la chequea periódicamente (ping, conexión TCP o mensaje de prueba); instancias que no responden pasan a la lista de no saludables (con reintentos: una cola desbordada puede recuperarse); el pool se dimensiona para tolerar fallas simultáneas. Los **clientes deben reenviar** requests sin respuesta y los **servicios deben tolerar requests duplicados**.
- **Estado**: servicios **stateful** (la historia en la instancia) vs. **stateless** (historia en el cliente o en una BD externa). **Práctica común: stateless** — la instancia que falla no pierde historia y cualquier instancia nueva responde igual. Si hace falta pegar una serie de mensajes a la misma instancia: sesión directa post-balanceador o **sticky sessions** — sólo en circunstancias especiales (riesgo de falla y de sobrecarga de la instancia pegajosa).

30.3 Tiempo, coordinación y autoscaling

- **Tiempo**: los relojes de hardware derivan (≈ 1 segundo cada 12 días); **NTP** sincroniza con exactitud ≈ 1 ms en LAN y ≈ 10 ms en redes públicas (100+ ms con congestión); los proveedores usan relojes atómicos como referencia. Diseño: **no requerir sincronización de relojes para la corrección**; casi siempre importa el **orden** de los eventos más que la hora (mercados, subastas) → mecanismos tipo **vector clocks** (contadores que trazan acciones, no relojes) para decidir "qué pasó antes".
- **Coordinación de datos**: un lock compartido entre servicios de máquinas distintas sufre dos males — el **two-phase commit** exige mensajes que pueden perderse, y el poseedor del lock puede morir con él. Solución: algoritmos de **consenso distribuido** (el **Paxos** de Lamport), notoriamente difíciles de diseñar e implementar bien. **Regla: no lo hagas vos mismo** — usar Zookeeper, Consul o etcd.

- **Autoscaling:** crea instancias nuevas cuando hacen falta y libera las sobrantes, coordinado con el load balancer e **invisible para los clientes**. Reglas configurables: imagen a lanzar y parámetros; umbral de CPU (sobre un intervalo — nunca sobre valores instantáneos, que tienen picos, y porque arrancar una VM tarda minutos: "crear una VM si CPU > 80% por 5 minutos"); umbrales de red; mínimo y máximo de instancias; agenda horaria (asignar más VMs antes del horario laboral, según datos históricos). Para quitar una instancia: avisar al LB, dejarla terminar lo que procesa (**draining** — tu servicio debe implementar la interfaz para recibir la orden), destruirla. Con contenedores la decisión es **de dos niveles**: ¿hace falta otro contenedor/pod? y ¿cabe en un runtime existente o hay que crear otro (y quizás otra VM)? — el software que escala contenedores es independiente del que escala VMs (portabilidad entre proveedores).

31. Sistemas móviles

De los smartphones y tablets a autos, trenes, aviones, barcos y drones. Difieren de los sistemas fijos en **cinco características**: energía, conectividad, sensores/actuadores, recursos y ciclo de vida.

Dimensión	Incumbencias del arquitecto
Energía	Baterías finitas (o generadores a combustible). (1) Monitorear la fuente : la batería inteligente con su BMS se consulta (% restante); un <i>battery manager</i> del kernel informa al usuario, activa el modo ahorro, avisa a las aplicaciones registradas del apagado inminente y estima el consumo por aplicación (las baterías envejecen: cambia su capacidad máxima y su corriente sostenida). (2) Throttle de energía : apagar/degradar consumidores — brillo y refresco de pantalla, núcleos activos, frecuencia de reloj, tasa de lectura de sensores (GPS por minuto en vez de por segundo; una sola fuente de ubicación). (3) Tolerar la pérdida de energía : requisitos tipo "a los 30 s de restaurada la energía, el sistema opera en modo nominal": hardware que no se daña al cortarse la luz, runtime que puede morir en cualquier momento sin corromper binarios/configuración/datos y con estado consistente al reiniciar, estrategia para los datos que llegan mientras la aplicación está inoperativa.
Conectividad	Comunicación inalámbrica por rangos: NFC (≤ 4 cm — pagos); Bluetooth/Zigbee (≤ 10 m, IEEE 802.15); Wi-Fi (≤ 100 m, 802.11); WiMAX (varios km, 802.16); celular/satélite (más). Decisiones: soportar sólo las interfaces estrictamente necesarias (energía, calor, espacio); transiciones sin costura entre protocolos durante una sesión (el video que pasa de Wi-Fi a celular en el túnel); elección dinámica del protocolo (costo, ancho de banda, consumo); modificabilidad (los protocolos evolucionan rápido); análisis de ancho de banda (distancia, volumen, latencia); conectividad intermitente/limitada/nula (integridad de datos ante cortes, reanudación consistente, modos degradados y de contingencia); seguridad (spoofing, eavesdropping, man-in-the-middle).
Sensores y actuadores	Sensor: características físicas del entorno → representación electrónica; actuador: la inversa. El sensor hub (coprocesador) integra y procesa datos descargando a la CPU. La pila de sensores (stack) convierte datos crudos en información: leer (driver, con frecuencia parametrizada), suavizar (el crudo trae ruido: media móvil, filtro de Kalman), convertir (formatos dispares a una forma común), fusión de sensores (combinar múltiples para una representación más exacta/completa/confiable: reconocer peatones de día o de noche combinando térmicos + radar + lidar + cámaras). Preocupaciones: representación fiel del entorno, respuesta a esa representación, seguridad/privacidad de los datos, y operación degradada (sin GPS en el túnel → dead reckoning).
Recursos	Tradeoff entre contribución del recurso y su volumen, peso y costo (millones de unidades sensibles al precio; límites térmicos y ambientales; safety exige backups que pesan y cuestan). Decisiones: asignar funciones a las ECUs (electronic control units) por ajuste función-procesador, críticidad (las ECUs potentes y confiables para lo crítico: el controlador del motor antes que el confort), ubicación en el vehículo, conectividad, localidad de comunicación (los que se hablan intensamente, juntos) y costo; derivar funcionalidad a la nube (¿alcanza la potencia local? ¿hay conectividad? ¿latencia? ¿privacidad?); apagar funciones según el modo (el "race mode" del deportivo desactiva el confort de la suspensión y activa el cálculo de torque y frenado); estrategia de display según resolución (motivo fuerte para MVC: intercambiar la vista según el dispositivo).

Ciclo de vida	Hardware first: el hardware suele elegirse antes de diseñar el software (stakeholders: gerencia, ventas, reguladores) — el arquitecto debe conducir activamente esas discusiones tempranas mostrando los tradeoffs. Testing: layouts en mil tamaños de pantalla (el botón generado de 1×1 píxel que nadie detecta); casos límite operativos (agotamiento de batería, UI asíncrona difícil de reproducir); uso de recursos (simuladores no miden bien batería); transiciones de red. En transporte/industria, cuatro niveles con trazabilidad total: componente de software → función (componentes juntos en entorno simulado) → dispositivo (desplegado en su ECU con entradas simuladas) → sistema (configuración completa en laboratorio y prototipo). Actualizaciones: consistencia de datos (upgrades one-way → datos en la nube), safety (no actualizar el control del motor en plena autopista: estados seguros para actualizar), despliegue parcial (actualizar sólo lo que cambia — modificabilidad + deployabilidad), retrofitting (componentes que llegan a fin de vida antes que el vehículo; tecnología nueva sobre plataformas viejas). Logging: descargar los logs fuera del dispositivo para incidentes y análisis.
----------------------	--

PARTE VI

La práctica del arquitecto

SAIP 4.^a ed., capítulos 19–26: descubrir los requerimientos que importan, diseñar con ADD, evaluar con ATAM, documentar con vistas, gestionar la deuda arquitectónica, convivir con Agile y crecer como arquitecto.

-
- 32** Requerimientos arquitectónicamente significativos
 - 33** Diseñar la arquitectura: el método ADD
 - 34** Evaluar la arquitectura: ATAM y evaluaciones livianas
 - 35** Documentar la arquitectura
 - 36** Gestionar la deuda arquitectónica
 - 37** El rol del arquitecto en los proyectos y en Agile
 - 38** Competencia arquitectónica
 - 39** Un vistazo al futuro: computación cuántica

32. Requerimientos arquitectónicamente significativos (ASRs)

Definición: ASR

Un **requerimiento arquitectónicamente significativo** (ASR) es aquel que tiene un **efecto profundo sobre la arquitectura** — la arquitectura bien podría ser dramáticamente distinta en su ausencia — y un **alto valor de negocio o misión**. Suelen (no siempre) tomar la forma de requerimientos de atributos de calidad. No se puede diseñar una arquitectura exitosa sin conocer los ASRs: lo primero que hacen los arquitectos experimentados es **hablar con los stakeholders importantes**.

32.1 Fuente 1: los documentos de requerimientos (sin ilusiones)

Parece obvio buscarlos allí, pero los documentos de requerimientos **fallan al arquitecto de dos maneras**: (1) la mayor parte de su información no afecta la arquitectura — las arquitecturas se moldean por los QAs, y la especificación típica se concentra en features; cuando aparecen QAs suelen ser intesteable ("el sistema será modular", "exhibirá alta usabilidad") — no son requerimientos útiles pero sí *invitaciones a empezar la conversación*; (2) mucho de lo que el arquitecto necesita **no figura en ningún documento de requerimientos** (metas de negocio de la organización, supuestos de equipos, intereses del desarrollador vs. los del adquirente). Aun así, hay que "olfatear" ASRs en las categorías: uso (roles vs. modos, internacionalización), tiempo, elementos externos (sistemas, protocolos, sensores), redes y sus propiedades de seguridad, orquestación de flujos, propiedades de seguridad (roles, permisos, autenticación), datos (persistencia, vigencia), recursos (tiempo, concurrencia, memoria, scheduling, energía, escalabilidad), gestión del proyecto (equipos, habilidades, coordinación), hardware previsto, flexibilidad/portabilidad/configuración y tecnologías o paquetes nombrados — y para cada ítem, **su evolución anticipada**.

32.2 Fuente 2: entrevistar stakeholders — el QAW

Los stakeholders **a menudo no saben cuáles son sus requerimientos de QA**: es una oportunidad de colaboración, no motivo de queja. El arquitecto aporta la experiencia de sistemas similares ("¿24/7? te explico cuánto cuesta y decidís el tradeoff con la afordabilidad"; "puedo entregarte algo mejor de lo que tenías en mente, ¿te sirve?"). El **Quality Attribute Workshop (QAW)** es un método facilitado y centrado en stakeholders para generar, priorizar y refinar escenarios **antes de que la arquitectura esté completa**:

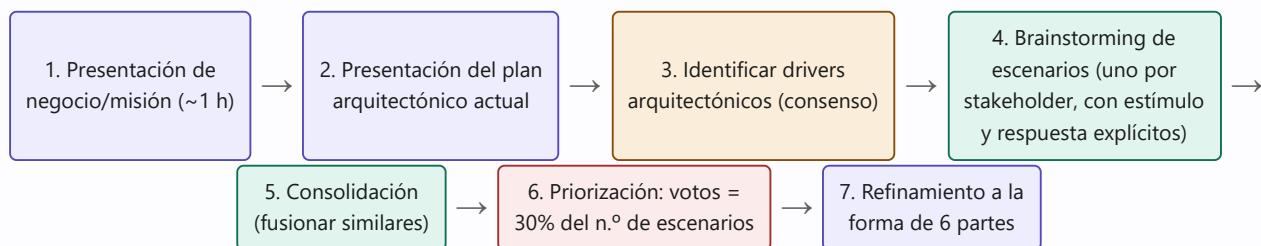


Figura 32.1 — Los pasos del Quality Attribute Workshop (QAW). Salida: drivers + escenarios QA priorizados por el grupo.

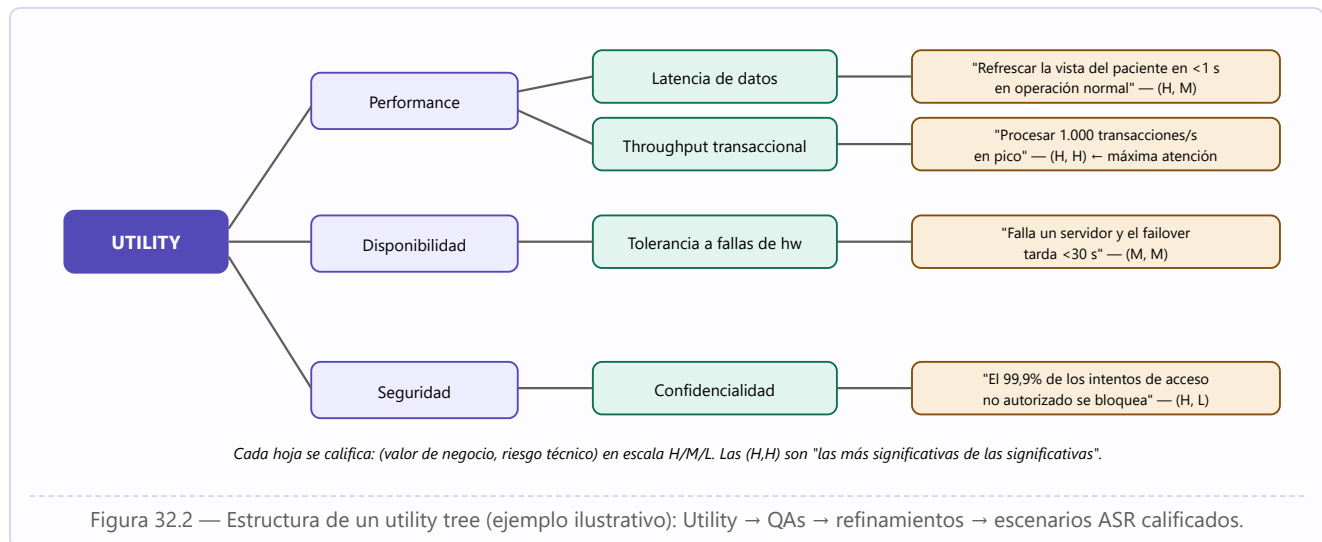
Ejemplo desarrollado: "no sé cuál debería ser el requerimiento" — la técnica de hacerse el tonto (R. Kazman)

Ante el clásico "no sé qué valor poner": — ¿Cuán rápido debe responder el sistema a la transacción? — No sé. — ¿24 horas está bien? — ¡¡No!! — ¿Una hora? — No. — ¿Cinco minutos? — No. — ¿Diez segundos? — Mmm... supongo que podría vivir con eso. **Un rango de valores aceptables basta para elegir mecanismos:** 24 horas, 10 minutos, 10 segundos o 100 milisegundos implican, para el arquitecto, enfoques arquitectónicos completamente distintos.

32.3 Fuente 3: las metas de negocio (PALM)

Las metas de negocio son la razón de ser del sistema. Tres relaciones posibles con la arquitectura: (1) **conducen a requerimientos de QA** (todo requerimiento de calidad se origina en algún propósito de valor: diferenciarse de la competencia → tiempo de respuesta inusualmente exigente — y conocer la meta permite *cuestionar* el requerimiento con sentido); (2) **afectan la arquitectura directamente sin inducir ningún QA** — anécdota del libro: el gerente exigió agregar una base de datos que el arquitecto había evitado elegantemente... porque la organización tenía una unidad de DBAs caros y ociosos que necesitaban trabajo (ninguna especificación capturaría eso, y sin embargo la arquitectura sin BD habría sido "deficiente"); (3) no influyen en absoluto. Método **PALM**: taller que elicit las metas (expresadas como *business goal scenarios* de siete partes), las consolida y prioriza, y por cada una identifica el QA y la medida de respuesta que la lograrían. Categorías de metas para no olvidar ninguna: crecimiento y continuidad; objetivos financieros; objetivos personales; responsabilidad hacia empleados / sociedad / estado / accionistas; posición de mercado; mejora de procesos; calidad y reputación del producto; gestión del cambio del entorno.

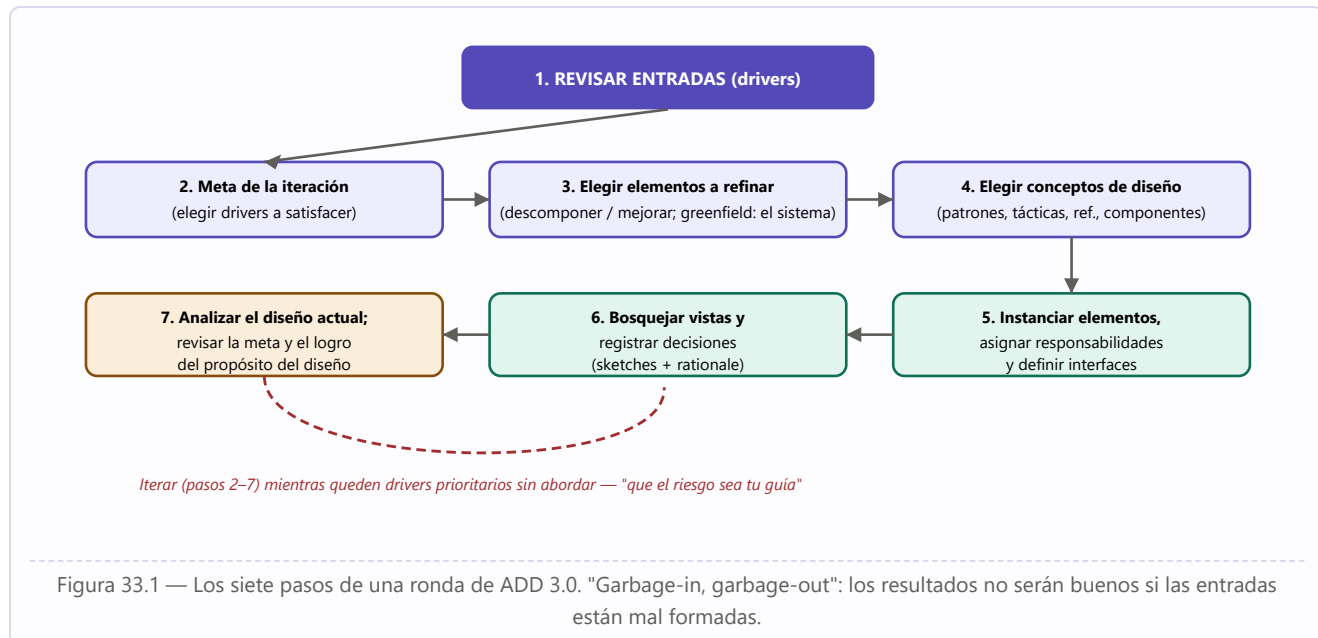
32.4 El utility tree



Cuando los "primary sources" no están disponibles, el arquitecto construye un **utility tree**: raíz "Utility" (la bondad global del sistema) → QAs principales (como meros placeholders) → **refinamientos** relevantes al sistema → **escenarios ASR** en las hojas, calificados por **valor de negocio** (H = imprescindible; M = importante pero no fatal; L = deseable) y **riesgo técnico** (H = me quita el sueño; L = confianza total). Chequeos: un QA sin escenarios invita a investigar si faltan; demasiados (H,H) ponen en duda que el sistema sea alcanzable. Y siempre: **el cambio sucede** — mantener el canal abierto con los stakeholders y hasta adelantarse (diseñar preliminarmente el cambio del que hay rumores y compartir su costo temprano).

33. Diseñar la arquitectura: el método ADD

Attribute-Driven Design (ADD), hoy en su versión 3.0: un método para diseñar la arquitectura de manera **sistemática, repetible y costo-efectiva** — "repetibilidad y enseñabilidad son las marcas de una disciplina de ingeniería" (que el diseño deje de ser sólo para gurús). El diseño convierte **drivers** en **estructuras**: los drivers son los ASRs *más* la funcionalidad primaria, las restricciones, los concerns y el **propósito del diseño**. Antes de empezar: establecer el **alcance del sistema** (qué queda dentro/fuera, con qué entidades externas interactúa — diagrama de contexto). El diseño se hace en **rondas** (lo hecho dentro de un ciclo de desarrollo), cada una con una o más **iteraciones**:



33.1 Detalle de los pasos críticos

- **Paso 1 — Revisar entradas:** propósito de la ronda (diseño para estimación temprana / refinar para un incremento / prototipo para mitigar riesgos), funcionalidad primaria (casos de uso o historias priorizados por los stakeholders — el arquitecto debe "apropiárselos": ¿faltó algún stakeholder? ¿cambió el negocio?), escenarios QA priorizados, restricciones y concerns. Los drivers alimentan el **backlog arquitectónico**.
- **Paso 4 — Elegir conceptos de diseño** (probablemente la decisión más difícil): identificar alternativas entre **tácticas, patrones, arquitecturas de referencia y componentes desarrollados externamente** — no reinventar la rueda: "la creatividad reside en identificar, combinar y adaptar soluciones existentes". Identificación por: catálogos y mejores prácticas (mucho material, calidad y sesgos desconocidos); experiencia propia (rápida, pero "si sólo tenés un martillo..."); pares (brainstorming). Selección: tabla de pros y contras, restricciones (una licencia no aprobada descarta un framework útil), compatibilidad con decisiones previas. Si la incertidumbre persiste: **prototipos descartables** — justificados si hay tecnología emergente o nueva en la empresa, QAs riesgosos, falta de información confiable, opciones de configuración por probar o dudas de integración. Para decidir si experimentar: la técnica **VoI (Value of Information)** calcula, vía el teorema de Bayes, el valor esperado de la información perfecta (EVPI: máximo a pagar por experimentos concluyentes) y de la imperfecta (EVSI).

- **Paso 5 — Instanciar y asignar responsabilidades:** instanciar = **customizar** la arquitectura de referencia (agregar/quitar elementos), **adaptar** el patrón (el cliente-servidor no dice cuántos clientes ni protocolos: instanciar llena los huecos), **realizar** la táctica (con código propio, con un patrón que la incluya, o con un componente externo — ej. autenticar actores: login propio, patrón de seguridad o framework), **configurar** el componente externo. Principio: elementos con **alta cohesión, responsabilidades acotadas, bajo acoplamiento**. Definir **interfaces**: las externas son restricciones (una por sistema externo); las internas surgen de analizar cómo colaboran los elementos en los casos de uso y escenarios — los **diagramas de secuencia/actividad** revelan las relaciones y la información intercambiada; si se eligió un stack tecnológico completo, muchas interfaces vienen "horneadas".
- **Paso 6 — Bosquejar vistas y registrar decisiones:** los sketches (pizarra, papel, herramienta) son la **documentación preliminar**; disciplina: **escribir las responsabilidades de cada elemento en el momento de identificarlo** (foto de la pizarra + tabla de responsabilidades). Documentar según el propósito (análisis / construcción / educación) y los riesgos. Registrar el **rationale** de las decisiones significativas (tradeoffs) — sin él, después nadie entiende cómo se llegó al resultado.
- **Paso 7 — Analizar y decidir si seguir:** revisar (mejor con otro: no comparte tus supuestos — igual que testing) y evaluar el avance contra el propósito. ¿Más iteraciones? **Que el riesgo guíe:** al menos los drivers de mayor prioridad deben estar tratados o "suficientemente bien". Seguimiento con **backlog** (drivers + prototipos pendientes + issues de revisiones + concerns que aparecen — ej.: la referencia web elegida no trae manejo de sesión → nuevo ítem) y **tablero Kanban** de tres columnas: *Not Yet / Partially / Completely Addressed*, con criterios explícitos para mover (analizado o probado en prototipo) y colores por prioridad.

34. Evaluar la arquitectura: ATAM y evaluaciones livianas

Frank Lloyd Wright

"Un médico puede enterrar sus errores, pero un arquitecto sólo puede aconsejar a sus clientes que planten enredaderas."

Evaluación = determinar el grado en que la arquitectura es **apta para el propósito** al que se destina. Es una actividad de **reducción de riesgo**: un riesgo tiene probabilidad e impacto (costo esperado = probabilidad × costo del impacto); la evaluación es como un **seguro** — cuánto necesitas depende de tu exposición (¿misión crítica multimillonaria o jugueto de consola? ¿dominio precedentado o primera vez?) y de tu tolerancia. Regla de oro: **el costo de evaluar debe ser menor que el valor que aporta**. Su salida principal: **riesgos identificados** (arreglarlos es una decisión costo/beneficio posterior).

Actividades esenciales de toda evaluación: (1) los revisores entienden el estado actual de la arquitectura (documentación y/o presentación del arquitecto); (2) determinan los **drivers** que guiarán la revisión (los escenarios QA de mayor prioridad — no los casos de uso puramente funcionales); (3) por cada escenario, deciden si **se satisface** (el arquitecto "camina" la arquitectura) y si otras decisiones lo **comprometen**, proponiendo alternativas para lo riesgoso; (4) capturan los **problemas potenciales** (si son reales: arreglar, o aceptar el riesgo explícitamente). Profundidad del análisis según: importancia de la decisión, número de alternativas, y *good enough* antes que perfecto. ¿Quién evalúa? El **arquitecto** (cada decisión clave — parte integral del diseño), los **pares** (revisión acotada en tiempo, p. ej. al final del paso 7 de ADD) o **externos** (más objetividad, conocimiento especializado, y — justo o no — los gerentes les creen más; suelen evaluar arquitecturas completas).

34.1 El ATAM

El **Architecture Tradeoff Analysis Method**: dos décadas evaluando arquitecturas grandes (automotriz, financiera, defensa); diseñado para que los evaluadores no necesiten familiaridad previa con la arquitectura ni el sistema esté construido. **Tres grupos**: el **equipo evaluador** (externo al proyecto, 3–5 personas con roles — líder, líder de evaluación, escriba de escenarios, escriba de actas, interrogadores —; competentes, imparciales, sin agendas); los **decisores del proyecto** (PM, cliente, y **el arquitecto, siempre** — regla cardinal: el arquitecto participa de buena gana); y los **stakeholders de la arquitectura** (10–25 en sistemas grandes: desarrolladores, testers, integradores, mantenedores, ingenieros de performance, usuarios, constructores de sistemas vecinos — su trabajo: articular las metas de QA que harían del sistema un éxito).

Salidas del ATAM	Contenido
1. Presentación concisa de la arquitectura	En ≤ 1 hora — concisa y comprensible (subproducto valioso por sí mismo).
2. Articulación de las metas de negocio	Con frecuencia, parte del público las oye por primera vez ; quedan como legado del proyecto.
3. Escenarios QA priorizados	Base del análisis (en forma de 6 partes).
4. Riesgos y no-riesgos	Riesgo : decisión arquitectónica que puede acarrear consecuencias indeseables frente a los requerimientos de QA; no-riesgo : decisión que, analizada, se considera segura. Los riesgos son el output principal — semilla del plan de mitigación.

5. Temas de riesgo (risk themes)	Al final, el equipo agrupa los riesgos por debilidades sistémicas (documentación desactualizada como práctica, backup insuficiente...) y los liga a qué metas de negocio amenazan — eso eleva "cuestiones técnicas esotéricas" a la atención de la gerencia.
6. Mapeo decisiones ↔ requerimientos de calidad	Por cada escenario analizado: qué decisiones lo logran (rationale documentado).
7. Sensitivity y tradeoff points	Punto de sensibilidad: decisión con efecto marcado sobre una respuesta de QA. Punto de tradeoff: decisión a la que ≥2 respuestas de QA son sensibles, una mejorando y otra empeorando (ej.: la frecuencia del heartbeat — más frecuencia = mejor disponibilidad, peor performance).

Fases: 0 — "Asociación y preparación" (logística, lista de stakeholders, acuerdos, el equipo estudia la documentación); 1 y 2 — "Evaluación" (fase 1 con los decisores: pasos 1–6; hiato de ~una semana; fase 2 con los stakeholders: recapitulación + pasos 7–9 — la analogía del libro: fase 1 es probar tu programa con tus propios criterios; fase 2, dárselo al equipo de QA independiente); 3 — "Seguimiento" (informe final circulado para corregir malentendidos). **Los nueve pasos:**

1. **Presentar el ATAM** (el líder explica proceso y salidas).
2. **Presentar las metas de negocio** (un decisor: funciones más importantes, restricciones técnicas/gerenciales/económicas/políticas, contexto, stakeholders mayores, drivers).
3. **Presentar la arquitectura** (el arquitecto, al nivel apropiado: restricciones técnicas — SO, plataformas, sistemas vecinos — y sobre todo los **enfoques/patrones/tácticas** usados para cumplir los requerimientos; con vistas: contexto, C&C, descomposición o capas, despliegue).
4. **Identificar los enfoques arquitectónicos** (catalogar patrones y tácticas conocidos por sus efectos sobre los QAs: capas → portabilidad; pub-sub → escalabilidad; redundancia activa → disponibilidad).
5. **Generar el utility tree** (arquitecto + decisores: escenarios con importancia de negocio y riesgo técnico — "modificable ¿en qué?, ¿throughput cuán alto?").
6. **Analizar los enfoques** (el equipo interroga los escenarios de mayor rango uno a uno; el arquitecto explica cómo la arquitectura los soporta; se documentan decisiones, riesgos, no-riesgos, sensibilidades, tradeoffs; análisis "de servilleta" — la meta es convencerse de que la instanciación del enfoque es apropiada, no ser exhaustivos).
7. **Brainstorming y priorización de escenarios** (los stakeholders proponen escenarios operacionalmente significativos según sus roles — el mantenedor propondrá modificabilidad, el usuario facilidad de operación, QA testabilidad; fusión de similares; votación con **30% del total de escenarios en votos por persona**; el resultado se compara con el utility tree: la concordancia valida al arquitecto; una **discrepancia grande es en sí un riesgo** — desacuerdo sobre las metas entre stakeholders y arquitecto).
8. **Analizar los enfoques con los nuevos escenarios** (los 5–10 primeros).
9. **Presentar resultados** (riesgos agrupados en temas ligados a metas de negocio; enfoques, escenarios, utility tree, riesgos/no-riesgos, sensibilidades y tradeoffs).

Ejemplo desarrollado: "Going off script" — ninguna evaluación sale según el libreto (P. Clements)

Historias reales de ATAMs: llegar y descubrir que **no había arquitectura** que evaluar (sólo pilas de diagramas de clases — la evaluación produjo los QAs articulados, una arquitectura "de pizarra" y obligaciones de documentación); perder al arquitecto a mitad de ejercicio (renunció entre la preparación y la ejecución; siguió su aprendizaje); descubrir a mitad de camino que la arquitectura evaluada estaba siendo **reemplazada por una nueva** que nadie había mencionado; y el caso del gerente maquiavélico que usó la

evaluación para que su **equipo de diseñadores junior emergiera como verdadero equipo de arquitectura** mientras se deshacía de un arquitecto autocrático — los junior, juntos, dibujaron en medio día una imagen del sistema más coherente que la del arquitecto en dos días. ¿Por qué siempre "sale bien"? (1) quienes encargan la evaluación quieren que funcione; (2) honestidad total del equipo evaluador (jamás "bluffear"); (3) los métodos construyen consenso continuo: **no hay sorpresas al final**.

34.2 Evaluación liviana (LAE) y cuestionarios

Lightweight Architecture Evaluation: para revisión interna por pares, regular, sobre lo que cambió desde la última revisión o una porción no examinada; muchos pasos del ATAM se omiten o abrevian (todos son de la casa: entender el contexto toma minutos); duración de **un par de horas a un día completo**; convocada y conducida por el arquitecto del proyecto; sin informe final (el escriba comparte los hallazgos). Contra: el equipo interno es menos objetivo (menos ideas nuevas y menos disenso); a favor: barato, fácil de convocar, bajo protocolo — un "sanity check" de calidad arquitectónica desplegable en cualquier momento. Aún más liviano: el **cuestionario basado en tácticas** (~1 hora por QA — introspección del arquitecto o sesión pregunta-respuesta con un evaluador).

Anécdota-advertencia: el cifrado XOR

Evaluando un sistema de datos de salud con el cuestionario de seguridad, ante "¿el sistema soporta el cifrado de datos?" el arquitecto sonrió avergonzado: tenían el requisito de que nada viajara "en claro" por la red...

así que aplicaban XOR a todos los datos antes de enviarlos. Requisito cumplido "en sentido estricto" — con un cifrado que rompe un estudiante de secundaria. Eso descubre un cuestionario de tácticas en minutos y a costo casi nulo.

35. Documentar la arquitectura

La documentación **habla por el arquitecto**: hoy (mientras hace otras cosas en vez de contestar cien preguntas) y mañana (cuando olvidó los detalles o ya no está). No es un artefacto burocrático: es el **receptáculo de las decisiones de diseño** a medida que se confirman — idealmente, **diseñar y documentar son el mismo trabajo**. Es tanto **prescriptiva** (restringe decisiones futuras) como **descriptiva** (cuenta las tomadas). Cuatro usos: **educación** (introducir gente al sistema — nuevos miembros, o el cliente al que le mostrás la solución por primera vez), **comunicación entre stakeholders** (el lector más ávido: **el futuro arquitecto** — "la documentación de arquitectura es una carta de amor que le escribís a tu yo futuro"), **base de análisis y construcción**, y **forense** (tras un incidente).

Notaciones: informales (cajas y líneas, semántica en lenguaje natural — no analizable formalmente), **semiformales** (UML, SysML: sintaxis verificable sin semántica completa — lo que usan las herramientas comerciales) y **formales** (ADLs con semántica matemática — análisis y generación; "en la práctica, su uso es raro"). A más formalidad: menos ambigüedad y más análisis, a más costo. Y ninguna notación sirve para todo: un diagrama de clases no razona schedulability.

Idea clave — El principio fundamental de la documentación

Documentar una arquitectura = documentar las vistas relevantes + agregar la documentación que aplica a más de una vista. ¿Cuáles vistas? Depende de los usos (educación, análisis, generación, planificación...): cada vista expone unos QAs (módulos → mantenibilidad; despliegue → performance y fiabilidad) y tiene costo/beneficio propio. La elección de vistas suele venir de la necesidad de documentar los **patrones** del diseño (de módulos, de C&C o de asignación).

Vistas de módulos — propiedades a documentar por módulo: nombre (a menudo con jerarquía A.B.C), **responsabilidades** (con detalle suficiente; trazadas a requerimientos), información de implementación (mapeo a archivos fuente, test info, gestión, restricciones, historial). Sirven para explicar el sistema a extraños y para el nuevo desarrollador; **no** sirven para razonar conducta runtime. **Vistas C&C** — componentes (con posible subarquitectura propia) y conectores (ipueden ser n-arios y complejos: un bus de servicios entero!); propiedades: fiabilidad, performance, recursos, funcionalidad, seguridad, concurrencia, extensibilidad runtime; se "animan" recorriendo hilos de actividad de punta a punta. **Vistas de asignación** — mapeo a entornos (hardware, archivos, equipos); estáticas o dinámicas (condiciones y disparadores de reasignación). Y las **quality views** (espíritu de ISO/IEC/IEEE 42010: vistas movidas por los concerns de los stakeholders): extraen de las vistas estructurales lo relevante a una incumbencia — vista de **seguridad** (medidas, componentes con rol de seguridad, protocolos, respuesta a amenazas), de **comunicaciones** (canales, QoS, concurrencia — para sistemas globalmente dispersos), de **excepciones/manejo de errores**, de **fiabilidad**, de **performance**. Vistas **combinadas** (overlay) cuando la asociación es fuerte: C&C entre sí; despliegue + C&C de procesos; descomposición + asignación de trabajo/implementación/usos/capas.

Documentar el comportamiento: notaciones **de traza** (casos de uso; **diagramas de secuencia** UML — lifelines, mensajes síncronos/asíncronos/de retorno, barras de ejecución; **de comunicación** — grafo con interacciones numeradas; **de actividad** — flowcharts con bifurcación condicional y **concurrencia** vía fork/join) vs. notaciones **comprehsivas** (máquinas de estados UML: estados y transiciones "evento [guarda] / acción" — el comportamiento completo del elemento). **Más allá de las vistas**: mapeos entre vistas (tablas many-to-many: "is implemented by" / "implements"), documentación de los patrones usados (y por qué),

diagramas de contexto, **guía de variabilidad** (cómo ejercitar los puntos de variación), **rationale** (por qué el diseño es como es: decisiones, alternativas rechazadas, tradeoffs, supuestos — registrarlo al decidir: después no te acordás), glosario y control del documento.

Stakeholders y sus vistas (selección): el **PM** — descomposición, despliegue, asignación de trabajo, contexto; el **desarrollador** — casi todo + interfaces + variabilidad + rationale; **testers/integradores** — usos, C&C, despliegue, interfaces y especificaciones de conducta (¡y merecen atención: el testing es ~la mitad del esfuerzo!); **diseñadores de sistemas vecinos** — interfaces y modelo de datos; **mantenedores** — como desarrolladores + descomposición + rationale; **usuarios finales** — C&C de flujo y despliegue (revisiones con usuarios destapan discrepancias); **analistas** — lo que alimente sus modelos (schedulability, fiabilidad, model checking); **infraestructura**; y el **futuro arquitecto**, interesado en todo. **Prácticas**: herramientas de modelado (SysML) con multi-usuario, versionado y trace links; wikis compartidos con permisos; **estrategia de releases** de la documentación (a tiempo para cada hito); **arquitecturas dinámicas** (el browser que se auto-reconfigura instalando plug-ins, los shops que despliegan varias veces al día: documentar los **invariantes** de todas las versiones y las **formas permitidas de cambio**; generar la documentación de interfaces automáticamente); **trazabilidad** a requerimientos y metas de negocio (si todos los ASRs están "cubiertos" por trace links, la parte arquitectónica está bien).

36. Gestionar la deuda arquitectónica

Definición: Deuda arquitectónica

Forma de entropía del diseño: sin atención y esfuerzo, los diseños se vuelven más difíciles de mantener y evolucionar. Es una forma de deuda técnica **importante y muy costosa**, más difícil de detectar y erradicar que la deuda de código porque involucra **incumbencias no locales** — las herramientas de deuda de código (inspecciones, quality checkers) no la ven. No toda deuda es mala: a veces se viola un principio por un tradeoff que vale la pena (time-to-market, performance).

Proceso de análisis (automatizable e integrable a CI) sobre tres insumos: **código fuente** (dependencias estructurales, por análisis estático), **historial de revisiones** (co-evolución: qué archivos cambian juntos) e **issue tracker** (razón de los cambios: bug o feature). Representación: la **DSM (design structure matrix)** — archivos en filas y columnas (mismo orden); las celdas anotan dependencias estructurales ("dp" depende, "im" implementa, "ex" extiende, "ih" hereda) y **evolutivas** (número de co-commits). Una DSM sana es **rala y triangular inferior** (cada archivo depende de pocos, y sólo "hacia abajo" — sin ciclos). Caso Apache Camel: la matriz estructural era rala (parecía poca deuda)... pero al superponer los co-cambios del control de versiones quedó **densa y con anotaciones sobre la diagonal** — acoplamiento evolutivo en todas direcciones: alta deuda, confirmada por sus arquitectos ("cada cambio es costoso y complejo; predecir cuándo estará listo un feature es un desafío").

36.1 Hotspots: los seis anti-patrones

Anti-patrón	Cómo se detecta
Unstable interface	Un archivo influyente (un servicio/abstracción importante) con muchos dependientes que cambia frecuentemente junto con ellos según el historial.
Modularity violation	Módulos estructuralmente desacoplados (sin dependencia entre sí) que co-cambian con frecuencia — comparten un "secreto" no encapsulado.
Unhealthy inheritance	El padre depende de sus hijos; o un cliente depende del padre y de uno o más hijos. (Ej. real en Apache Cassandra: <code>SSTable</code> ↔ <code>SSTableReader</code> , herencia + dependencia inversa + 68 co-commits .)
Cyclic dependency / clique	Grupo de archivos que forman un grafo fuertemente conexo (hay camino de dependencias entre cualquier par).
Package cycle	Dos o más paquetes dependen mutuamente, en vez de formar jerarquía.
Crossing	Archivo con alto fan-in y alto fan-out que además co-cambia frecuentemente con sus dependientes y sus dependencias.

Hotspots = conjuntos de archivos con contribución desmedida a los costos de mantenimiento (alta tasa de bugs y churn asociada a anti-patrones). Se pesa cada anti-patrón sumando bug-fixes, cambios y churn de sus archivos → refactoring dirigido (romper el ciclo invirtiendo una dependencia; mover funcionalidad del hijo al padre; encapsular el secreto compartido como abstracción propia).

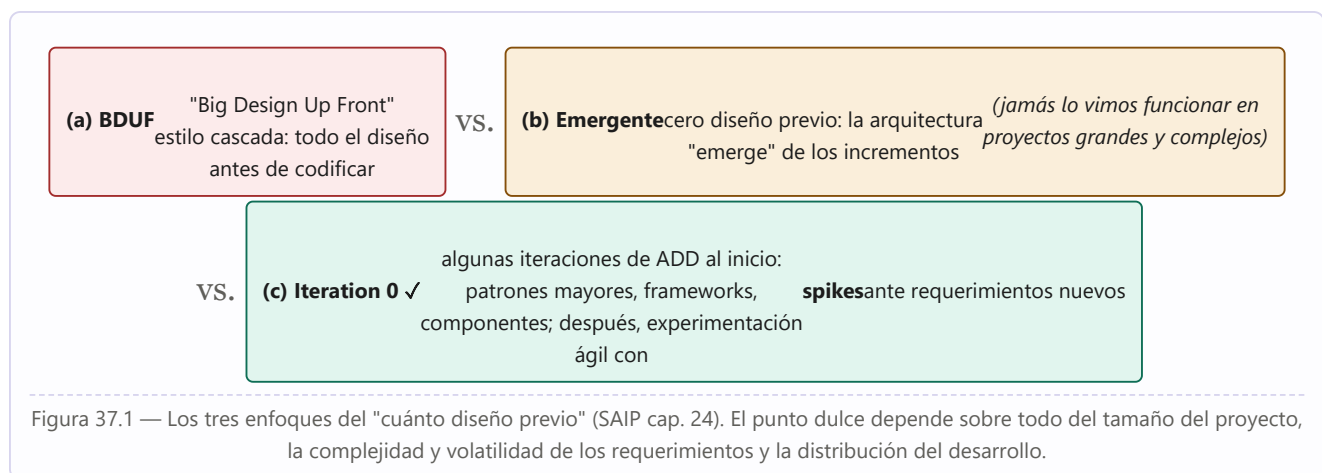
Ejemplo desarrollado: el caso SoftServe (SS1)

Sistema de 797 archivos, 2 años de historia: 2.756 issues (1.079 bugs), 3.262 commits. El análisis identificó **3 clusters** con los anti-patrones más dañinos: **291 archivos (~1/3 del proyecto) concentraban el 89% de los defectos**. El arquitecto jefe reconoció los problemas ("pero no sabía explicar por qué") y diseñó refactorings guiados por los anti-patrones. Números: costo estimado del refactoring = **14 persona-meses**; ahorro anual estimado (con supuestos conservadores: los archivos refactorizados tendrán la tasa *promedio* de bugs) = **41,35 persona-meses por año**. Ese es el *business case* que convence a un gerente — no "dame tres meses para refactorizar y no te doy funcionalidad nueva".

37. El rol del arquitecto en los proyectos y en Agile

Arquitecto y project manager: roles complementarios — el PM responsable de la cara **externa** (presupuesto, calendario, staffing), el arquitecto de la **interna** (técnica); la vista externa debe reflejar fielmente la situación interna y viceversa. El PM se apoya en el arquitecto en la mayoría de las áreas del PMBOK. **Arquitectura incremental:** liberar la arquitectura (su documentación) en incrementos al ritmo del proyecto; primer incremento recomendado: **descomposición de módulos** (define equipos → organización del proyecto), **estructura de usos** (planifica los incrementos), la(s) **C&C** que mejor transmita(n) la solución, y un **despliegue a grandes trazos** (¿móvil? ¿nube?). Y usar la influencia para que los primeros releases enfrenten los QAs más desafiantes — sin sorpresas arquitectónicas tardías.

37.1 Arquitectura y Agile

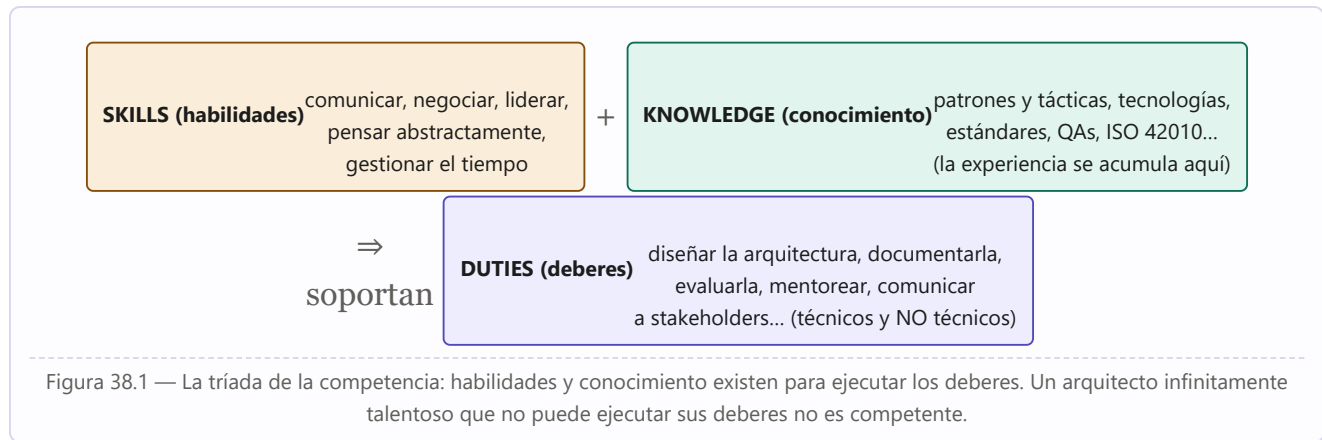


Agile nació como rebelión contra procesos rígidos, documentación agobiante y la entrega única final. Su relación con la arquitectura fue tirante ("si no entregás software que funciona, no hacés nada de valor → architecture, schmarchitecture!"), pero para sistemas medianos y grandes esa visión **colapsó bajo el peso de la experiencia**: las soluciones de seguridad, performance o safety **no se atornillan después** — deben diseñarse desde el principio aunque los primeros 20 incrementos no las ejerciten. "Sí: empezá a codificar y la arquitectura emergerá — pero será la equivocada." Un **spike** es una tarea time-boxed en una rama separada para contestar una pregunta técnica; si tiene éxito, se fusiona. De los 12 principios ágiles, el comentario arquitectónico del libro: 6 acuerdos totales, 4 acuerdos generales y **2 desacuerdos fuertes**. A escala empresarial, **SAFe** reconoce la "intentional architecture" (estrategias planificadas que guían la sincronización entre equipos) balanceada con "emergent design".

Desarrollo distribuido: beneficios (costo — con matices: sin expertise de dominio ni gestión adaptada, el retrabajo se come el ahorro —, disponibilidad de talento, conocimiento del mercado local); desafío central: la **coordinación** (el cambio de una interfaz que antes se resolvía en la máquina de café ahora exige una teleconferencia a medianoche de alguien). Métodos: contactos informales (sólo co-ubicados), documentación (bien escrita y organizada), reuniones, comunicación asíncrona electrónica. Consecuencias arquitectónicas: la asignación de responsabilidades a equipos importa más; **minimizar las dependencias entre módulos de equipos remotos**; la documentación pesa más que nunca.

38. Competencia arquitectónica

¿Qué hace bueno a un arquitecto — y a una organización que produce arquitecturas? La competencia **individual** descansa en la tríada:



Ejemplos calibradores: "diseñar la arquitectura" es un **deber**; "capacidad de pensar abstractamente" es una **habilidad**; "patrones y tácticas" son **conocimiento**. Llamativa cantidad de deberes **no técnicos** (liderazgo, gestión, comunicación): quien aspire a arquitecto debe atender esa dimensión de su formación. Para mejorar: (1) **experiencia ejecutando los deberes** (el aprendizaje con mentores es el camino productivo — la educación sola aumenta el conocimiento, no la competencia); (2) **habilidades no técnicas** (cursos de liderazgo, gestión del tiempo); (3) **dominar y mantener al día el cuerpo de conocimiento** (la nube no era tema hace unos años: cursos, certificaciones, libros, conferencias, comunidades). Sobre la experiencia: "la única fuente de conocimiento es la experiencia" (Einstein) — pero por suerte no hace falta quemarse para saber que la estufa quema: también se aprende de maestros. — ¿Cómo se llega al Carnegie Hall? *Practice, practice, practice.*

Definición: Competencia arquitectónica de una organización

"La capacidad de la organización para **hacer crecer, usar y sostener las habilidades y el conocimiento necesarios para ejecutar efectivamente prácticas centradas en la arquitectura**, en los niveles individual, de equipo y organizacional, para producir arquitecturas con costo aceptable que conduzcan a sistemas alineados con las metas de negocio."

Deberes organizacionales — de **personal**: contratar arquitectos talentosos, carrera de arquitecto, prestigio del puesto, mentoría, formación, certificación, medición del desempeño; de **proceso**: prácticas de arquitectura de toda la organización, responsabilidades y autoridad claras, foro de arquitectos, **architecture review board**, hitos arquitectónicos en los planes de proyecto, input del arquitecto en la definición de producto y en la estructura de los equipos, influencia durante todo el ciclo de vida; de **tecnología**: repositorios de arquitecturas y artefactos reutilizables, repositorio de design concepts, recursos centralizados de herramientas. (Lista útil también para entrevistas: preguntale a tu futura empresa cuántas de estas cosas hace.) Camino final de mejora: **ser mentoreado y mentorear** — enseñar un concepto es el test de fuego de haberlo entendido.

39. Un vistazo al futuro: computación cuántica

Los sistemas actuales viven décadas: si el tuyo toca áreas que la computación cuántica afectará, conviene entenderla. El **qubit** es la unidad de cómputo: se caracteriza por dos **amplitudes** (probabilidades de que la medición dé 0 o 1, con $|\alpha|^2 + |\beta|^2 = 1$) y una **fase**; un qubit con ambas probabilidades no nulas está en **superposición**; la **medición destruye el estado** (colapsa a 0 o 1) y por eso **no se puede copiar un qubit** (no-cloning). Operaciones (casi todas **invertibles**, salvo READ): NOT (intercambia amplitudes), Z (suma π a la fase), HAD (Hadamard: superposición igualitaria), CNOT (dos qubits: el primero controla el NOT del segundo). El **entrelazamiento** — sin análogo clásico — correlaciona las mediciones de dos qubits sin importar tiempo ni distancia, y habilita la **teleportación cuántica**: copiar el estado de un qubit a otro remoto (destruyendo el original) usando tres qubits + **dos bits clásicos**; inherentemente segura (el espía sólo ve los dos bits) — base de **HTTQP**, el protocolo que el NIST considera como reemplazo de HTTPS *antes* de que existan las máquinas capaces de romperlo.

Impacto en seguridad: el algoritmo de **Grover** invierte funciones (hashes) con speedup cuadrático — las contraseñas hasheadas "seguras por siglos" se vuelven vulnerables; el de **Shor** factoriza el producto de dos primos grandes eficientemente — rompe el cifrado moderno basado en esa dificultad (NP-hard clásicamente). Pendientes: la **QRAM** (memoria cuántica para alimentar datos masivos — aún no existe) y algoritmos como **HHL** (inversión de matrices, corazón del machine learning, con restricciones). Consejo al arquitecto: seguir el campo; **aislar las partes del sistema** que la computación cuántica pueda afectar (sobre todo criptografía) para minimizar la disrupción cuando llegue.

PARTE VII

Patrones de diseño (GoF)

Basada en Gamma, Helm, Johnson y Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software (1995): qué es un patrón, el catálogo de los 23, los principios del diseño OO reutilizable, el caso Lexi y las comparaciones entre patrones.

-
- 40 Qué es un patrón de diseño
 - 41 El catálogo: clasificación y los 23 patrones
 - 42 Cómo los patrones resuelven los problemas de diseño
 - 43 Caso de estudio: el editor Lexi
 - 44 Comparaciones y discusión de los patrones
 - 45 Los patrones hoy: la lectura de Refactoring.Guru

40. Qué es un patrón de diseño

Diseñar software OO es difícil; diseñar software OO **reutilizable** es aún más difícil. Los diseñadores expertos saben algo que los novatos no: **no resuelven cada problema desde primeros principios** — reutilizan soluciones que les funcionaron antes. Por eso en los sistemas OO se encuentran **patrones recurrentes de clases y objetos comunicantes**. La analogía de los autores: los novelistas no inventan sus tramas de cero — siguen patrones como "el héroe trágicamente imperfecto" o "la novela romántica". El propósito del libro es **registrar la experiencia de diseño OO como patrones**: un catálogo de 23, con formato consistente — ninguno nuevo ni no probado (todos aplicados más de una vez en sistemas distintos, la mayoría nunca antes documentados: eran folklore de la comunidad).

Christopher Alexander (el origen, desde la arquitectura de edificios)

"Cada patrón describe un problema que ocurre una y otra vez en nuestro entorno, y luego describe el núcleo de la solución a ese problema, de tal manera que puedas usar esa solución un millón de veces, sin hacerlo nunca dos veces de la misma manera."

Definición: los cuatro elementos esenciales de un patrón

1. El nombre: un asa para describir el problema, la solución y las consecuencias en una o dos palabras — aumenta el **vocabulario de diseño**, permite diseñar a mayor abstracción y comunicarlo (encontrar buenos nombres fue de lo más difícil del catálogo). **2. El problema:** cuándo aplicar el patrón, su contexto, a veces condiciones previas. **3. La solución:** los elementos del diseño, sus relaciones, responsabilidades y colaboraciones — **plantilla abstracta**, no un diseño concreto. **4. Las consecuencias:** resultados y tradeoffs (espacio/tiempo, flexibilidad, extensibilidad, portabilidad) — críticas para evaluar alternativas.

Nivel de abstracción preciso: **no** son diseños codificables tal cual (listas enlazadas, tablas hash) **ni** diseños complejos de dominio para aplicaciones enteras: son "descripciones de objetos y clases comunicantes, personalizadas para resolver un problema general de diseño en un contexto particular". Formato del catálogo (plantilla uniforme para aprender, comparar y usar): nombre y clasificación, **intent**, also known as, motivación, aplicabilidad, estructura (notación OMT), participantes, colaboraciones, consecuencias, implementación, sample code (C++/Smalltalk), known uses (≥ 2 dominios), related patterns.

40.1 El ejemplo canónico: MVC de Smalltalk-80

MVC construye interfaces de usuario con tres clases de objetos: el **Modelo** (el objeto de aplicación), la **Vista** (su presentación en pantalla) y el **Controlador** (cómo reacciona la UI al input). Antes de MVC se amontonaban juntos; MVC los desacopla para flexibilidad y reutilización. Dentro de MVC viven tres patrones:

- Modelo y vistas desacoplados por un protocolo **subscribe/notify** (el modelo notifica; cada vista se actualiza — múltiples vistas del mismo modelo: planilla, histograma y torta): es el patrón **Observer** generalizado (desacoplar objetos de modo que el cambio en uno afecte a cualquier número de otros sin que el cambiado los conozca).
- Las vistas se **anidan** (panel de botones = vista compuesta de vistas botón) vía la clase CompositeView: es **Composite** (agrupar objetos y tratar al grupo como a un individuo).

- La relación Vista–Controlador (cambiar cómo responde la vista al input sin cambiar su presentación — deshabilitarla con un controlador que ignora eventos): es **Strategy** (un objeto que representa un algoritmo intercambiable).
- MVC usa además **Factory Method** (la clase de controlador por defecto de una vista) y **Decorator** (agregar scrolling).

41. El catálogo: clasificación y los 23 patrones

Dos criterios de clasificación: el **propósito** — **creacionales** (el proceso de creación de objetos), **estructurales** (la composición de clases u objetos), **de comportamiento** (cómo interactúan y distribuyen responsabilidad) — y el **alcance** — de **clase** (relaciones entre clases y subclases, establecidas por herencia, **estáticas**, fijadas en compilación) o de **objeto** (relaciones entre objetos, cambiables en runtime, dinámicas — la mayoría de los patrones).

	Creacionales	Estructurales	De comportamiento
Clase	Factory Method	Adapter (de clase)	Interpreter · Template Method
Objeto	Abstract Factory · Builder · Prototype · Singleton	Adapter (de objeto) · Bridge · Composite · Decorator · Facade · Flyweight · Proxy	Chain of Responsibility · Command · Iterator · Mediator · Memento · Observer · State · Strategy · Visitor

Figura 41.1 — El "espacio de patrones de diseño" (Tabla 1.1 del GoF): propósito × alcance.

Las fichas completas → Parte X

Esta tabla da la **intención declarada** de cada patrón, que es lo que se escanea para encontrar candidatos. La **ficha completa** de cada uno — problema, solución y participantes, aplicabilidad textual del GoF, consecuencias a favor y en contra, qué te deja variar y patrones relacionados — está en la **Parte X** (caps. 55–58).

41.1 Los 23 intents

Patrón	Intent (propósito declarado)
CREACIONALES — abstraen la instanciación; encapsulan el conocimiento de qué clases concretas usa el sistema y ocultan cómo se crean y combinan sus instancias. Configuración estática (compilación) o dinámica (runtime).	
Abstract Factory	Proveer una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.
Builder	Separar la construcción de un objeto complejo de su representación , de modo que el mismo proceso de construcción pueda crear representaciones diferentes.
Factory Method	Definir una interfaz para crear un objeto, pero dejar que las subclases decidan qué clase instanciar ; difiere la instanciación a las subclases.
Prototype	Especificar los tipos de objetos a crear usando una instancia prototípica , y crear los nuevos copiándola .
Singleton	Asegurar que una clase tenga una sola instancia , con un punto de acceso global a ella.
ESTRUCTURALES — cómo componer clases y objetos en estructuras mayores: los de clase usan herencia para componer interfaces o implementaciones; los de objeto componen objetos, con la flexibilidad de cambiar la composición en runtime.	
Adapter	Convertir la interfaz de una clase en otra que los clientes esperan; hace trabajar juntas clases que no podrían por interfaces incompatibles.
Bridge	Desacoplar una abstracción de su implementación para que ambas puedan variar independientemente.
Composite	Componer objetos en estructuras de árbol para representar jerarquías parte-todo; los clientes tratan uniformemente a los objetos individuales y a las composiciones.

Decorator	Adjuntar responsabilidades adicionales a un objeto dinámicamente ; alternativa flexible a la subclasificación para extender funcionalidad.
Facade	Proveer una interfaz unificada a un conjunto de interfaces de un subsistema; una interfaz de más alto nivel que lo hace más fácil de usar.
Flyweight	Usar compartición para soportar eficientemente grandes cantidades de objetos de grano fino.
Proxy	Proveer un sustituto o representante de otro objeto para controlar el acceso a él.
DE COMPORTAMIENTO — algoritmos y asignación de responsabilidades; patrones de comunicación entre objetos: quitan el foco del flujo de control para ponerlo en cómo se interconectan los objetos.	
Chain of Responsibility	Evitar acoplar el emisor de una petición a su receptor, dando a más de un objeto la posibilidad de manejarla ; se encadenan los receptores y la petición viaja por la cadena hasta que alguno la maneja.
Command	Encapsular una petición como un objeto , permitiendo parametrizar clientes con distintas peticiones, encolarlas o registrarlas (log), y soportar operaciones deshacibles .
Interpreter	Dado un lenguaje, definir una representación de su gramática junto con un intérprete que la usa para interpretar sentencias.
Iterator	Acceder secuencialmente a los elementos de un agregado sin exponer su representación subyacente.
Mediator	Definir un objeto que encapsula cómo interactúa un conjunto de objetos ; promueve acoplamiento débil evitando referencias explícitas mutuas, y permite variar la interacción independientemente.
Memento	Sin violar el encapsulamiento , capturar y externalizar el estado interno de un objeto, para poder restaurarlo más tarde.
Observer	Definir una dependencia uno-a-muchos entre objetos: cuando uno cambia de estado, todos sus dependientes son notificados y actualizados automáticamente .
State	Permitir que un objeto altere su conducta cuando cambia su estado interno ; el objeto parecerá cambiar de clase.
Strategy	Definir una familia de algoritmos , encapsular cada uno y hacerlos intercambiables ; el algoritmo varía independientemente de los clientes que lo usan.
Template Method	Definir el esqueleto de un algoritmo en una operación, difiriendo pasos a las subclases , que los redefinen sin cambiar la estructura del algoritmo.
Visitor	Representar una operación a realizar sobre los elementos de una estructura de objetos ; permite definir operaciones nuevas sin cambiar las clases de los elementos visitados.

42. Cómo los patrones resuelven los problemas de diseño

42.1 Encontrar los objetos y su granularidad

Lo difícil del diseño OO es **descomponer el sistema en objetos** (compiten encapsulamiento, granularidad, dependencia, flexibilidad, performance, evolución, reutilización...). El diseño estricto "modelo del mundo real" refleja el hoy pero no el mañana: **las abstracciones que emergen durante el diseño** — sin contraparte física — **son la clave de la flexibilidad**. Los patrones ayudan a encontrar esas abstracciones menos obvias: **Strategy** describe familias intercambiables de algoritmos-objeto; **State** representa cada estado como objeto — objetos que rara vez se descubren en el análisis. También resuelven la granularidad: **Facade** (un subsistema entero como objeto), **Flyweight** (montones de objetos finísimos), **Abstract Factory/Builder** (objetos cuya única responsabilidad es crear otros), **Visitor/Command** (objetos cuya única responsabilidad es implementar un pedido sobre otros).

42.2 Interfaces, clases y tipos

La **firma** de una operación = nombre + parámetros + retorno; la **interfaz** de un objeto = el conjunto de firmas de sus operaciones; un **tipo** = un nombre que denota una interfaz (un objeto puede tener muchos tipos; objetos de clases distintas pueden compartir tipo). El **dynamic binding** — la asociación en runtime del pedido con el objeto y su operación — permite sustituir en ejecución objetos de idéntica interfaz: el **polimorfismo**, concepto clave del OO. Distinción crítica: la **clase** de un objeto define su **implementación**; su **tipo**, sólo su interfaz. Igualmente: **herencia de clase** = compartir implementación/código; **herencia de interfaz (subtipado)** = cuándo un objeto puede usarse en lugar de otro (C++ y Eiffel confunden ambas; el subtipado puro se aproxima heredando de clases abstractas puras). Muchos patrones dependen de la distinción: los objetos de un Chain of Responsibility comparten **tipo**, no implementación; Command, Observer, State y Strategy suelen implementarse con clases abstractas puras.

Principio 1 del diseño OO reutilizable

"Programá contra una interfaz, no contra una implementación." No declarar variables como instancias de clases concretas: comprometerse sólo con la interfaz de una clase abstracta. Beneficios: los clientes ignoran los tipos concretos que usan (mientras respeten la interfaz) e ignoran las clases que los implementan. Los **patrones creacionales garantizan este principio**: abstraen el acto de crear, dando maneras de asociar interfaz e implementación de modo transparente en la instanciación.

42.3 Herencia vs. composición (y delegación)

Las dos técnicas de reutilización más comunes: la **herencia de clase** — reutilización **de caja blanca** (los internos del padre son visibles al hijo): definida estáticamente, directa en el lenguaje, fácil de modificar... pero **no cambia en runtime** y, peor, **"la herencia rompe el encapsulamiento"** — la implementación del hijo se liga tanto a la del padre que cualquier cambio del padre fuerza cambios en el hijo, limitando la flexibilidad y la reutilización (cura parcial: heredar sólo de clases abstractas). La **composición de objetos** — reutilización **de caja negra**: se obtiene funcionalidad nueva ensamblando objetos vía sus interfaces; definida **dinámicamente en runtime**, respeta el encapsulamiento (cualquier objeto es reemplazable por otro del mismo tipo), mantiene cada clase encapsulada y **enfocada en una tarea**, y las jerarquías permanecen pequeñas — a cambio, la conducta del sistema pasa a depender de las interrelaciones entre muchos objetos y no de una clase.

Principio 2 del diseño OO reutilizable

"Favorecé la composición de objetos por sobre la herencia de clases." La experiencia de los autores: los diseñadores **abusan de la herencia**; los diseños suelen volverse más reutilizables (y más simples) dependiendo más de la composición. Idealmente no habría que crear componentes nuevos: todo se lograría ensamblando los existentes — rara vez el conjunto es tan rico, y ahí la herencia ayuda a fabricar componentes nuevos combinables: **trabajan juntas**.

Delegación: composición tan potente como la herencia — dos objetos manejan el pedido: el receptor **delega** operaciones en su delegado, pasándose a sí mismo para que la operación delegada pueda referirlo (análogo al `this/self` de la herencia). Ejemplo: en vez de que Window **sea** un Rectangle (herencia), Window **tiene** un Rectangle y le delega el cálculo del área — y puede volverse circular en runtime reemplazándolo por un Circle del mismo tipo. Ventaja: componer conductas en ejecución; desventaja: el software dinámico y muy parametrizado es **más difícil de entender** que el estático (usarla en formas estilizadas: los patrones). Dependen fuertemente de la delegación: State, Strategy, Visitor; en menor medida: Mediator, Chain of Responsibility, Bridge. Tercera vía de reutilización: los **tipos parametrizados/genéricos/templates** (para parametrizar un sort por su comparación: subclase con Template Method, objeto Strategy, o parámetro de template — ni herencia ni templates cambian en runtime).

42.4 Estructura de compilación vs. de ejecución, y diseñar para el cambio

La estructura runtime de un programa OO **se parece poco a su estructura de código**: el código son clases con herencias fijas; la ejecución, redes cambiantes de objetos comunicantes ("intentar entender una desde la otra es como entender los ecosistemas vivos desde la taxonomía estática, y viceversa"). Ejemplo: **agregación** (un objeto es dueño/parte de otro; vidas idénticas) vs. **acquaintance/asociación** (sólo se conocen; relación más débil y dinámica) — **se implementan igual** (referencias) pero significan cosas muy distintas: las determina más la **intención** que el lenguaje. Los patrones (Composite, Decorator, Observer, Chain) capturan explícitamente la distinción compilación/ejecución.

La clave de la reutilización es anticipar el cambio — diseñar sistemas que puedan evolucionar. Las **ocho causas comunes de rediseño** y los patrones que las evitan:

Causa de rediseño	Patrones que la abordan
1. Crear un objeto nombrando su clase concreta (compromete con una implementación; crear indirectamente)	Abstract Factory, Factory Method, Prototype
2. Dependencia de operaciones específicas (una sola manera "hardcodeada" de satisfacer el pedido)	Chain of Responsibility, Command
3. Dependencia de la plataforma de hardware/software (APIs y SO diferentes)	Abstract Factory, Bridge
4. Dependencia de las representaciones o implementaciones de los objetos (clientes que saben cómo se almacena/ubica/implementa)	Abstract Factory, Bridge, Memento, Proxy
5. Dependencias algorítmicas (los algoritmos se extienden, optimizan y reemplazan; aislar los que cambiarán)	Builder, Iterator, Strategy, Template Method, Visitor

6. Acoplamiento fuerte (sistemas monolíticos, "masa densa" difícil de aprender, portar y mantener)	Abstract Factory, Bridge, Chain of Responsibility, Command, Facade, Mediator, Observer
7. Extender funcionalidad por subclasificación (costo fijo por clase; comprensión profunda del padre; explosión de subclases)	Bridge, Chain of Responsibility, Composite, Decorator, Observer, Strategy
8. Imposibilidad de alterar clases convenientemente (sin código fuente, o cambiar exigiría tocar muchas subclases)	Adapter, Decorator, Visitor

El rol de los patrones según el tipo de software: en **programas de aplicación** priorizan reutilización interna, mantenibilidad y extensión; en **toolkits** (bibliotecas de clases de propósito general — el equivalente OO de las bibliotecas de subrutinas; no imponen un diseño) evitan supuestos que limiten la flexibilidad; en **frameworks** — conjuntos de clases cooperantes que constituyen un **diseño reutilizable para una clase de software** — son críticos: el framework **dicta la arquitectura** de tu aplicación (particionado, responsabilidades, colaboraciones, hilo de control) y produce la **inversión de control**: "cuando usás un toolkit, escribís el cuerpo principal y llamás al código reutilizado; cuando usás un framework, **reutilizás el cuerpo principal y escribís el código que él llama**". Frameworks vs. patrones: los patrones son **más abstractos** (los frameworks se encarnan en código ejecutable; de los patrones sólo hay ejemplos), **elementos arquitectónicos más pequeños** (un framework típico contiene varios patrones; jamás al revés) y **menos especializados** (los frameworks tienen dominio; los patrones sirven en casi cualquier aplicación). Documentar un framework con sus patrones aplana la curva de aprendizaje.

42.5 Seleccionar y usar un patrón (y cuándo no)

Para encontrar el patrón: repasar cómo los patrones resuelven los problemas de diseño; escanear los intents; estudiar las interrelaciones entre patrones; estudiar los de propósito similar; examinar las causas de rediseño; y — el enfoque inverso — considerar **qué querés que pueda variar sin rediseño** (la Tabla 1.2 del GoF: Strategy varía *el algoritmo*; State, *los estados de un objeto*; Observer, *cuántos objetos dependen de otro*; Memento, *qué información privada se guarda fuera y cuándo*; Iterator, *cómo se recorre un agregado*; Facade, *la interfaz a un subsistema*; Adapter, *la interfaz a un objeto*; Bridge, *su implementación...*). **Para usarlo:** leerlo completo una vez (Aplicabilidad y Consecuencias primero); estudiar Estructura/Participantes/Colaboraciones; mirar el código de ejemplo; **nombrar los participantes con sentido de la aplicación** (SimpleLayoutStrategy, TeXLayoutStrategy); definir las clases; nombrar las operaciones consistentemente (prefijo "Create-" para factory methods); implementar.

Advertencia final del GoF

Los patrones no deben aplicarse indiscriminadamente. Suelen lograr su flexibilidad introduciendo **niveles adicionales de indirección**, lo que complica el diseño y/o cuesta performance. Un patrón sólo debe aplicarse **cuando la flexibilidad que otorga es realmente necesaria**. Las secciones de Consecuencias son la mejor guía para evaluar beneficios y pasivos.

43. Caso de estudio: el editor Lexi

El capítulo 2 del GoF diseña **Lexi**, un editor de documentos WYSIWYG (basado en el editor Doc de Calder), y enseña **ocho patrones por el ejemplo**, a partir de siete problemas de diseño:

Problema de diseño en Lexi	Patrón que lo resuelve
1. Estructura del documento: la representación interna (caracteres, líneas, polígonos, filas, columnas...) afecta todo el diseño; el usuario manipula subestructuras (una figura como unidad) que anidan recursivamente.	Composite — la jerarquía parte-todo como árbol de glifos tratados uniformemente.
2. Formateo: ¿qué objetos arman líneas y columnas? ¿cómo conviven distintas políticas de formateo (con distintos tradeoffs calidad/velocidad)?	Strategy — cada algoritmo de composición encapsulado e intercambiable.
3. Embellecer la UI: bordes, scrollbars y sombras que van y vienen con la evolución de la interfaz, sin tocar el resto.	Decorator — responsabilidades visuales agregadas dinámicamente sin subclassificar.
4. Múltiples estándares de look-and-feel (Motif, Presentation Manager) sin modificación mayor.	Abstract Factory — crear la familia completa de widgets de cada estándar sin nombrar clases concretas.
5. Múltiples sistemas de ventanas: independencia de la plataforma de ventanas.	Bridge — la abstracción Window separada de sus implementaciones por plataforma, variando ambas independientemente.
6. Operaciones de usuario: la funcionalidad tras botones y menús está desparramada por todo el sistema; se necesita un mecanismo uniforme de acceso... y de undo .	Command — cada pedido como objeto, con historial para deshacer/rehacer.
7a. Recorrer la estructura del documento para analizarla sin exponer su representación.	Iterator — acceso y recorrido abstraídos.
7b. Análisis (ortografía, guionado — y los que vengan) sin modificar las clases del documento ni ensuciar su implementación.	Visitor — número abierto de capacidades analíticas fuera de las clases de la estructura.

Figura 43.1 — Los siete problemas de Lexi y los ocho patrones aplicados (GoF cap. 2).

Ninguno de estos problemas es exclusivo de un editor: "una aplicación de análisis financiero usaría Composite para carteras de subcarteras; un compilador usaría Strategy para esquemas de asignación de registros; cualquier aplicación gráfica probablemente aplique al menos Decorator y Command tal como aquí".

44. Comparaciones y discusión de los patrones

44.1 Discusión de los creacionales

Dos maneras de parametrizar un sistema por las clases de objetos que crea: (a) **subclasificar la clase creadora** — Factory Method; inconveniente: puede requerir una subclase nueva sólo para cambiar el producto, y los cambios **cascadean**; (b) **composición**: definir un "objeto fábrica" que conozca la clase de los productos y hacerlo parámetro del sistema — es el corazón de **Abstract Factory** (fábrica que produce objetos de varias clases), **Builder** (fábrica que construye un producto complejo incrementalmente con un protocolo correspondiente) y **Prototype** (la fábrica y el prototipo son **el mismo objeto**: el prototipo retorna el producto clonándose). Guía del ejemplo del editor de dibujo (parametrizar GraphicTool por la clase de Graphic): Factory Method es lo más simple al comienzo pero prolifera subclases que hacen poco; Abstract Factory no mejora mucho (jerarquía paralela de fábricas); **Prototype suele ser lo mejor** ahí (sólo exige implementar Clone, reutilizable además para duplicar). En general: los diseños **empiezan con Factory Method y evolucionan** hacia los demás creacionales cuando se descubre dónde hace falta más flexibilidad; conocer muchos patrones da más opciones de tradeoff.

44.2 Discusión de los estructurales

Se parecen entre sí porque dependen del mismo pequeño conjunto de mecanismos (herencia simple/múltiple para los de clase, composición para los de objeto); difieren en sus **intents**:

- **Adapter vs. Bridge**: ambos dan un nivel de indirección y reenvían pedidos; pero Adapter **resuelve incompatibilidades entre dos interfaces existentes** — acoplamiento imprevisto, generalmente para no reimplementar; Bridge **conecta una abstracción con sus muchas implementaciones**, previsto de antemano, dejándolas evolucionar por separado. "Adapter hace que las cosas funcionen **después** de diseñadas; Bridge, **antes**." Y Facade no es "un adaptador de un conjunto de objetos": **Facade define una interfaz nueva; Adapter reutiliza una vieja**.
- **Composite vs. Decorator vs. Proxy**: Composite y Decorator comparten los diagramas (ambos usan **composición recursiva** para organizar un número abierto de objetos), pero el Decorator **agrega responsabilidades sin subclasificar** (evita la explosión combinatoria de subclases) mientras Composite **representa jerarquías** tratando muchos objetos como uno: "su foco no es el embellecimiento sino la representación". Son complementarios y suelen usarse juntos (visto desde Decorator, un composite es un ConcreteComponent; visto desde Composite, un decorator es un Leaf). Proxy también compone un objeto ofreciendo la misma interfaz, pero su intent es **ser sustituto para controlar el acceso** (objeto remoto, carga bajo demanda, protección): en Proxy el sujeto define la funcionalidad clave y el proxy da (o niega) acceso; en Decorator el componente da sólo parte de la funcionalidad y los decoradores el resto; la relación proxy-sujeto es estática, no recursiva.

44.3 Discusión de los de comportamiento

- **Encapsular la variación**: cuando un aspecto cambia con frecuencia, un objeto lo encapsula y el patrón se llama como él — **Strategy** encapsula *un algoritmo*, **State** *una conducta dependiente del estado*, **Mediator** *el protocolo entre objetos*, **Iterator** *la forma de acceder y recorrer un agregado*. (El tema corre también por Abstract Factory/Builder/Prototype — encapsulan el conocimiento de la creación —, Decorator — la responsabilidad agregable —, Bridge — separa abstracción de implementación.)

- **Objetos como argumentos:** el **Visitor** es el argumento de la operación polimórfica **Accept**; **Command** y **Memento** son "tokens mágicos" para pasar e invocar después — en Command el token representa un pedido (con ejecución polimórfica); en Memento, el estado interno de un objeto en un momento (interfaz tan estrecha que se pasa sólo como valor, sin polimorfismo).
- **¿Comunicación encapsulada o distribuida? Mediator y Observer son competidores:** Observer **distribuye** la comunicación introduciendo Subject y Observer (más reutilizables, colaboración por interconexión: un sujeto con muchos observadores; más difícil de seguir el flujo); Mediator la **centraliza** (la responsabilidad de mantener la restricción vive en el mediador: flujo más fácil de entender, objetos menos reutilizables; sus dispatchs ad hoc pueden sacrificar type safety). En Smalltalk los observadores parametrizables son aún más reutilizables — el Smalltalkero usa Observer donde el C++ero usaría Mediator.
- **Desacoplar emisores de receptores:** **Command** (el objeto Command define la conexión emisor-receptor con una interfaz simple — Execute); **Observer** (interfaz para señalar cambios; el binding más laxo — número variable de observadores en runtime; ideal cuando el desacople responde a *dependencias de datos*); **Mediator** (referencias indirectas a través del mediador, que rutea y centraliza); **Chain of Responsibility** (el pedido viaja por una cadena de receptores potenciales — ideal si la cadena ya es parte de la estructura del sistema, y la cadena se cambia o extiende fácil).
- **Los de comportamiento se complementan:** una clase de una Chain suele usar Template Method; la cadena puede representar los pedidos con Command; Interpreter puede usar State; un Iterator recorre un agregado y un Visitor aplica la operación a cada elemento; un sistema con Composite puede usar Visitor para operar la composición, Chain para que los componentes accedan a propiedades globales vía su padre, Decorator para redefinirlas en partes, Observer para atar estructuras y State para variar la conducta; la composición misma puede crearse con Builder y ser un Prototype para otra parte del sistema. "Los sistemas OO bien diseñados son exactamente así: **múltiples patrones incrustados** — la composición en el nivel de los patrones logra una sinergia que costaría lograr clase a clase."

44.4 Qué esperar de los patrones (conclusión del GoF)

- **Un vocabulario común de diseño:** los expertos no piensan en sintaxis sino en estructuras conceptuales mayores; los patrones permiten decir "acá usemos un Observer" y diseñar (y discutir el diseño) a mayor nivel de abstracción.
- **Ayuda de documentación y aprendizaje:** la mayoría de los sistemas OO grandes usa estos patrones — quien los conoce entiende el sistema mucho antes ("la queja del novato — herencia enrevesada, flujo imposible de seguir — suele ser no conocer los patrones"); describir un sistema por sus patrones evita la ingeniería inversa del diseño.
- **Complemento de los métodos de análisis y diseño:** los patrones muestran cómo usar las primitivas (objetos, herencia, polimorfismo) y capturan **el porqué** del diseño (Aplicabilidad, Consecuencias); son especialmente útiles para **convertir el modelo de análisis en modelo de implementación** — la transición nunca es suave y el diseño reutilizable contiene objetos que no están en el análisis.
- **Blanco para el refactoring:** ciclo de vida en tres fases (Foote): **prototipado** (jerarquías que espejan el dominio; reutilización de caja blanca) → **expansión** (más requerimientos, clases y operaciones sin relación) → **consolidación/refactoring** (separar componentes generales de específicos, subir/bajar operaciones, racionalizar interfaces; emergen los frameworks; la composición reemplaza a la herencia — **caja negra reemplaza caja blanca**). Los patrones capturan muchas estructuras que resultan de refactorizar: **aplicados temprano, previenen refactorings; descubiertos tarde, indican cómo hacerlos.**

*Datos históricos del catálogo: nació en la tesis doctoral de Erich Gamma (1991-92); en OOPSLA'91-'92 se sumaron Helm, Vlissides y Johnson; los nombres evolucionaron — "Wrapper"→**Decorator**, "Glue"→**Facade**, "Solitaire"→**Singleton**, "Walker"→**Visitor**. "Notar que algo es un patrón es la parte fácil; lo difícil es describirlo para que quien no lo conoce lo entienda y sepa por qué importa."*

45. Los patrones hoy: la lectura de *Refactoring.Guru*

Fuente de este capítulo: refactoring.guru/design-patterns (Alexander Shvets), la referencia contemporánea más difundida sobre el catálogo GoF. Se incluye porque reformula los mismos conceptos con un vocabulario más accesible, agrega el **encuadre por niveles** y — sobre todo — sistematiza las **críticas** a los patrones, tema que el libro de 1995 apenas roza.

45.1 La definición y la analogía del plano

Definición (Refactoring.Guru)

"Los patrones de diseño son **soluciones típicas a problemas que ocurren comúnmente en el diseño de software**. Son como **planos prefabricados** que podés personalizar para resolver un problema de diseño recurrente en tu código." Advertencia inmediata: **un patrón no se copia como se copia una librería** — es un concepto general que requiere adaptación a cada implementación.

Idea clave — patrón ≠ algoritmo

"Una analogía de un **algoritmo** es una **receta de cocina**: ambos tienen pasos claros para alcanzar una meta. Un **patrón**, en cambio, es más parecido a un **plano (blueprint)**: podés ver cuál es el resultado y sus características, pero **el orden exacto de la implementación queda a tu criterio**." Es la misma idea que el GoF expresa al decir que el patrón "es como una plantilla que puede aplicarse en muchas situaciones distintas" y que "no describe un diseño o implementación concretos".

45.2 De qué partes consta la descripción de un patrón

Sección	Qué aporta (Refactoring.Guru)	Equivalente en el GoF
Intent (propósito)	"Describe brevemente tanto el problema como la solución ."	<i>Intent</i> — idéntico; es la sección que se escanea para buscar un patrón.
Motivation (motivación)	"Explica con más detalle el problema y la solución."	<i>Motivation</i> — el escenario que ilustra el problema y cómo lo resuelven las estructuras.
Structure (estructura)	"De clases: muestra cada parte del patrón y cómo se relacionan."	<i>Structure</i> + <i>Participants</i> + <i>Collaborations</i> .
Code example	"En uno de los lenguajes de programación populares, hace más fácil captar la idea."	<i>Sample Code</i> (C++ y Smalltalk en el original; hoy Java, C#, Python, TypeScript, Go, PHP, Rust, Swift...).

Comparado con la plantilla de 13 secciones del GoF, esta versión reducida **omite** Aplicabilidad, Consecuencias, Implementación, Known Uses y Related Patterns — que son justamente las que el GoF señala como más valiosas para **decidir** si aplicar el patrón. Buena práctica de estudio: usar Refactoring.Guru para entender *qué* hace el patrón y volver al GoF para saber *si conviene* aplicarlo.

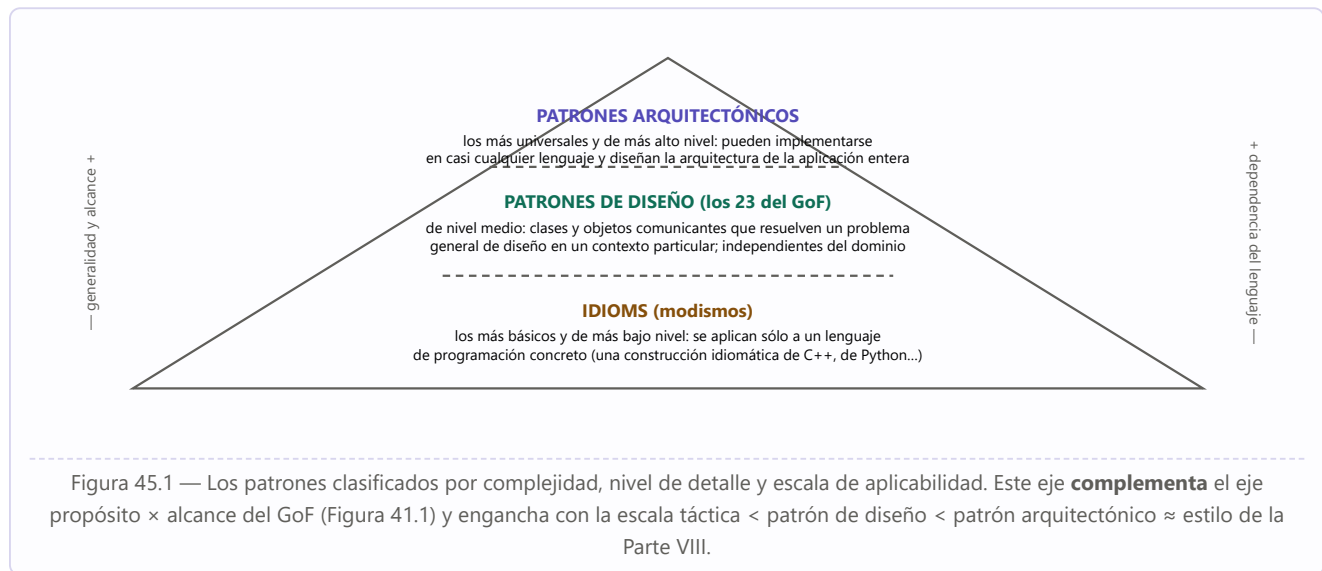
45.3 La historia, en su versión canónica breve

"El concepto de patrones fue descrito primero por **Christopher Alexander** en *A Pattern Language: Towns, Buildings, Construction*", obra que describía "un **lenguaje** para diseñar el entorno urbano", con patrones que iban desde la altura de las ventanas hasta la cantidad de niveles de un edificio. Cuatro ingenieros de software —

Erich Gamma, John Vlissides, Ralph Johnson y Richard Helm — adaptaron el concepto a la programación y en **1994** publicaron *Design Patterns: Elements of Reusable Object-Oriented Software*, con **23 patrones** que resolvían "varios problemas del diseño orientado a objetos"; se volvió best-seller rápidamente y, por lo largo del título, pasó a conocerse como "**el libro GoF**" (*Gang of Four*, la Banda de los Cuatro). Desde entonces "se han descubierto docenas de otros patrones orientados a objetos" y el enfoque se extendió a otros campos de la programación.

Nota de consistencia entre fuentes: Refactoring.Guru fecha el libro en 1994 (año del copyright de Addison-Wesley) y Reynoso lo cita como 1995 (año de la edición difundida); el propio prefacio del GoF relata que el catálogo se volvió independiente en OOPSLA'91 y que la decisión de convertirlo en libro llegó tras ECOOP'93.

45.4 Los tres niveles: idioms, patrones de diseño y patrones arquitectónicos



45.5 Los tres propósitos, reformulados

Grupo	Formulación de Refactoring.Guru
Creacionales	"Proveen diversos mecanismos de creación de objetos , que aumentan la flexibilidad y la reutilización del código existente." (<i>Factory Method, Abstract Factory, Builder, Prototype, Singleton</i>)
Estructurales	"Explican cómo ensamblar objetos y clases en estructuras más grandes , manteniendo esas estructuras flexibles y eficientes." (<i>Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy</i>)
De comportamiento	"Se ocupan de los algoritmos y de la asignación de responsabilidades entre objetos." (<i>Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor</i>)

Detalle no trivial: Refactoring.Guru lista **10** patrones de comportamiento y no incluye **Interpreter** en su catálogo principal, mientras el GoF lista 11 incluyéndolo. Es un buen recordatorio de que **los catálogos son productos históricos y editoriales**, no taxonomías cerradas: el propio GoF admite que "un par de patrones se dejaron caer porque no parecían suficientemente importantes" y que los nombres cambiaron en el camino (Wrapper→Decorator, Glue→Facade, Solitaire→Singleton, Walker→Visitor).

45.6 Por qué aprenderlos

- **Un kit de herramientas probadas.** "Los patrones de diseño son un **kit de soluciones probadas y testeadas** a problemas comunes del diseño de software." Y el beneficio secundario, más importante que el catálogo mismo: aprenderlos **enseña los principios del diseño orientado a objetos**, aplicables a problemas que ningún patrón cubre. (Es exactamente el argumento del GoF sobre "programá contra una interfaz" y "composición sobre herencia": los principios son lo que queda cuando el catálogo envejece.)
- **Un lenguaje común.** "Los patrones de diseño definen un **lenguaje común** que vos y tus compañeros de equipo pueden usar para comunicarse más eficientemente": alcanza con nombrar el patrón, sin explicaciones largas. El GoF lo formula igual — "tu vocabulario de diseño cambiará: te vas a encontrar diciendo *usamos un Observer acá o hacemos un Strategy con estas clases*" — y el SAIP lo eleva a razón n.º 12 para la arquitectura entera: **restringir voluntariamente el vocabulario de alternativas a soluciones probadas** es lo que convierte al diseño en disciplina de ingeniería.

45.7 Las tres críticas a los patrones

Cuidado — el lado oscuro del catálogo

1. Son parches para un lenguaje de programación débil. Los patrones aparecen cuando los desarrolladores eligen un lenguaje que carece del **nivel de abstracción necesario**: se vuelven *workarounds* que compensan las limitaciones del lenguaje. Ejemplo canónico: **el patrón Strategy puede reemplazarse por simples funciones anónimas** (lambdas) en lenguajes modernos. *Nótese que el GoF ya lo anticipaba: "si hubiéramos asumido lenguajes procedurales, podríamos haber incluido patrones llamados Herencia, Encapsulamiento y Polimorfismo"; y "CLOS tiene multi-métodos, lo cual disminuye la necesidad de un patrón como Visitor".*

2. Producen soluciones ineficientes. Los practicantes suelen implementar los patrones **"al pie de la letra, sin adaptarlos al contexto de su proyecto"**, tomando la estandarización como dogma en lugar de como guía. *El GoF insiste en lo contrario: el patrón es una plantilla, y hay que nombrar a sus participantes con los nombres del dominio de la aplicación.*

3. Uso injustificado. Los novatos los sobre-aplican indiscriminadamente: **"si todo lo que tenés es un martillo, todo parece un clavo"** — aplicar patrones donde código más simple alcanzaría. *Coincide literalmente con la advertencia del GoF ("los patrones no deben aplicarse indiscriminadamente: suelen lograr flexibilidad introduciendo niveles adicionales de indirección, y eso complica el diseño y cuesta performance; un patrón sólo debe aplicarse cuando la flexibilidad que otorga es realmente necesaria") y con la misma frase que el SAIP usa para criticar el diseño por costumbre: "si todo lo que tenés es un martillo, todo el mundo parece un clavo".*

Síntesis para el estudiante

Las tres críticas no invalidan el catálogo: **lo relocalizan**. Un patrón no es una meta ("usemos Visitor") sino una respuesta a una fuerza concreta del problema; el criterio para aplicarlo es idéntico al criterio arquitectónico de todo este documento — **¿qué atributo de calidad mejora, a costa de qué otro, y necesito realmente esa flexibilidad?** El GoF lo resuelve con la sección Consecuencias, el SAIP con el análisis de tradeoffs del ATAM, y Reynoso con el "menos es más" de los estilos. Es la misma prudencia, en tres escalas.

PARTE VIII

Síntesis integradora

Las correlaciones entre las tres fuentes: cómo Reynoso, SAIP y el GoF cuentan — a distintas escalas y desde distintas épocas — la misma historia sobre estructura, abstracción y reutilización.

46 Correlaciones entre las tres fuentes

47 Glosario esencial

46. Correlaciones entre las tres fuentes

46.1 Una sola línea de tiempo

Los tres documentos comparten los mismos hitos fundacionales. Reynoso narra la génesis (Dijkstra 1968 → Parnas 1972 → Brooks 1975 → Perry & Wolf 1992 → escuela de CMU → GoF 1995/POSA 1996 → IEEE 1471/REST 2000 → métodos del SEI); el **SAIP es el producto maduro de esa historia**: sus autores (Bass, Clements, Kazman) son los protagonistas de la "arquitectura procesual" que Reynoso describe como quinta corriente, y su sección de lecturas recomendadas remite exactamente a los mismos clásicos (Dijkstra 68 sobre THE y las capas; Parnas 72 sobre information hiding; Parnas 76 sobre familias de programas; Shaw & Garlan como creadores del campo). El **GoF, a su vez, es uno de los "dos textos fundamentales" del movimiento de patrones** que Reynoso ubica en los 90, con Christopher Alexander como origen común de toda la metáfora.

46.2 La escala: táctica < patrón de diseño < patrón arquitectónico ≈ estilo



46.3 Tabla maestra de correlaciones

Concepto	Reynoso (2004)	SAIP 4.ª ed.	GoF (1995)
Definición de arquitectura	IEEE 1471-2000: "organización fundamental... componentes, relaciones, principios de diseño y evolución" (la "oficial").	"Conjunto de estructuras necesarias para razonar sobre el sistema: elementos, relaciones y propiedades" — heredera directa de la anterior, con el acento en el <i>razonamiento</i> .	(Fuera de su alcance: opera en el nivel del diseño OO; los frameworks "dictan la arquitectura".)
Arquitectura vs. diseño	Perry 97: componentes/interacciones/restricciones vs. algoritmos/procedimientos/tipos; grano grueso vs. fino. TM00: nivel de abstracción superior.	"La arquitectura es diseño, pero no todo diseño es arquitectura"; test: si la decisión puede posponerse al bajo nivel, no es arquitectónica.	Su territorio es exactamente el que Perry excluye de la arquitectura: clases, algoritmos, colaboraciones — el "diseño del código" de Shaw & Garlan.

Ocultamiento / encapsulamiento	Parnas 72: information hiding, módulos- caja-negra, decisiones de diseño ocultas — "el introductor de las nociones más esenciales".	Encapsular = táctica-madre de modificabilidad e integrabilidad; recomendación estructural n.º 1 (módulos por information hiding).	"Programá contra una interfaz, no contra una implementación"; la herencia rompe el encapsulamiento → composición.
Vistas / separación de incumbencias	Historia y catálogo de marcos (4+1, Zachman, TOGAF, RM-ODP, IEEE 1471: view vs. viewpoint); origen en Ross 1977.	Las tres categorías de estructuras + quality views; "documentar = documentar vistas"; ISO/IEC/IEEE 42010 (sucesor del 1471).	MVC: la separación modelo/vista/controlador como separación de incumbencias en el nivel del diseño.
MVC	Vistas Conceptual/Lógica/Física de Microsoft "familiares desde los días de OMT".	Patrón de usabilidad (cap. 18): variantes MVP/MVVM/MVA; con Observer en un proceso, pub- sub distribuido.	Origen: Smalltalk-80; descompuesto en Observer + Composite + Strategy (+ Factory Method + Decorator).
Escenarios	Noción emergente ("guión/libreto"); casos de uso + casos de cambio; base de SAAM/ATAM/ALMA; sexta corriente (holandesa).	EL instrumento central: escenario de 6 partes, escenarios generales y concretos, utility tree, QAW, análisis del ATAM.	(El "escenario" del GoF es la sección Motivación de cada patrón: un problema ilustrado.)
Evaluación	Cataloga los métodos del SEI (SAAM, ATAM, CBAM, ARID...); la "puerta de peaje" de SAAM.	Desarrolla ATAM completo (fases, 9 pasos, riesgos/no-riesgos, sensibilidades, tradeoffs) + LAE + cuestionarios de tácticas.	(Las "Consecuencias" de cada patrón son su micro-evaluación de tradeoffs.)
Reutilización	Familias de programas (Parnas) → estilos como encarnación de la reusabilidad; "la idea dominante" compartida con los patrones.	Razón n.º 10 (modelo transferible, líneas de producto) y n.º 12 (vocabulario restringido de soluciones probadas: tácticas y patrones).	Todo el libro: capturar la experiencia como patrones; frameworks = reutilización de diseño; refactoring hacia los patrones.
"Menos es más"	Los estilos se identifican a grandes rasgos; la restricción a pocas elecciones da reutilización, análisis e interoperabilidad (CN96).	Integridad conceptual ("lo mismo, igual, en todas partes"); pocos patrones de interacción simples; "fewer is better" en vistas.	No aplicar patrones indiscriminadamente: la indirección cuesta; sólo cuando la flexibilidad es necesaria.
El cambio como eje	Las "decisiones tempranas" de Parnas = las que permanecen; la evolución como virtud n.º 5 de la AS.	Modificabilidad: qué/cuán probable/cuándo-quién/cuánto cuesta; cambios locales, no locales y arquitectónicos; deuda arquitectónica.	"La clave de la reutilización es anticipar los requerimientos nuevos": las 8 causas de rediseño y qué patrón previene cada una.
Conectores	De primera clase en los estilos y ADLs (lo que UML no expresa); tabla de Bosch: la industria no los tiene.	Mitad de las estructuras C&C: pueden ser complejos y n-arios (un bus entero); attachment como relación.	(Implícitos: las "colaboraciones" y los objetos intermediarios — Mediator, Observer — son conectores en miniatura.)

46.4 De la táctica al patrón: el puente SAIP ↔ GoF

El SAIP cita explícitamente al GoF (su cap. de integrabilidad remite a "Gamma 94" para bridge, wrapper y adapter) y varios de sus patrones de QA **son** patrones GoF puestos a trabajar para un atributo:

Táctica (SAIP)	Patrón que la realiza	Origen GoF
Undo (usabilidad)	Memento	Memento (comportamiento): capturar y restaurar estado sin violar encapsulamiento.
Usar intermediario / notificación de cambios	Observer (MVC en un proceso)	Observer: dependencia uno-a-muchos con notificación automática.
Defer binding — polimorfismo / sustitución en runtime	Strategy, State	Strategy/State: algoritmo y conducta encapsulados e intercambiables.
Controlar/observar estado (testabilidad)	Dependency injection, Strategy, intercepting filter	La inversión de control es la esencia de los frameworks según el GoF.
Tailor interface / abstraer servicios comunes (integrabilidad)	Wrapper, Bridge, Mediator	Adapter (interfaces existentes incompatibles), Bridge (abstracción/implementación), Mediator (protocolo centralizado en runtime).
Encapsular pedidos, encolar, loggear	—	Command: la base conceptual de colas de trabajo y operaciones deshacibles.
Restringir dependencias (capas)	Layers	Facade: la interfaz unificada de un subsistema es la puerta de cada capa.

46.5 Lo que cada fuente aporta al estudiante

Cómo usar las tres fuentes juntas

Reynoso da el mapa histórico y conceptual: de dónde viene la disciplina, por qué sus términos significan lo que significan, qué corrientes existen y qué problemas siguen abiertos — el "porqué" de todo lo demás. **SAIP** da la ingeniería practicable: cómo especificar (escenarios), diseñar (tácticas, patrones, ADD), evaluar (ATAM), documentar (vistas) y sostener (deuda, competencia) una arquitectura real. **GoF** da la caja de herramientas del nivel inmediatamente inferior: cuando el ADD llega al paso 4-5 y hay que instanciar los conceptos de diseño en clases y objetos que colaboran, los 23 patrones y sus dos principios (interfaz sobre implementación; composición sobre herencia) son el vocabulario con que se escribe la solución. La cadena completa: **meta de negocio** → **ASR/escenario** → **decisión arquitectónica (táctica)** → **patrón arquitectónico/estilo** → **patrón de diseño** → **código**.

47. Glosario esencial

Término	Definición operativa (fuente)
Arquitectura de software	Conjunto de estructuras necesarias para razonar sobre el sistema: elementos de software, relaciones y propiedades de ambos (SAIP). / Organización fundamental encarnada en componentes, relaciones y principios de diseño y evolución (IEEE 1471, en Reynoso).
Estructura / vista	Estructura: conjunto de elementos unidos por una relación (existe en el sistema). Vista: <i>representación</i> de una o más estructuras (existe en la documentación) (SAIP).
Estilo arquitectónico	Forma de articulación u organización arquitectónica: componentes + conectores + configuraciones + restricciones (Perry & Wolf / Reynoso).
ADL	Lenguaje de descripción arquitectónica: modela y analiza la arquitectura antes de programar (Reynoso; SAIP los ubica entre las notaciones formales, "raras en la práctica").
Atributo de calidad (QA)	Propiedad medible o testeable que indica cuán bien el sistema satisface las necesidades más allá de la función básica (SAIP).
Escenario de QA	Especificación testeable de 6 partes: fuente → estímulo → artefacto → entorno → respuesta → medida (SAIP).
Táctica	Decisión de diseño que influye en el logro de la respuesta de un atributo de calidad (SAIP).
Patrón (arquitectónico)	Solución probada a un problema recurrente en un contexto; describe roles, responsabilidades y colaboraciones; empaqueta tácticas y tradeoffs (SAIP).
Patrón de diseño	Descripción de clases y objetos comunicantes, personalizada para resolver un problema general de diseño en un contexto particular; nombre + problema + solución + consecuencias (GoF).
ASR	Requerimiento con efecto profundo en la arquitectura y alto valor de negocio (SAIP).
Utility tree	Árbol Utility → QAs → refinamientos → escenarios calificados por (valor de negocio, riesgo técnico) (SAIP).
ADD	Attribute-Driven Design: diseño por rondas e iteraciones de 7 pasos guiadas por drivers (SAIP).
ATAM	Architecture Tradeoff Analysis Method: evaluación por escenarios con equipo externo; produce riesgos, no-riesgos, puntos de sensibilidad y de tradeoff (SAIP; catalogado por Reynoso).
Punto de sensibilidad / de tradeoff	Decisión con efecto marcado sobre una respuesta de QA / decisión a la que ≥ 2 QAs son sensibles en direcciones opuestas (SAIP).
Falta / error / falla	Causa → estado anómalo intermedio → desviación observable de la especificación (SAIP, disponibilidad).
MTBF / MTTR	Tiempo medio entre fallas / de reparación; disponibilidad = $MTBF/(MTBF+MTTR)$ (SAIP).
Cohesión / acoplamiento	Unidad de propósito de un módulo / probabilidad de propagación de cambios entre módulos (SAIP, modificabilidad; los términos datan de los 60).
Information hiding	Modularizar por decisiones de diseño ocultas: cada módulo, una caja negra con interfaz estable (Parnas 1972, vía Reynoso y SAIP).
Inversión de control	El framework reutiliza el cuerpo principal y llama al código que escribís (GoF); base de dependency injection (SAIP, testabilidad).
Deuda arquitectónica	Entropía del diseño: anti-patrones estructurales/evolutivos (cliques, violaciones de modularidad, herencia malsana...) que concentran bugs y churn; se detecta con DSMs y se paga refactorizando (SAIP).

DSM	Design structure matrix: archivos × archivos con dependencias estructurales y de co-cambio; sana = rala y triangular inferior (SAIP).
Sistema esquelético / walking skeleton	Infraestructura mínima de punta a punta sobre la que se crece incrementalmente (SAIP, razón 8).
Spike	Tarea time-boxed en rama separada para responder una pregunta técnica (SAIP, Agile).
Blue/green · rolling upgrade · canary	Patrones de despliegue: conmutación total con 2N instancias · reemplazo gradual N+1 · prueba en producción con usuarios seleccionados (SAIP, deployabilidad).
Región / availability zone	Agrupamientos geográfico-lógicos de data centers del proveedor de nube; zonas con fallas independientes (SAIP, nube).
Long tail latency	Cola derecha de la distribución de latencias: ~5% de requests hasta 5–10× el promedio; mitigada con hedged/alternative requests (SAIP).
Delegación	Composición donde el receptor delega la operación pasándose a sí mismo al delegado (GoF).
Caja blanca / caja negra	Reutilización por herencia (internos visibles) / por composición (sólo interfaces) (GoF).
Framework vs. toolkit	Diseño reutilizable que dicta la arquitectura e invierte el control / biblioteca de clases de propósito general que no la dicta (GoF).

Cierre

*Tres épocas, una misma convicción: la de Parnas en 1972 ("la estructura es primordial"), la de Perry y Wolf en 1992 ("la década de la arquitectura"), la del GoF en 1995 ("capturar la experiencia en forma utilizable") y la del SAIP hoy ("estructuras para razonar"). Del mismo modo que el tour de force de Shaw y Garlan demostró — cuatro estilos comparados sin escribir una línea de código — que Dijkstra tenía razón, este recorrido muestra que la historia, la ingeniería de los atributos de calidad y el catálogo de patrones no son tres materias: son **tres niveles de zoom sobre el mismo objeto de estudio**.*

PARTE IX

Catálogo de estilos arquitectónicos

Los seis grandes estilos, uno por uno: componentes, conectores, restricciones, cuándo usarlos, qué atributos de calidad favorecen y cuáles sacrifican — con el patrón equivalente del SAIP y ejemplos reales.

-
- 48 Flujo de datos: tubería y filtro, y batch secuencial
 - 49 Centrados en datos: repositorio y pizarra
 - 50 Jerárquicos: capas y microkernel
 - 51 Invocación implícita: publish-subscribe y broker
 - 52 Cliente-servidor y peer-to-peer
 - 53 Intérprete y máquina virtual
 - 54 Comparación, composición y el caso KWIC

Nota de alcance — de dónde sale este catálogo

El documento de Reynoso incluido en la carpeta de la materia cubre **los puntos 1.1 a 1.9** de su temario: **enumera** los estilos y define qué es un estilo, pero su **inventario y descripción** son el **capítulo 2 de su serie, publicado como documento aparte que no está en AYD_2** ("Hemos tratado el tema de las definiciones, los catálogos de estilos, las propiedades de cada uno [...] en un documento separado"). Por eso esta parte se construye con tres fuentes, siempre identificadas: **(a)** lo que Reynoso sí dice — la definición de estilo, la lista de los seis, la tesis de la composición y el caso KWIC; **(b)** los **patrones del SAIP** que desarrollan varios de estos estilos con beneficios y tradeoffs (capas, cliente-servidor, publish-subscribe, plugin, microservicios, SOA, map-reduce, spares); **(c)** la **literatura clásica** del campo (Shaw & Garlan; la serie POSA de Buschmann) para los estilos que ninguno de los dos documentos desarrolla — tubería y filtro, repositorio y pizarra, peer-to-peer, intérprete y máquina virtual —, señalada como *[clásico]*.

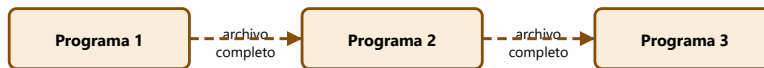
48. Flujo de datos: tubería y filtro, y batch secuencial

TUBERÍA Y FILTRO (pipes and filters)



Cada filtro **no conoce a sus vecinos**, no comparte estado y consume/produce un **stream**: se pueden reordenar y correr en paralelo.

BATCH SECUENCIAL (el caso degenerado)

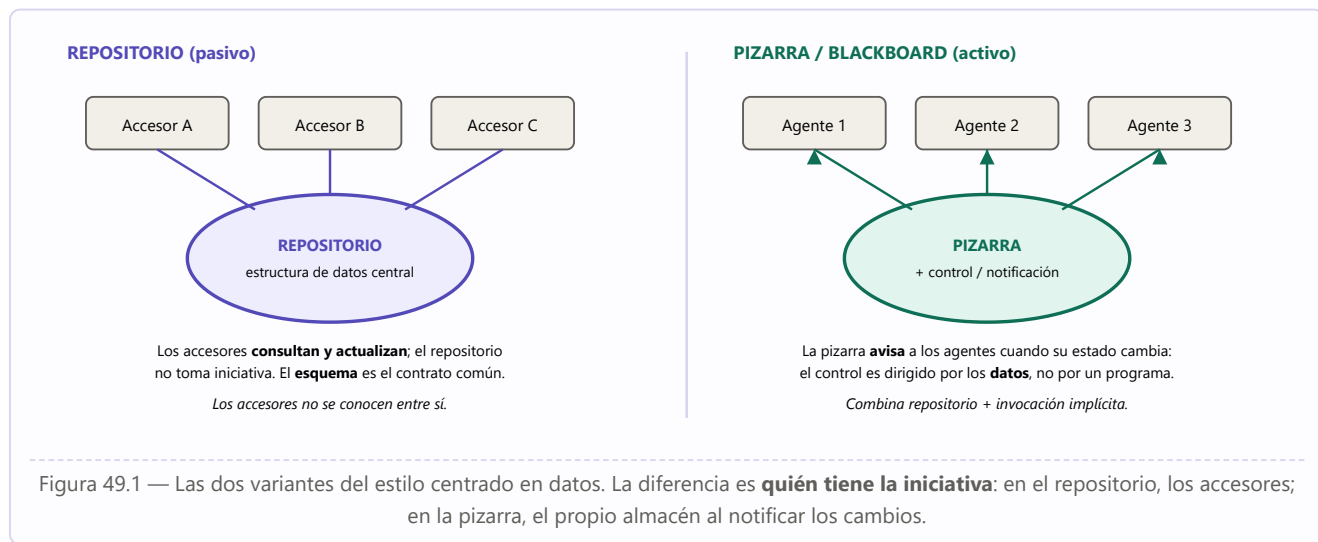


Sin concurrencia: cada etapa espera a que la anterior **termine del todo**. Es el estilo de los procesos nocturnos de mainframe.

Figura 48.1 — Tubería y filtro frente a batch secuencial: mismo esquema de transformación en cadena, pero con streams incrementales y concurrencia en el primero, y archivos completos sin solapamiento en el segundo.

Aspecto	Tubería y filtro <i>[clásico]</i>
Componentes	Filtros : transforman un flujo de datos de entrada en un flujo de salida, de forma incremental (empiezan a producir antes de terminar de consumir). Casos especiales: <i>fuentes</i> (solo produce) y <i>sumideros</i> (solo consume).
Conectores	Tuberías (pipes) : flujos de datos unidireccionales, que preservan el orden y no transforman nada. El SAIP los menciona explícitamente como ejemplo de conector: "pipes que representan flujos de datos asíncronos que preservan el orden".
Restricciones	Los filtros son independientes : no comparten estado ni conocen la identidad de sus vecinos; toda la comunicación pasa por las tuberías. La corrección del conjunto no debe depender del orden en que los filtros procesan (más allá del orden del flujo).
Cuándo usarlo	Cuando el problema se expresa naturalmente como una serie de transformaciones sucesivas sobre datos que fluyen, y no hace falta interacción con el usuario en el medio ni estado compartido.
Favorece	Modificabilidad y reutilización (los filtros se reordenan, se sustituyen y se recombinan sin tocarse entre sí); performance por paralelismo (los filtros corren concurrentemente sobre distintas porciones del flujo); testabilidad (cada filtro se prueba en aislamiento dándole un stream de entrada); comprensibilidad del conjunto como composición de funciones.
Sacrifica	Interactividad (no sirve para sistemas guiados por eventos del usuario); estado compartido (si los filtros necesitan un contexto común, el estilo se rompe); latencia por transformación (cada filtro suele serializar/parsear en su frontera, y ese costo se paga en cada etapa); manejo de errores pobre y difícil de propagar hacia atrás.
Ejemplos reales	Los shells de UNIX (<code>ps grep java wc -l</code>) son el ejemplo canónico; las fases de un compilador (léxico → sintáctico → semántico → generación); el procesamiento de imágenes y señales; y en el material del curso, el pipeline de despliegue del capítulo 18 (desarrollo → integración → staging → producción) es un batch secuencial con etapas y quality gates.
Parientes en el SAIP	Map-reduce (cap. 22) es un flujo de datos con dos etapas masivamente paralelas y un shuffle entre ellas; y las tácticas de performance manage sampling rate y bound queue sizes son las que se usan para controlar un pipeline saturado.

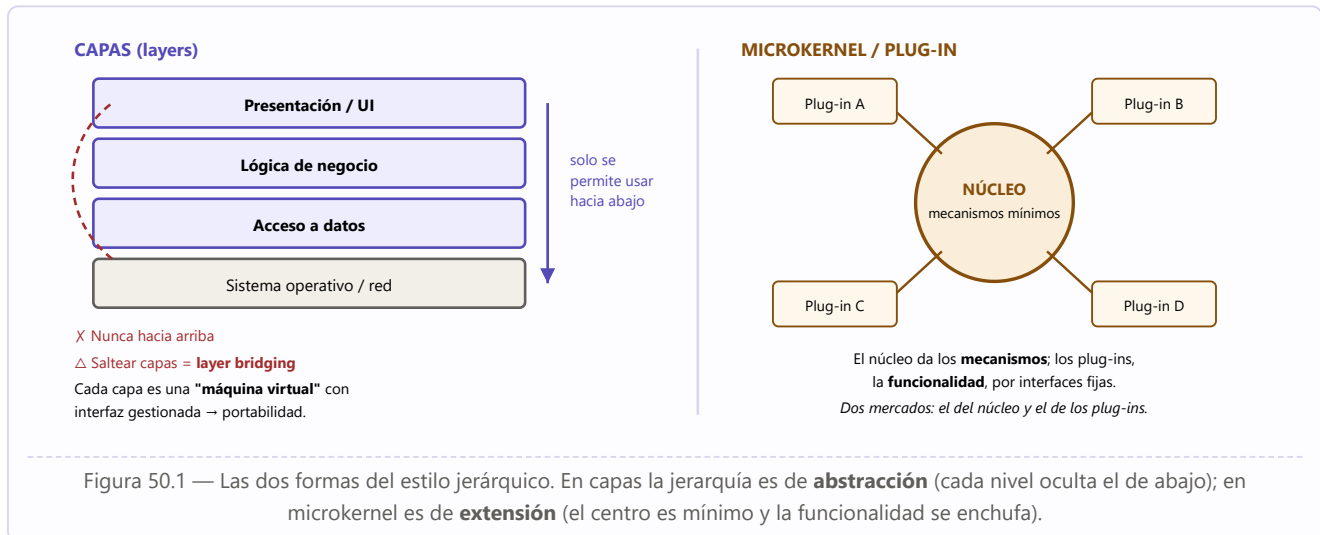
49. Centrados en datos: repositorio y pizarra



Aspecto	Centrados en datos [clásico]
Componentes	Un almacén central (base de datos, repositorio, pizarra) más un conjunto de accesores o agentes independientes que operan sobre él.
Conectores	Consultas y actualizaciones — típicamente transacciones . En la variante pizarra se suma la notificación del almacén hacia los agentes suscritos.
Restricciones	Los accesorios solo interactúan a través del almacén , nunca directamente entre sí. El esquema de datos es el contrato compartido por todos.
Cuándo usarlo	Cuando el problema gira alrededor de datos de larga vida que muchos componentes deben leer y modificar, con requisitos de integridad y consistencia; o cuando el orden de las contribuciones no se puede predeterminar (pizarra).
Favorece	Integrabilidad y modificabilidad : agregar un accesor nuevo no obliga a tocar los existentes (es el mecanismo de la táctica <i>usar un intermediario</i>); integridad centralizada (un solo lugar donde se validan y versionan los datos); escalabilidad de lectura con réplicas y caché.
Sacrifica	Performance y disponibilidad : el almacén es cuello de botella y punto único de falla (de ahí que el SAIP le dedique tácticas de replicación de datos y de spares); acoplamiento oculto : el esquema une a todos los accesorios — cambiarlo los impacta a todos, y ese acoplamiento no aparece en el código (es exactamente la distancia semántica de datos del capítulo 20); dificultad de distribución (coordinación y locks distribuidos, cap. 30).
Ejemplos reales	Los sistemas de información de gestión con base de datos central; los IDE con un modelo compartido del proyecto; el object store de Git ; y en IA clásica el sistema HEARSAY-II de reconocimiento del habla, el caso canónico de pizarra. En el propio SAIP, la figura del capítulo 13 muestra "un repositorio compartido al que acceden servidores y un componente administrativo".
Parientes en el SAIP	La táctica de modificabilidad repositorios compartidos (binding en runtime); el data model como estructura de módulos (cap. 13); y los patrones de coordinación de datos distribuida — Paxos, Zookeeper, etcd — cuando el repositorio deja de ser una sola máquina (cap. 30).

50. Jerárquicos: capas y microkernel

Es el estilo que Reynoso llama "jerárquico" y el único de los seis que el SAIP desarrolla **como patrón** con todo detalle. Su origen es directamente el **THE de Dijkstra (1968)**: capas que se comunican solo con las adyacentes y se superponen "como capas de cebolla".



Aspecto	Capas [SAIP cap. 21]	Microkernel / plug-in [SAIP cap. 21]
Componentes	Capas: agrupaciones cohesivas de módulos que ofrecen un conjunto de servicios por una interfaz pública.	Un núcleo con la funcionalidad central y plug-ins especializados que la extienden.
Conectores / relación	La relación " tiene permitido usar ", con una restricción clave: debe ser unidireccional y estrictamente ordenada.	Un conjunto fijo de interfaces ; el binding ocurre en el build o después (incluso en runtime).
Restricciones	Partición completa del software; nada de usos hacia arriba; normalmente solo la capa inmediata inferior. Usar una capa no adyacente es layer bridging .	Los plug-ins solo se comunican con el núcleo por sus interfaces; las dependencias entre plug-ins se evitan.
Favorece	Modificabilidad (cambiar una capa sin afectar las de arriba si la interfaz no cambia); portabilidad (la capa baja aísla el SO o la red); testabilidad (se testea de abajo hacia arriba con confianza); menos interfaces por equipo.	Extensibilidad controlada; evolución independiente del núcleo y de las extensiones; habilita dos mercados (quien hace el producto y quien hace los plug-ins).
Sacrifica	Performance (una llamada del tope puede atravesar muchas capas antes de ejecutarse); si la estratificación está mal diseñada estorba — no da las abstracciones que arriba necesitan; y mucho <i>bridging</i> destruye las metas de portabilidad que justificaban el estilo.	Seguridad y privacidad: como los plug-ins pueden venir de terceros, es más fácil introducir vulnerabilidades.
Ejemplos	El modelo OSI , UNIX System V (la figura del cap. 13), las arquitecturas de 3 capas, los sistemas con capa de portabilidad.	Los micro-kernels de SO (el núcleo da espacios de direcciones, hilos e IPC; los plug-ins, drivers y gestión de tareas), VS Code , los navegadores, R .

Idea clave — capa ≠ tier

Una **capa** es una partición del código: vive en la estructura de **módulos** (visión estática). Un **tier** es una partición del despliegue: vive en la estructura de **asignación**. Se pueden tener tres capas en un solo tier (un ejecutable monolítico bien estratificado) o una capa repartida en varios tiers. Confundirlos es el error clásico al leer un diagrama de "arquitectura en capas". Ver capítulo 13 y la pregunta de discusión del SAIP: "el patrón cliente-servidor es el microkernel con binding en runtime — discuta".

51. Invocación implícita: publish-subscribe y broker

Reynoso lo lista como uno de los seis estilos típicos; el SAIP lo desarrolla como el patrón **publish-subscribe**. Su rasgo definitorio: **el componente que emite no invoca a nadie directamente** — anuncia un evento y la infraestructura decide a quién le llega. De ahí el nombre: la invocación de los suscriptores es *implícita*.



Figura 51.1 — Publish-subscribe: tres tipos de elementos (publicador, suscriptor y bus de eventos) y desacoplamiento total entre los extremos.

Aspecto	Invocación implícita (SAIP cap. 21 + clásica)
Componentes	Publicadores (emiten mensajes), suscriptores (se registran y reciben) y el bus de eventos , que es parte de la infraestructura de runtime y gestiona suscripciones y despacho.
Conectores	Mensajes asíncronos sobre eventos o <i>topics</i> . El SAIP los nombra como conector típico: "multicast de eventos que soporta interacciones publish-subscribe", y aclara que los conectores no son binarios : uno de pub-sub puede tener un número arbitrario de publicadores y suscriptores.
Restricciones	El publicador no invoca directamente a ningún componente y no conoce la identidad de los suscriptores; estos solo conocen tipos de mensaje.
Variante: broker	Cuando además hay que localizar al destinatario y reenviarle la petición esperando respuesta, el intermediario se llama broker (POSA). El SAIP lo menciona al hablar de reducir indirección: "algunos brokers permiten comunicación directa entre cliente y servidor después de establecer la relación por el broker, eliminando la indirección en las peticiones siguientes".
Favorece	Modificabilidad e integrabilidad máximas: agregar o cambiar suscriptores no toca al publicador; se puede cambiar la conducta del sistema cambiando qué se publica (encender o apagar features suprimiendo mensajes); testabilidad por logging : los eventos se registran fácil y habilitan <i>record/playback</i> para reproducir errores difíciles.
Sacrifica	Performance (latencia del despacho, difícil razonar sobre ella); determinismo (el orden de invocación puede variar entre implementaciones); testabilidad estructural (un cambio mínimo en el bus tiene impacto amplio); seguridad : como publicador y suscriptor no se conocen, el cifrado de extremo a extremo es limitado — exigiría que todos compartan la misma clave; y no se sabe cuánto tarda un mensaje en llegar.
Ejemplos	Los sistemas de eventos de las GUI; las colas y <i>topics</i> de mensajería (Kafka, RabbitMQ); los sistemas reactivos; y en el material, la implementación distribuida de MVC (cap. 26), que usa pub-sub donde la versión en un solo proceso usa Observer.
Parientes de diseño	En el nivel de clases y objetos, este estilo es el patrón Observer del GoF (cap. 62) — misma idea, otra escala: allá son objetos en un proceso; acá, componentes distribuidos.

52. Cliente-servidor y peer-to-peer

Los dos estilos que definen **quién tiene la iniciativa** en un sistema distribuido: asimétrico (unos piden, otros responden) o simétrico (todos hacen ambas cosas). Reynoso lista *peer-to-peer* entre los seis estilos típicos; el SAIP desarrolla cliente-servidor como patrón, junto con sus descendientes modernos: microservicios y SOA.

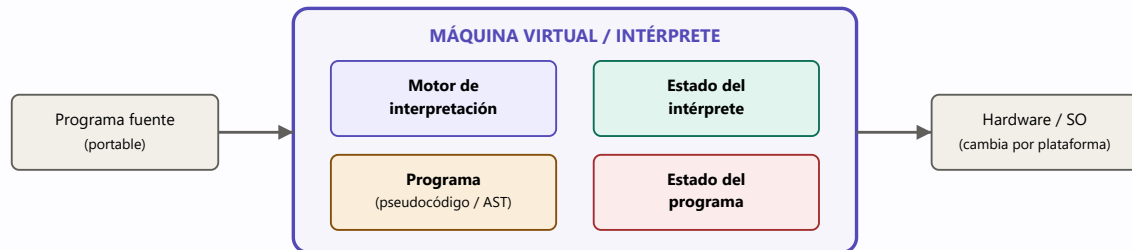
Aspecto	Cliente-servidor [SAIP cap. 21]	Peer-to-peer [clásico]
Componentes	Servidores que proveen servicios y clientes que los consumen. Puede haber múltiples instancias del servidor si los clientes exceden su capacidad.	Pares (peers) simétricos: cada uno actúa como cliente y como servidor a la vez.
Conectores	Petición-respuesta por protocolo acordado, iniciada siempre por el cliente , precedida por un descubrimiento para ubicar al servidor.	Protocolos de red simétricos, con descubrimiento de pares y a menudo enrutamiento entre ellos.
Restricciones	La comunicación la inicia el cliente ; los clientes no se comunican entre sí. Si el servidor guarda estado del cliente, cada petición debe identificarlo y hace falta un "fin de sesión" o un timeout que libere recursos.	No hay coordinador central; cualquier par puede entrar y salir en cualquier momento (<i>churn</i>).
Favorece	Bajo acoplamiento servidor-clientes (conexión dinámica, el servidor no conoce a sus clientes a priori) y nulo entre clientes; escalabilidad (limitada por la capacidad del servidor, que también puede escalar); evolución independiente ; servicios comunes compartidos; la interacción con el usuario queda aislada en el cliente — lo que permitió desarrollar lenguajes y herramientas especializadas de UI.	Escalabilidad sin servidor central (cada par aporta recursos); resiliencia (no hay punto único de falla); aprovecha capacidad ociosa distribuida.
Sacrifica	Performance impredecible (la comunicación va por red, con congestión y la cola larga de latencia del cap. 30); el servidor es cuello de botella y foco de disponibilidad; seguridad : en redes compartidas hay que proveer confidencialidad e integridad explícitamente.	Complejidad : descubrimiento, consistencia y seguridad son mucho más difíciles; latencia variable; difícil de administrar y auditar.
Ejemplos	La web (navegador-servidor); las bases de datos; y sus descendientes: n-tiers , SOA (proveedores y consumidores de distintas organizaciones, con WSDL/SOAP y SLA) y microservicios (un sistema, una organización, comunicación solo por mensajes).	BitTorrent, las redes blockchain, Gnutella; y los algoritmos de consenso distribuido (Paxos) del cap. 30, que son coordinación entre pares.

Cuidado — microservicios NO es sinónimo de "cliente-servidor moderno"

El SAIP los distingue con precisión: **SOA** supone servicios **heterogéneos y de organizaciones distintas**, reutilizables y con SLA; **microservicios** componen **un solo sistema gestionado por una sola organización**, con equipos pequeños ("regla de las dos pizzas"), dependencias acíclicas y la prohibición estricta de cualquier comunicación que no sea por mensajes: sin enlace directo, sin leer el almacén de datos de otro equipo, sin memoria compartida, **sin puertas traseras**.

53. Intérprete y máquina virtual

El sexto estilo de la lista de Reynoso. Su idea: en vez de ejecutar directamente sobre el hardware, se construye **una máquina abstracta en software** que ejecuta un programa expresado en un lenguaje propio. Es el estilo que hace posible que "escribí una vez, corré en cualquier parte".



El programa **no sabe** sobre qué máquina corre: la VM absorbe la diferencia. Se reimplementa la VM, no los programas.

Figura 53.1 — Los cuatro elementos del estilo intérprete: el motor, la representación del programa, el estado del programa y el estado del propio intérprete (el "program counter" de la máquina abstracta).

Aspecto	Intérprete / máquina virtual <i>(clásico)</i>
Componentes	Motor de interpretación ; la representación del programa a ejecutar (pseudocódigo, bytecode o árbol sintáctico); el estado del programa en ejecución; y el estado del propio intérprete (dónde va, qué sigue).
Conectores	Llamadas al motor y acceso directo a las estructuras de estado compartidas.
Restricciones	El programa interpretado no accede al hardware : todo pasa por la máquina abstracta, que define el conjunto de operaciones legales.
Cuándo usarlo	Cuando hace falta portabilidad sobre plataformas heterogéneas; cuando la lógica debe poder cambiar sin recompilar (reglas de negocio, scripts de usuario); cuando se simula una máquina que no existe o no está disponible; o cuando conviene ofrecer un lenguaje de dominio a los usuarios.
Favorece	Portabilidad (se reimplementa la VM, no los programas); modificabilidad en runtime — es la táctica de <i>diferir la ligadura</i> llevada al extremo: interpretar parámetros ; testabilidad (el estado es explícito e inspeccionable, y se puede sandboxear: la propia noción de <i>sandbox</i> del cap. 25 es una máquina abstracta con recursos virtualizados, como el reloj virtual); seguridad por contención (el programa no puede hacer lo que la VM no le permite).
Sacrifica	Performance : se paga una capa de indirección en cada operación (el SAIP mide el mismo fenómeno en virtualización: overheads de ~10% con hipervisores, y aclara que un emulador —que simula la ejecución de otro juego de instrucciones— es más caro que un hipervisor, que exige la misma ISA); depuración más difícil (dos niveles de estado: el del programa y el de la VM).
Ejemplos	La JVM y el CLR; Python; los motores de reglas ; SQL sobre un planificador de consultas; los emuladores como QEMU (cap. 29); las macros y los lenguajes embebidos.
Parientes de diseño	En el nivel de clases, el patrón Interpreter del GoF (cap. 63): representa la gramática como jerarquía de clases y la interpreta recorriendo el árbol — el mismo estilo, a escala de diseño y "cuando la gramática es simple".

54. Comparación, composición y el caso KWIC

54.1 Qué atributo de calidad favorece cada estilo

Estilo	Favorece	Sacrifica	Táctica dominante que encarna
Tubería y filtro	Modificabilidad · reutilización · paralelismo	Interactividad · latencia por etapa · estado compartido	Introducir concurrencia; abstraer servicios comunes
Batch secuencial	Simplicidad · repetibilidad	Latencia total · uso de recursos	Bound execution times
Repositorio	Integrabilidad · integridad de datos	Performance (cuello de botella) · acoplamiento por el esquema	Usar un intermediario; mantener múltiples copias de datos
Pizarra	Extensibilidad con orden de contribución impredecible	Determinismo · trazabilidad del control	Repositorios compartidos + invocación implícita
Capas	Modificabilidad · portabilidad · testabilidad	Performance (atravesar niveles)	Restringir dependencias
Microkernel	Extensibilidad · evolución independiente	Seguridad (código de terceros)	Diferir la ligadura (binding)
Publish-subscribe	Modificabilidad máxima · integrabilidad	Latencia · determinismo · cifrado E2E	Usar un intermediario
Cliente-servidor	Escalabilidad · servicios compartidos · evolución independiente	Performance por red · disponibilidad del servidor	Mantener múltiples copias de cómputos
Peer-to-peer	Escalabilidad sin centro · resiliencia	Consistencia · seguridad · administrabilidad	Discover (descubrimiento)
Intérprete / VM	Portabilidad · modificabilidad en runtime · contención	Performance · depurabilidad	Interpretar parámetros; sandbox

54.2 Los estilos se componen

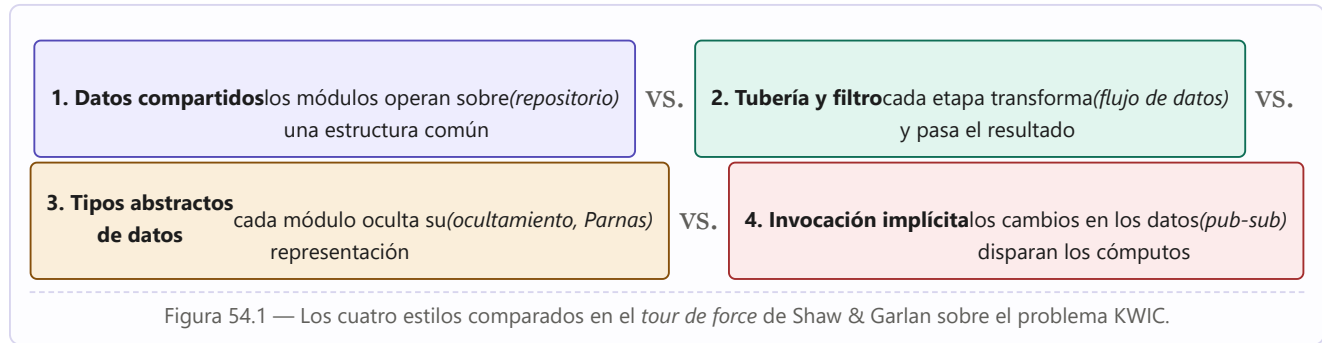
Idea clave — el rasgo más importante del catálogo

Reynoso lo dice dos veces: "el conjunto de los estilos cataloga las formas básicas posibles de estructuras de software, mientras que **las formas complejas se articulan mediante composición de los estilos fundamentales**", y "las arquitecturas complejas o compuestas resultan del agregado o la composición de estilos más básicos". Ningún sistema real es "de un estilo": **los sistemas reales son mezclas**. Ejemplo del propio material: un sistema de microservicios (cliente-servidor) que se comunica por un bus de eventos (invocación implícita), donde cada servicio está internamente en capas, corre sobre una máquina virtual o contenedor (intérprete/VM), guarda su estado en un repositorio y se despliega por un pipeline (batch secuencial). **Cinco estilos en un solo sistema** — y eso es lo normal, no la excepción.

54.3 El caso KWIC: el mismo problema en cuatro estilos

Es la demostración que Reynoso elige como la más elocuente de la disciplina — "si hubiera que singularizar un trabajo simple y elegante, representativo del estado del arte, sin duda escogeríamos esta demostración". Shaw y Garlan resuelven **un mismo problema** — la indexación de palabras clave en contexto (KWIC) — en **cuatro**

estilos distintos, y comparan los resultados **antes de escribir una sola línea de código**:



El estudio "articula un modelo que permite estimar relaciones de **dependencia, modularidad, refinamiento, reutilización, ventajas y desventajas** de cada arquitectura antes de escribir una sola línea de código; demuestra también de qué manera los diferentes estilos definen **tácticas específicas de descomposición funcional** y establecen la pauta que habrá de seguirse en el desarrollo", y cierra con tablas de comparación de atributos. Es, a la vez, la mejor prueba de las tres tesis centrales de toda esta guía: que **la funcionalidad no determina la arquitectura** (cuatro arquitecturas, la misma función), que **los atributos de calidad sí la determinan**, y que **una arquitectura se puede evaluar antes de construirla**.

PARTE X

Las 23 fichas del catálogo GoF

*Una ficha por patrón: **problema** que resuelve, **solución** y participantes, **cuándo aplicarlo** (aplicabilidad textual del libro), **consecuencias** a favor y en contra, y **patrones relacionados**. El material sale del catálogo del GoF (págs. 79–350, secciones Applicability y Consequences de cada patrón) más el fraseo problema→solución de refactoring.guru.*

-
- 55 Cómo leer una ficha
 - 56 Los 5 patrones creacionales
 - 57 Los 7 patrones estructurales
 - 58 Los 11 patrones de comportamiento

55. Cómo leer una ficha

Cada ficha tiene seis campos, y conviene saber para qué sirve cada uno al estudiar:

Campo	Para qué lo usás
Problema	Reconocer la situación en tu código. Es el disparador: si no reconocés el problema, no busques el patrón.
Solución	Entender el mecanismo y los participantes (las clases y objetos que el patrón introduce, con su rol).
Cuándo aplicarlo	La sección <i>Applicability</i> del GoF: la lista de condiciones que deben darse. Es lo que decide si el patrón corresponde o no.
A favor / En contra	La sección <i>Consequences</i> : los tradeoffs. El GoF insiste en que es "la más útil para evaluar los beneficios y pasivos de un patrón".
Qué te deja variar	El eje de la Tabla 1.2 del GoF: qué aspecto del sistema podés cambiar sin rediseñar . Es la forma más rápida de elegir patrón.
Relacionados	Con qué patrones se combina, con cuáles compite y con cuáles se confunde.

Antes de aplicar cualquiera de los 23

Recordá las dos advertencias que atraviesan todo el catálogo: **(1)** "los patrones no deben aplicarse indiscriminadamente: suelen lograr flexibilidad introduciendo **niveles adicionales de indirección**, y eso complica el diseño y cuesta performance — un patrón sólo debe aplicarse **cuando la flexibilidad que otorga es realmente necesaria**"; **(2)** "si todo lo que tenés es un martillo, todo parece un clavo". Y al implementarlo, **nombrá a los participantes con el vocabulario de tu dominio**, no con el del libro: `TeXLayoutStrategy`, no `ConcreteStrategy1`.

56. Los 5 patrones creacionales

Abstraen la instanciación: encapsulan qué clases concretas usa el sistema y ocultan cómo se crean y se combinan sus instancias. Ganan importancia a medida que el sistema depende más de la composición de objetos que de la herencia de clases.

56.1 Abstract Factory — objeto · creacional

Intención	Proveer una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.
Problema	Tu sistema debe funcionar con varias familias de productos que deben usarse juntas y no mezclarse (todos los widgets Motif, o todos los de Presentation Manager — nunca la mitad y la mitad), y no querés que el código del cliente nombre ni una sola clase concreta.
Solución	Una AbstractFactory declara una operación de creación por cada tipo de producto; cada ConcreteFactory implementa la familia completa. Participantes: AbstractFactory · ConcreteFactory · AbstractProduct · ConcreteProduct · Client. El cliente solo conoce las interfaces abstractas.
Cuándo	Cuando (a) el sistema debe ser independiente de cómo se crean, componen y representan sus productos; (b) debe configurarse con una de varias familias ; (c) una familia está diseñada para usarse junta y hay que hacer cumplir esa restricción ; (d) querés ofrecer una biblioteca de productos revelando solo sus interfaces , no sus implementaciones.
A favor	1. Aísla las clases concretas : los nombres de las clases producto quedan encerrados en la fábrica concreta y no aparecen en el código cliente. 2. Facilita intercambiar familias completas : la clase de la fábrica concreta aparece una sola vez en la aplicación (donde se instancia), así que cambiar toda la familia es cambiar un objeto — "pasamos de widgets Motif a Presentation Manager simplemente cambiando la fábrica y recreando la interfaz". 3. Promueve consistencia entre los productos de una familia.
En contra	Soportar tipos nuevos de producto es difícil : la interfaz de AbstractFactory fija el conjunto de productos creables, así que agregar uno obliga a cambiar la clase abstracta y todas sus subclases .
Varía	Las familias de objetos producto.
Relacionados	Suele implementarse con Factory Method (o con Prototype, para no crear una jerarquía paralela de fábricas). Prototype es su alternativa frecuente. La fábrica concreta suele ser un Singleton . Compíte con Prototype y Builder: los tres usan un "objeto fábrica"; la diferencia es que en Prototype la fábrica es el prototipo que se clona. Resuelve las causas de rediseño 1, 3, 4 y 6.

56.2 Builder — objeto · creacional

Intención	Separar la construcción de un objeto complejo de su representación , de modo que el mismo proceso de construcción pueda crear representaciones distintas.
Problema	Un objeto complejo se arma en muchos pasos y querés obtener varias representaciones del resultado sin duplicar el algoritmo de armado (leer un RTF una vez y poder emitir texto plano, TeX o un widget de texto).
Solución	Un Director conduce el proceso llamando paso a paso a un Builder abstracto; cada ConcreteBuilder construye y ensambla su propio Product y es quien sabe recuperarlo. Participantes: Builder · ConcreteBuilder · Director · Product.
Cuándo	Cuando (a) el algoritmo para crear un objeto complejo debe ser independiente de las partes que lo componen y de cómo se ensamblan; (b) el proceso de construcción debe permitir distintas representaciones del objeto construido.

A favor	1. Permite variar la representación interna del producto: el Builder oculta la estructura interna y cómo se ensambla; para cambiarla, se define un builder nuevo. 2. Aísla el código de construcción del de representación: mejora la modularidad; el cliente no sabe nada de las clases internas del producto, y cada ConcreteBuilder concentra todo el código de armado, que se escribe una vez y distintos directores reutilizan. 3. Da control más fino del proceso: a diferencia de los creacionales que fabrican de un tiro, construye paso a paso bajo control del director, que recupera el producto solo cuando está terminado.
En contra	Requiere una clase builder por representación y un protocolo de construcción más elaborado: es el creacional con más andamiaje , injustificado si el objeto es simple.
Varía	Cómo se crea un objeto compuesto.
Relacionados	Suele construir un Composite . Puede usar Abstract Factory o Prototype internamente para decidir qué componentes fabrica. Es el patrón asociado a la causa de rediseño 5 (dependencias algorítmicas).

56.3 Factory Method — *clase · creacional*

Intención	Definir una interfaz para crear un objeto, pero dejar que las subclasses decidan qué clase instanciar . Difiere la instanciación a las subclasses.
Problema	Una clase (un framework, típicamente) tiene que crear objetos cuyo tipo concreto solo conoce quien la extiende : el framework sabe <i>cuándo</i> crear un documento, pero no <i>de qué clase</i> .
Solución	El Creator declara un método de fábrica (a veces con implementación por defecto) y lo llama donde necesita el producto; cada ConcreteCreator lo sobrescribe. Participantes: Product · ConcreteProduct · Creator · ConcreteCreator.
Cuándo	Cuando (a) una clase no puede anticipar la clase de los objetos que debe crear; (b) una clase quiere que sus subclasses especifiquen los objetos que crea; (c) una clase delega responsabilidad en una de varias subclasses ayudantes y querés localizar el conocimiento de cuál es la delegada.
A favor	Elimina la necesidad de atar clases específicas de la aplicación al código del framework: este solo trata con la interfaz Product. 1. Provee ganchos (hooks) para subclasses : crear con un método de fábrica siempre es más flexible que crear directamente. 2. Conecta jerarquías paralelas de clases, cuando una clase delega parte de sus responsabilidades en otra jerarquía.
En contra	Los clientes pueden verse forzados a subclasificar el Creator solo para crear un producto concreto — está bien si igual iban a subclasificarlo, pero si no, se agrega "otro punto de evolución" a mantener. Y su uso masivo prolifera subclasses que hacen muy poco .
Varía	La subclase del objeto que se instancia.
Relacionados	Es el creacional más simple para empezar : "los diseños suelen arrancar con Factory Method y evolucionar hacia Abstract Factory, Prototype o Builder cuando se descubre dónde hace falta más flexibilidad". Template Method suele llamar factory methods. En MVC define el controlador por defecto de una vista.

56.4 Prototype — *objeto · creacional*

Intención	Especificar los tipos de objetos a crear usando una instancia prototípica , y crear los nuevos copiándola (clonándola).
Problema	Las clases a instanciar se deciden en tiempo de ejecución (carga dinámica, paleta de herramientas configurable), o construir una jerarquía de fábricas paralela a la de productos sería un desperdicio.
Solución	Un Prototype declara <code>Clone()</code> ; cada ConcretePrototype se copia a sí mismo; el cliente pide clones a un prototipo que le fue configurado. Participantes: Prototype · ConcretePrototype · Client (+ a menudo un gestor de prototipos).

Cuándo	Cuando el sistema debe ser independiente de cómo se crean sus productos y además: (a) las clases a instanciar se especifican en runtime , por ejemplo por carga dinámica; (b) querés evitar una jerarquía de fábricas paralela a la de productos; (c) las instancias de una clase tienen pocas combinaciones de estado distintas — es más cómodo instalar otros tantos prototipos y clonarlos que instanciar la clase a mano cada vez con el estado adecuado.
A favor	Comparte con Abstract Factory y Builder que oculta las clases concretas al cliente. Además: 1. Agregar y quitar productos en runtime (registrar una instancia prototípica y listo — más flexible que los otros creacionales). 2. Especificar objetos nuevos variando valores : se definen "clases" nuevas sin programar, clonando y configurando; reduce drásticamente el número de clases ("una sola clase GraphicTool puede crear una variedad ilimitada de objetos musicales"). 3. Especificar objetos nuevos variando la estructura : si el compuesto implementa Clone como copia profunda, un subcircuito completo puede agregarse como prototipo a la paleta.
En contra	Cada subclase debe implementar Clone , que puede ser difícil: los objetos con referencias circulares o que no soportan copia lo complican, y hay que decidir en cada caso entre copia superficial y profunda .
Varía	La clase del objeto que se instancia.
Relacionados	Alternativa a Abstract Factory . En el ejemplo del editor de dibujo del GoF resulta la mejor de las tres opciones (solo pide implementar Clone, y Clone sirve además para un "Duplicar" del menú). Suele usar Singleton en su implementación y trabaja con Composite y Decorator .

56.5 Singleton — *objeto · creacional*

Intención	Asegurar que una clase tenga una sola instancia y proveer un punto de acceso global a ella.
Problema	Algo debe existir una única vez en el sistema (un spooler de impresión, un registro de configuración, el gestor de ventanas) y todos deben llegar a esa misma instancia — pero una variable global no controla ni cuándo se crea ni que no se duplique.
Solución	La clase se encarga de sí misma : constructor no público y una operación de clase (<code>Instance()</code>) que crea la instancia la primera vez y la devuelve siempre. Participante único: Singleton.
Cuándo	Cuando (a) debe haber exactamente una instancia de una clase, accesible a los clientes desde un punto conocido; (b) esa única instancia debe poder extenderse por subclasificación y los clientes deben poder usar la instancia extendida sin modificar su código .
A favor	1. Acceso controlado a la instancia única (la clase encapsula cómo y cuándo se accede). 2. Reduce el espacio de nombres : es una mejora sobre las variables globales, que contaminan el namespace. 3. Permite refinar operaciones y representación (se puede subclasificar y configurar la app con la subclase en runtime). 4. Permite un número variable de instancias : si mañana hacen falta tres, solo cambia la operación de acceso. 5. Más flexible que las operaciones de clase : los métodos estáticos de C++ no son virtuales, así que las subclases no pueden redefinirlos polimórficamente.
En contra	Es el patrón más criticado del catálogo: introduce estado global y una dependencia oculta que daña la testabilidad (no se puede sustituir por un doble de prueba fácilmente — de ahí que el SAIP recomiende dependency injection justamente para esto) y complica la concurrency (la creación debe protegerse).
Varía	La instancia única de una clase.
Relacionados	Muchos patrones se implementan con Singleton: Abstract Factory, Builder, Prototype (sus fábricas y gestores). En el material del SAIP, la lógica de votación de la táctica <i>voting</i> se implementa "como un singleton simple, rigurosamente revisado y probado".

57. Los 7 patrones estructurales

Se ocupan de cómo componer clases y objetos en estructuras más grandes. Los de clase usan herencia para componer interfaces o implementaciones; los de objeto componen objetos, con la ventaja de poder cambiar la composición en runtime.

57.1 Adapter — clase y objeto · estructural

Intención	Convertir la interfaz de una clase en otra interfaz que los clientes esperan . Permite que colaboren clases que no podrían por interfaces incompatibles.
Problema	Tenés una clase que hace lo que necesitás pero su interfaz no coincide con la que tu sistema espera, y no podés (o no querés) modificarla.
Solución	Un Adapter implementa la interfaz Target que el cliente usa y traduce las llamadas al Adaptee . Dos variantes: de clase (hereda de ambos) y de objeto (contiene una referencia al adaptee — la preferida).
Cuándo	Cuando (a) querés usar una clase existente y su interfaz no es la que necesitás ; (b) querés crear una clase reutilizable que cooperen con clases imprevistas , sin interfaces necesariamente compatibles; (c) (<i>solo el de objeto</i>) necesitás usar varias subclases existentes y sería impracticable adaptar la interfaz subclasificando cada una.
A favor	Adapter de clase : permite sobrescribir parte de la conducta del adaptee (es su subclase) e introduce un solo objeto, sin indirección extra. Adapter de objeto : un mismo adapter sirve para el adaptee y todas sus subclases , y puede agregarles funcionalidad a todas de una vez.
En contra	El de clase no funciona si querés adaptar una clase y todas sus subclases (queda atado a una concreta). El de objeto hace más difícil sobrescribir la conducta del adaptee. Y en ambos hay que decidir cuánto adapta : hay un espectro que va del simple cambio de nombres de operaciones a soportar un conjunto de operaciones enteramente distinto. Variante avanzada: los pluggable adapters , que llevan la adaptación de interfaz dentro de la clase para minimizar los supuestos que otras clases deben hacer.
Varía	La interfaz a un objeto.
Relacionados	Bridge tiene estructura parecida pero intención distinta: Adapter resuelve incompatibilidades entre interfaces ya existentes (hace que las cosas funcionen después de diseñadas); Bridge separa una abstracción de sus implementaciones previsto de antemano . Facade define una interfaz nueva ; Adapter reutiliza una vieja . Decorator no cambia la interfaz, la mantiene. En el SAIP este patrón aparece como el wrapper de las tácticas de integrabilidad. Resuelve la causa de rediseño 8 .

57.2 Bridge — objeto · estructural

Intención	Desacoplar una abstracción de su implementación para que ambas puedan variar independientemente.
Problema	Una abstracción tiene varias implementaciones posibles (una ventana en X11, en Windows, en macOS) y resolverlo por herencia produce una explosión combinatoria de subclases — las "generalizaciones anidadas" de Rumbaugh — y liga la implementación en tiempo de compilación.
Solución	Dos jerarquías separadas: Abstraction (con su RefinedAbstraction) mantiene una referencia a un Implementor (con sus ConcretImplementor) y le delega el trabajo. Las dos evolucionan por separado.
Cuándo	Cuando (a) querés evitar un binding permanente entre abstracción e implementación — por ejemplo porque la implementación se elige o se cambia en runtime ; (b) tanto abstracciones como implementaciones deben ser extensibles por subclasificación independientemente; (c) los cambios en la implementación no deben impactar a los clientes (ni obligar a recompilar); (d) (C++) querés ocultar completamente la implementación al cliente; (e) hay una proliferación de clases que indica que el objeto debería partirse en dos; (f) querés compartir una implementación entre varios objetos (con conteo de referencias) sin que el cliente lo note.

A favor	1. Desacopla interfaz e implementación: la implementación se configura — incluso se cambia — en runtime; se eliminan las dependencias de compilación, lo que es esencial para mantener compatibilidad binaria entre versiones de una biblioteca; y fomenta una estratificación mejor (la parte alta solo conoce Abstraction e Implementor). 2. Mejor extensibilidad: las dos jerarquías crecen por separado. 3. Oculta detalles al cliente (como el conteo de referencias del implementor compartido).
En contra	Agrega una indirección y duplica la jerarquía: injustificado si hay una sola implementación o si nunca va a haber más.
Varía	La implementación de un objeto.
Relacionados	Abstract Factory suele crear y configurar el bridge. Se distingue de Adapter por el momento y la intención (ver arriba). Si la abstracción y una implementación calzan bien, la abstracción "simplemente delega" — la delegación es su mecanismo. Resuelve las causas de rediseño 3, 4, 6 y 7. En Lexi resuelve el soporte de múltiples sistemas de ventanas .

57.3 Composite — objeto · estructural

Intención	Componer objetos en estructuras de árbol para representar jerarquías parte-todo, permitiendo que los clientes traten uniformemente a los objetos individuales y a las composiciones.
Problema	El cliente tiene que distinguir constantemente entre "un elemento" y "un grupo de elementos", lo que llena el código de condicionales y verificaciones de tipo.
Solución	Una interfaz común Component ; las Leaf son los objetos primitivos y los Composite contienen hijos (leaves u otros composites) y delegan en ellos las operaciones. Recursión hasta cualquier profundidad.
Cuándo	Cuando (a) querés representar jerarquías parte-todo de objetos; (b) querés que los clientes puedan ignorar la diferencia entre composiciones y objetos individuales, tratando a todos de forma uniforme.
A favor	Simplifica el cliente: no sabe (ni debe importarle) si trata con una hoja o con un compuesto, lo que evita las funciones "con etiqueta y case" sobre las clases de la composición. Facilita agregar componentes nuevos: las subclases nuevas de Leaf o Composite funcionan automáticamente con las estructuras y el código cliente existentes.
En contra	Puede volver el diseño demasiado general: la contracara de que sea fácil agregar componentes es que se vuelve difícil restringir los componentes de un compuesto. Si querés que un compuesto acepte solo cierto tipo de hijos, "no podés apoyarte en el sistema de tipos: hay que usar chequeos en runtime".
Varía	La estructura y composición de un objeto.
Relacionados	Se usa muchísimo con otros : Decorator (comparten la composición recursiva y suelen usarse juntos: desde Decorator un composite es un ConcreteComponent, y desde Composite un decorator es una Leaf); Iterator y Visitor para recorrerlo y operar sobre él; Chain of Responsibility para que los hijos accedan a propiedades globales vía el padre; Flyweight para compartir hojas (con la consecuencia de que la hoja no puede guardar puntero al padre: se pasa como estado extrínseco); se crea con Builder . En Lexi resuelve la estructura del documento .

57.4 Decorator — objeto · estructural

Intención	Adjuntar responsabilidades adicionales a un objeto dinámicamente . Alternativa flexible a la subclasificación para extender funcionalidad.
Problema	Querés agregar capacidades a objetos individuales (este campo de texto con borde y scroll; ese otro sin nada) y cubrir todas las combinaciones por herencia produciría una explosión de subclases.

Solución	El Decorator implementa la misma interfaz que el componente que envuelve, le delega y agrega su comportamiento antes o después. Al implementar la misma interfaz, los decoradores se anidan recursivamente.
Cuándo	Para (a) agregar responsabilidades a objetos individuales dinámica y transparentemente , sin afectar a otros objetos; (b) responsabilidades que se pueden retirar ; (c) cuando extender por subclasificación es impracticable — hay muchas extensiones independientes y cubrir cada combinación explotaría en subclases, o la definición de la clase está oculta y no se puede subclasificar.
A favor	1. Más flexible que la herencia estática: las responsabilidades se agregan y quitan en runtime enganchando y desenganchando decoradores; se pueden mezclar y combinar, e incluso aplicar dos veces la misma (dos BorderDecorators = borde doble; heredar dos veces de Border es propenso a error). 2. Evita clases sobrecargadas de features en lo alto de la jerarquía: propone un enfoque "pagá lo que usás" — una clase simple a la que se le suma funcionalidad incremental, así "la aplicación no paga por features que no usa".
En contra	3. El decorador y su componente no son idénticos: actúa como envoltorio transparente, pero desde el punto de vista de la identidad de objeto un componente decorado no es el componente — no hay que apoyarse en la identidad cuando se usan decoradores. 4. Muchos objetitos: los diseños con Decorator terminan con montones de objetos pequeños que se parecen entre sí y difieren solo en cómo están interconectados, lo que dificulta aprender y depurar el sistema.
Varía	Las responsabilidades de un objeto, sin subclasificar .
Relacionados	Se distingue de Composite (embellecer vs. representar) y de Proxy (agregar vs. controlar el acceso). Strategy es la alternativa cuando conviene cambiar las "entrañas" en vez de la "piel". Resuelve las causas de rediseño 7 y 8. En Lexi embellece la UI (bordes y scrollbars), y en MVC agrega el scrolling a una vista.

57.5 Facade — *objeto · estructural*

Intención	Proveer una interfaz unificada a un conjunto de interfaces de un subsistema: una interfaz de más alto nivel que lo hace más fácil de usar.
Problema	Un subsistema creció y se volvió complejo (justamente porque aplicar patrones produce más clases y más chicas); los clientes que solo quieren el caso común tienen que conocer demasiado.
Solución	Una clase Facade que conoce a qué clases del subsistema delegar cada pedido y ofrece las operaciones de alto nivel. Los clientes hablan con la fachada.
Cuándo	Cuando (a) querés una interfaz simple a un subsistema complejo: "una fachada puede dar una vista por defecto simple que alcanza para la mayoría de los clientes; solo los que necesiten personalizar mirarán más allá"; (b) hay muchas dependencias entre los clientes y las clases de implementación de una abstracción — la fachada las desacopla y promueve independencia y portabilidad; (c) querés estratificar los subsistemas: la fachada define el punto de entrada de cada nivel y, si los subsistemas dependen entre sí, se simplifican las dependencias haciendo que se comuniquen solo por sus fachadas .
A favor	1. Escuda a los clientes de los componentes del subsistema: menos objetos con los que lidiar. 2. Promueve acoplamiento débil entre el subsistema y sus clientes: se pueden variar los componentes sin afectarlos; ayuda a estratificar y a eliminar dependencias complejas o circulares ; y reduce las dependencias de compilación , algo vital en sistemas grandes (limita la recompilación por un cambio chico y facilita portar, porque construir un subsistema ya no obliga a construir todos). 3. No impide que las aplicaciones usen las clases del subsistema si lo necesitan: se elige entre facilidad de uso y generalidad .
En contra	Puede convertirse en un objeto dios acoplado a todo el subsistema si se le agregan responsabilidades en vez de solo delegación.
Varía	La interfaz a un subsistema .

Relacionados	Abstract Factory puede ser la fachada de creación del subsistema. Mediator se parece pero su propósito es abstraer la comunicación entre colegas, no simplificar el acceso desde afuera . Suele ser Singleton . Es el patrón que da la interfaz gestionada de cada capa en el estilo de capas (cap. 50), y resuelve la causa de rediseño 6.
---------------------	---

57.6 Flyweight — objeto · estructural

Intención	Usar compartición para soportar eficientemente grandes cantidades de objetos de grano fino .
Problema	El diseño "correcto" pide un objeto por cada elemento (un objeto por carácter del documento) y eso no cabe en memoria por la pura cantidad.
Solución	Separar el estado intrínseco (compartible, independiente del contexto: la letra "a") del extrínseco (dependiente del contexto: su posición y su fuente, que pasa el cliente en cada operación). Una FlyweightFactory administra el pool de instancias compartidas.
Cuándo	Su efectividad "depende mucho de cómo y dónde se use". Aplícalo cuando se cumple todo esto: la aplicación usa gran cantidad de objetos; el costo de almacenamiento es alto por la pura cantidad; la mayor parte del estado puede volverse extrínseco ; muchos grupos de objetos pueden reemplazarse por pocos compartidos al quitarles el estado extrínseco; y la aplicación no depende de la identidad de los objetos (los tests de identidad darán verdadero para objetos conceptualmente distintos).
A favor	Ahorro de memoria , que crece con la cantidad de flyweights compartidos y con la cantidad de estado compartido. El máximo ahorro se da cuando los objetos usan mucho estado de los dos tipos y el extrínseco se puede calcular en vez de almacenar : ahí se gana dos veces — se comparte el intrínseco y se cambia memoria por tiempo de cómputo.
En contra	Costos de runtime por transferir, buscar o calcular el estado extrínseco (sobre todo si antes era intrínseco). Y al combinarse con Composite , las hojas compartidas no pueden guardar un puntero a su padre : el padre viaja como estado extrínseco, lo que "impacta fuertemente en cómo se comunican los objetos de la jerarquía".
Varía	Los costos de almacenamiento de los objetos.
Relacionados	Se combina con Composite (grafo con hojas compartidas), y suele aplicarse a los State y Strategy para compartirlos. Es el patrón que resuelve la granularidad en el extremo fino, mientras Facade lo hace en el extremo grueso.

57.7 Proxy — objeto · estructural

Intención	Proveer un sustituto o representante de otro objeto para controlar el acceso a él.
Problema	Acceder al objeto real es caro, remoto o debe restringirse, pero el cliente no debería tener que saberlo ni cambiar su código.
Solución	Un Proxy con la misma interfaz que el RealSubject , que mantiene una referencia a él y controla su acceso, creándolo o contactándolo cuando corresponde.
Cuándo	"Siempre que se necesite una referencia a un objeto más versátil o sofisticada que un simple puntero". Los cuatro tipos canónicos: 1. Proxy remoto — representante local de un objeto en otro espacio de direcciones (Coplien lo llama "embajador"). 2. Proxy virtual — crea objetos caros bajo demanda (la imagen que no se carga hasta que hay que dibujarla). 3. Proxy de protección — controla el acceso cuando distintos clientes tienen distintos derechos . 4. Referencia inteligente — reemplaza un puntero crudo agregando acciones: contar referencias para liberar automáticamente (<i>smart pointers</i>), cargar un objeto persistente en memoria al primer acceso, o verificar que el objeto esté bloqueado antes de accederlo.

A favor	Introduce un nivel de indirección con muchos usos: el remoto oculta que el objeto está en otro espacio de direcciones; el virtual permite optimizaciones como la creación bajo demanda; los de protección y las referencias inteligentes agregan tareas de mantenimiento en cada acceso. Y habilita el copy-on-write : copiar un objeto grande se posterga hasta que alguien lo modifique — "aseguramos pagar el precio de copiar solo si se modifica", lo que reduce significativamente el costo de copiar objetos pesados.
En contra	La indirección tiene costo, y el copy-on-write exige que el sujeto lleve conteo de referencias con la disciplina que eso implica.
Varía	Cómo se accede a un objeto y dónde está ubicado.
Relacionados	Se distingue de Decorator (controlar acceso vs. agregar responsabilidades; relación estática vs. recursiva) y de Adapter (misma interfaz vs. interfaz distinta). Los smart pointers son la táctica de <i>exception prevention</i> del SAIP (cap. 17). Resuelve la causa de rediseño 4. En el SAIP, el sidecar del service mesh es "una clase de proxy que acompaña a cada microservicio".

58. Los 11 patrones de comportamiento

Se ocupan de los algoritmos y de la asignación de responsabilidades entre objetos. No describen solo patrones de objetos o clases, sino los **patrones de comunicación** entre ellos: quitan el foco del flujo de control para ponerlo en cómo se interconectan los objetos. Los dos de clase (*Interpreter* y *Template Method*) usan herencia; los nueve de objeto, composición y delegación.

58.1 Chain of Responsibility — objeto · comportamiento

Intención	Evitar acoplar el emisor de una petición a su receptor dando a más de un objeto la posibilidad de manejarla : se encadenan los receptores y la petición viaja por la cadena hasta que alguno la maneja.
Problema	Varios objetos podrían atender una petición y no se sabe de antemano cuál corresponde (la ayuda contextual: ¿la explica el botón, el diálogo o la aplicación?).
Solución	Un Handler abstracto declara la operación y mantiene una referencia a su sucesor ; cada ConcreteHandler decide si atiende o reenvía. El cliente le entrega la petición al primero de la cadena.
Cuándo	Cuando (a) más de un objeto puede manejar una petición y el handler no se conoce a priori — debe determinarse automáticamente; (b) querés emitir una petición a uno de varios objetos sin especificar el receptor explícitamente ; (c) el conjunto de objetos que puede manejarla debe especificarse dinámicamente .
A favor	1. Reduce el acoplamiento : libera al objeto de saber quién maneja la petición — solo sabe que "será manejada apropiadamente"; ni emisor ni receptor se conocen, y un objeto de la cadena no necesita conocer la estructura de la cadena. En vez de mantener referencias a todos los receptores candidatos, cada uno guarda una sola : su sucesor. 2. Flexibilidad para asignar responsabilidades : se pueden agregar o cambiar responsabilidades modificando la cadena en runtime , y combinarlo con subclasificación para especializar handlers estáticamente.
En contra	3. La recepción no está garantizada : como la petición no tiene receptor explícito, no hay garantía de que alguien la atienda — puede caerse por el final de la cadena sin ser manejada, o quedar sin atender si la cadena está mal configurada.
Varía	El objeto que puede satisfacer una petición .
Relacionados	Suele aplicarse sobre un Composite (la cadena es la jerarquía de padres). Las peticiones se representan con Command . Sus handlers casi siempre usan Template Method para decidir si atienden y a quién reenvían. Comparado con Mediator , Observer y Command como forma de desacoplar emisor y receptor: es la mejor opción "si la cadena ya es parte de la estructura del sistema". Como Mediator, puede requerir un despacho propio con las mismas desventajas de <i>type safety</i> .

58.2 Command — objeto · comportamiento

Intención	Encapsular una petición como un objeto , permitiendo parametrizar clientes con distintas peticiones, encolarlas o registrarlas, y soportar operaciones deshacibles .
Problema	Un menú o un botón tiene que ejecutar "algo" sin saber qué, y ese algo debe poder deshacerse, repetirse, encolarse o registrarse en un log.
Solución	Un Command abstracto con <code>Execute()</code> ; cada ConcreteCommand guarda a su Receiver y los parámetros necesarios. El Invoker (el ítem de menú) solo dispara <code>Execute()</code> .

Cuándo	Cuando querés (a) parametrizar objetos por una acción a realizar — "los commands son el reemplazo orientado a objetos de los <i>callbacks</i> "; (b) especificar, encolar y ejecutar peticiones en momentos distintos: un command tiene vida independiente de la petición original y, si el receptor se puede representar de forma independiente del espacio de direcciones, se puede transferir a otro proceso y cumplirlo allá; (c) soportar undo : <code>Execute</code> guarda el estado para revertir sus efectos y se agrega <code>Unexecute</code> ; los commands ejecutados se guardan en una lista de historia y recorrerla hacia atrás y adelante da undo y redo ilimitados ; (d) soportar logging de cambios para reaplicarlos ante una caída: agregando <i>load</i> y <i>store</i> a la interfaz se mantiene un log persistente y la recuperación consiste en recargar y reejecutar.
A favor	1. Desacopla al objeto que invoca la operación del que sabe cómo realizarla. 2. Los commands son objetos de primera clase : se manipulan y extienden como cualquier otro. 3. Se pueden ensamblar en commands compuestos (un <code>MacroCommand</code> — que es una instancia de <code>Composite</code>). 4. Es fácil agregar commands nuevos porque no hay que cambiar las clases existentes.
En contra	Nominalmente requiere una subclase por cada conexión emisor-receptor (aunque el propio patrón describe técnicas para evitar la subclasificación), y el undo obliga a decidir qué estado guardar y cuánto — de ahí que se combine con Memento.
Varía	Cuándo y cómo se satisface una petición.
Relacionados	Composite para los macro-commands, Memento para guardar el estado que hace falta revertir, y Prototype si hay que copiar un command antes de ponerlo en la historia. Es la base conceptual de las colas de trabajo y del logging para recuperación — el equivalente de diseño de las tácticas de <i>rollback</i> y <i>transactions</i> del capítulo 17. En Lexi implementa las operaciones de usuario deshacibles . Resuelve las causas de rediseño 2 y 6.

58.3 Interpreter — *clase · comportamiento*

Intención	Dado un lenguaje, definir una representación de su gramática junto con un intérprete que la usa para interpretar sentencias.
Problema	Un tipo de problema aparece tan seguido que conviene expresar sus instancias como sentencias de un lenguaje simple (expresiones regulares, fórmulas, reglas de negocio) e interpretarlas.
Solución	Una jerarquía de clases donde cada regla de la gramática es una clase : <code>TerminalExpression</code> y <code>NonterminalExpression</code> , todas con <code>Interpret(Context)</code> . Una sentencia es un árbol sintáctico abstracto de esas instancias, y se interpreta recorriéndolo.
Cuándo	Cuando hay un lenguaje que interpretar y las sentencias se pueden representar como árboles sintácticos . Funciona mejor cuando (a) la gramática es simple — "para gramáticas complejas la jerarquía de clases se vuelve grande e inmanejable, y ahí son mejores los generadores de parsers"; (b) la eficiencia no es crítica — los intérpretes más eficientes no interpretan el árbol directamente sino que primero traducen a otra forma (las expresiones regulares se transforman en máquinas de estado) aunque "incluso entonces el traductor se puede implementar con Interpreter".
A favor	1. Es fácil cambiar y extender la gramática : como usa clases para representar reglas, se usa herencia para cambiarla o extenderla; las expresiones existentes se modifican incrementalmente y las nuevas se definen como variaciones. 2. Implementar la gramática también es fácil : las clases del árbol tienen implementaciones similares, son sencillas de escribir y "a menudo su generación se puede automatizar con un compilador o un generador de parsers".
En contra	3. Las gramáticas complejas son difíciles de mantener : define al menos una clase por regla (y las reglas en BNF pueden requerir varias), así que una gramática con muchas reglas se vuelve inmanejable.
Varía	La gramática y la interpretación de un lenguaje.

Relacionados	El árbol es un Composite ; se recorre con Iterator ; las operaciones sobre los nodos pueden ir en un Visitor ; los símbolos terminales repetidos se comparten con Flyweight ; y puede usar State para definir contextos de parseo. Es el patrón hermano — a escala de diseño — del estilo intérprete/máquina virtual del capítulo 53. Nota: refactoring.guru no lo incluye en su catálogo principal.
---------------------	---

58.4 Iterator — objeto · comportamiento

Intención	Proveer una forma de acceder secuencialmente a los elementos de un agregado sin exponer su representación subyacente.
Problema	Querés recorrer una colección sin que el cliente sepa si por debajo hay un arreglo, una lista o un árbol — y a veces querés recorrerla de varias formas o varias veces a la vez .
Solución	Un Iterator con la interfaz de recorrido (<code>First</code> , <code>Next</code> , <code>IsDone</code> , <code>CurrentItem</code>) que mantiene su propio estado de recorrido ; el Aggregate ofrece una operación para crearlo.
Cuándo	Para (a) acceder al contenido de un agregado sin exponer su representación interna ; (b) soportar múltiples recorridos del mismo agregado; (c) proveer una interfaz uniforme para recorrer estructuras agregadas distintas — es decir, para soportar iteración polimórfica .
A favor	1. Soporta variaciones en el recorrido : los agregados complejos se recorren de muchas maneras (un árbol sintáctico se puede recorrer <i>inorder</i> o <i>preorder</i>) y cambiar el algoritmo es reemplazar la instancia del iterador ; se pueden definir subclases para recorridos nuevos. 2. Simplifica la interfaz del agregado : al vivir el recorrido en el iterador, el agregado no necesita una interfaz de recorrido. 3. Puede haber más de un recorrido pendiente a la vez , porque cada iterador lleva su propio estado.
En contra	Un objeto extra por recorrido, y la clásica fragilidad de los iteradores robustos : qué pasa si el agregado se modifica mientras se lo recorre.
Varía	Cómo se accede y se recorren los elementos de un agregado.
Relacionados	Se usa muchísimo sobre Composite para recorrer estructuras recursivas. Los iteradores polimórficos se crean con Factory Method . Puede usar Memento para capturar el estado de una iteración. Y Visitor es su complemento: el iterador recorre, el visitor aplica la operación a cada elemento. En Lexi da el acceso y recorrido de la estructura del documento.

58.5 Mediator — objeto · comportamiento

Intención	Definir un objeto que encapsula cómo interactúa un conjunto de objetos : promueve el acoplamiento débil evitando que se referencien explícitamente, y permite variar su interacción de forma independiente.
Problema	Un conjunto de objetos se comunica de formas complejas y las interdependencias resultantes son desestructuradas y difíciles de entender (los controles de un diálogo: cambiar la lista habilita el botón, que valida el campo...).
Solución	Un Mediator concentra el protocolo: cada Colleague le avisa de sus cambios y él decide a quién notificar. Las relaciones muchos-a-muchos se vuelven uno-a-muchos con el mediador en el centro.
Cuándo	Cuando (a) un conjunto de objetos se comunica de maneras bien definidas pero complejas , con interdependencias desestructuradas y difíciles de entender; (b) reutilizar un objeto es difícil porque referencia y se comunica con muchos otros; (c) una conducta distribuida entre varias clases debe poder personalizarse sin subclasificar mucho .

A favor	1. Limita la subclasificación: localiza en un solo lugar una conducta que de otro modo estaría distribuida; cambiarla exige subclasificar solo el Mediator , y los colegas se reutilizan tal cual. 2. Desacopla a los colegas: se pueden variar y reutilizar colegas y mediador independientemente. 3. Simplifica los protocolos: reemplaza interacciones muchos-a-muchos por uno-a-muchos , que son "más fáciles de entender, mantener y extender". 4. Abstrae cómo cooperan los objetos: convertir la mediación en un concepto propio permite razonar sobre cómo interactúan los objetos aparte de su conducta individual.
En contra	5. Centraliza el control: "cambia complejidad de interacción por complejidad en el mediador". Como encapsula protocolos, puede volverse más complejo que cualquier colega individual y convertirse en "un monolito difícil de mantener". Si necesita despacho ad hoc, se pierde <i>type safety</i> .
Varía	Cómo y cuáles objetos interactúan entre sí.
Relacionados	Mediator y Observer son patrones que compiten: Observer distribuye la comunicación (más reutilizable, más difícil de seguir); Mediator la centraliza (más legible, menos reutilizable) — "es más fácil hacer Observers y Subjects reutilizables que hacer Mediators reutilizables". A menudo el mediador se comunica con sus colegas usando Observer . Se distingue de Facade : la fachada simplifica el acceso <i>desde afuera</i> ; el mediador coordina <i>entre</i> colegas. En el SAIP es el patrón mediator de integrabilidad, el único de los tres (wrapper/bridge/mediator) que decide la traducción en runtime .

58.6 Memento — *objeto · comportamiento*

Intención	Sin violar el encapsulamiento , capturar y externalizar el estado interno de un objeto para poder restaurarlo más tarde.
Problema	Necesitas poder volver atrás (undo, checkpoints, transacciones) pero exponer el estado interno del objeto para guardarlo rompería su encapsulamiento .
Solución	El Originator crea un Memento con una copia de su estado y sabe restaurarse desde él; el Caretaker lo custodia sin mirar adentro . El memento define dos interfaces : una <i>restringida</i> para el caretaker (solo guardarlo y pasarlo) y una <i>privilegiada</i> para el originator (leer y escribir el estado).
Cuándo	Cuando debe guardarse una instantánea de (parte de) el estado de un objeto para restaurarlo después, y una interfaz directa para obtener ese estado expondría detalles de implementación y rompería el encapsulamiento .
A favor	1. Preserva los límites del encapsulamiento: evita exponer información que solo el originator debería administrar pero que debe guardarse afuera; escuda a los demás objetos de las internas complejas del originator. 2. Simplifica al Originator: en otros diseños que preservan encapsulamiento, el originator debe guardar las versiones que los clientes pidieron y carga con toda la gestión del almacenamiento; que los clientes administren el estado que pidieron lo simplifica y evita que tengan que avisarle cuando terminaron.
En contra	3. Puede ser caro: si el originator debe copiar mucha información, o si los clientes crean y devuelven mementos muy seguido, el overhead es considerable — "a menos que encapsular y restaurar el estado sea barato, el patrón puede no ser apropiado". Es exactamente el tradeoff que el SAIP señala en el undo : cortar y pegar secciones enormes y deshacerlo todo enlentece notablemente al procesador de texto.
Varía	Qué información privada se guarda fuera del objeto, y cuándo .
Relacionados	Command lo usa para guardar el estado necesario para deshacer. Iterator puede usarlo para capturar el estado de un recorrido. Junto con Command es uno de los patrones de " token mágico " que se pasa y se invoca después; a diferencia de Command, la interfaz del memento es tan estrecha que se pasa como un valor y no ofrece operaciones polimórficas. En el SAIP es el patrón que implementa la táctica undo de usabilidad (cap. 26).

58.7 Observer — objeto · comportamiento

Intención	Definir una dependencia uno-a-muchos entre objetos de modo que, cuando uno cambia de estado, todos sus dependientes sean notificados y actualizados automáticamente .
Problema	Varios objetos deben reflejar el estado de otro (la planilla, el histograma y el gráfico de torta que muestran los mismos datos) y no querés que el que cambia conozca a los que dependen de él.
Solución	El Subject mantiene una lista de Observers que se registran, y al cambiar los notifica ; cada observador consulta el estado que le interesa y se actualiza. Participantes: Subject · Observer · ConcreteSubject · ConcreteObserver.
Cuándo	Cuando (a) una abstracción tiene dos aspectos, uno dependiente del otro , y encapsularlos en objetos separados permite variarlos y reutilizarlos por separado; (b) un cambio en un objeto exige cambiar otros y no se sabe cuántos ; (c) un objeto debe poder notificar a otros sin hacer suposiciones sobre quiénes son — es decir, sin acoplarse a ellos.
A favor	Permite variar sujetos y observadores independientemente : se reutilizan unos sin los otros y se agregan observadores sin tocar al sujeto ni a los demás. 1. Acoplamiento abstracto : el sujeto solo sabe que tiene una lista de observadores conformes a una interfaz simple; no conoce sus clases concretas. Al no estar fuertemente acoplados, pueden pertenecer a capas de abstracción distintas — un sujeto de bajo nivel puede informar a un observador de alto nivel manteniendo intacta la estratificación (si estuvieran fusionados, el objeto resultante violaría las capas o quedaría forzado a una sola). 2. Soporta comunicación por difusión : la notificación no especifica receptor, se emite automáticamente a todos los interesados; el sujeto no le importa cuántos hay, y se pueden agregar y quitar en cualquier momento — es el observador quien decide atender o ignorar.
En contra	3. Actualizaciones inesperadas : como los observadores no saben unos de otros, pueden ser ciegos al costo real de cambiar el sujeto — "una operación aparentemente inocua sobre el sujeto puede provocar una cascada de actualizaciones " en observadores y sus dependientes. Y si los criterios de dependencia no están bien definidos, aparecen actualizaciones espurias difíciles de rastrear . Se suman los problemas que anota el SAIP: si un observador no se des-registra , su memoria nunca se libera (fuga) y se lo sigue invocando; y el <i>impedance mismatch</i> desperdicia procesamiento (el sensor reporta centésimas de grado y la vista solo muestra grados).
Varía	Cuántos objetos dependen de otro y cómo se mantienen al día .
Relacionados	Es la mitad del MVC original de Smalltalk (junto con Composite y Strategy). Compite con Mediator (distribuir vs. centralizar la comunicación) y a menudo lo implementa . Se puede encapsular la semántica de actualización compleja en un <i>ChangeManager</i> , que suele ser un Singleton . Es el equivalente de diseño del estilo de invocación implícita (cap. 51): en un proceso se usa Observer; distribuido, publish-subscribe . Resuelve las causas de rediseño 6 y 7.

58.8 State — objeto · comportamiento

Intención	Permitir que un objeto altere su conducta cuando cambia su estado interno : el objeto parecerá cambiar de clase.
Problema	La conducta depende del estado y el código está lleno de switch o cadenas de if sobre una variable de estado, repetidos en varias operaciones.
Solución	Un State abstracto por cada estado posible; el Context delega en el objeto estado actual y cambia de estado reemplazando ese objeto. Cada rama del condicional se vuelve una clase .

Cuándo	Cuando (a) la conducta de un objeto depende de su estado y debe cambiar en runtime según él; (b) las operaciones tienen condicionales grandes y de varias partes que dependen del estado (representado por constantes enumeradas), y "a menudo varias operaciones contienen la misma estructura condicional". El patrón pone cada rama en su clase, lo que permite tratar el estado como un objeto en sí mismo que varía independientemente.
A favor	1. Localiza la conducta específica de cada estado y particiona la conducta entre estados: todo lo de un estado vive en un objeto, y agregar estados o transiciones es definir subclases nuevas. La alternativa — valores de datos y condicionales en el Context — desparrama condicionales calcados por toda la implementación, y agregar un estado obliga a cambiar varias operaciones. 2. Hace explícitas las transiciones: cuando el estado es solo un valor, las transiciones no tienen representación (son asignaciones a variables); con objetos separados se vuelven explícitas. "Encapsular cada transición y acción en una clase eleva el estado de ejecución a la categoría de objeto: impone estructura al código y hace su intención más clara."
En contra	Distribuye la conducta entre varias subclases: aumenta el número de clases y es menos compacto que una sola clase. El propio GoF matiza: "esa distribución es buena si hay muchos estados, que de otro modo requerirían condicionales enormes" — y los condicionales grandes, como los procedimientos largos, son monolíticos y difíciles de modificar.
Varía	Los estados de un objeto.
Relacionados	Los objetos estado suelen ser Flyweight compartidos (si no guardan datos propios) y a menudo Singleton . Se parece estructuralmente a Strategy — ambos delegan en un objeto intercambiable — pero la intención difiere: Strategy varía el algoritmo por decisión del cliente; State varía la conducta según el estado , y quien cambia el objeto delegado suele ser el propio contexto. "Un objeto tiene un solo estado, pero puede tener muchas strategies para distintas peticiones."

58.9 Strategy — objeto · comportamiento

Intención	Definir una familia de algoritmos , encapsular cada uno y hacerlos intercambiables : el algoritmo varía independientemente de los clientes que lo usan.
Problema	Hay varias formas de hacer lo mismo (romper texto en líneas: simple, TeX, con guionado) y elegir las con condicionales dentro de la clase la vuelve rígida e imposible de extender.
Solución	Una interfaz Strategy con la operación; cada ConcreteStrategy es un algoritmo; el Context tiene una referencia a la strategy y le delega. Se cambia el algoritmo cambiando el objeto.
Cuándo	Cuando (a) muchas clases relacionadas difieren solo en su conducta — las strategies permiten configurar una clase con una de muchas conductas; (b) necesitas variantes de un algoritmo (por ejemplo con distintos tradeoffs de espacio y tiempo); (c) el algoritmo usa datos que el cliente no debería conocer — evita exponer estructuras de datos complejas y específicas; (d) una clase define muchas conductas que aparecen como múltiples condicionales en sus operaciones: en vez de tantos condicionales, cada rama se mueve a su propia clase Strategy.
A favor	1. Familias de algoritmos reutilizables , con la herencia factorizando lo común. 2. Alternativa a la subclasificación: subclasificar el Context "clava la conducta en el Context, mezcla la implementación del algoritmo con la del contexto — lo que lo hace más difícil de entender, mantener y extender — y no permite variar el algoritmo dinámicamente"; encapsularlo aparte permite cambiarlo, entenderlo y extenderlo. 3. Elimina los condicionales: el <code>switch (_breakingStrategy)</code> con un case por algoritmo se convierte en una sola línea, <code>_compositor->Compose()</code> — "código con muchos condicionales suele indicar la necesidad de aplicar Strategy".
En contra	Los clientes deben conocer las strategies para elegir; hay overhead de comunicación entre Context y Strategy (a veces la strategy recibe parámetros que no usa); y aumenta el número de objetos . Además — la crítica moderna de refactoring.guru — en lenguajes con funciones anónimas el patrón se puede reemplazar por una lambda.

Varía	Un algoritmo .
Relacionados	Las strategies suelen ser buenos Flyweight . Se distingue de State (ver arriba), de Template Method (composición vs. herencia para variar pasos) y de Decorator ("cambiar las entrañas vs. cambiar la piel"). En MVC es la relación Vista–Controlador ; en Lexi, los algoritmos de formateo ; en el SAIP, un patrón de testabilidad (sustituir versiones de prueba) y la forma de realizar la táctica de <i>diferir la ligadura</i> por polimorfismo. Resuelve las causas de rediseño 5 y 7.

58.10 Template Method — *clase · comportamiento*

Intención	Definir el esqueleto de un algoritmo en una operación, difiriendo algunos pasos a las subclases : estas redefinen ciertos pasos sin cambiar la estructura del algoritmo.
Problema	Varias subclases repiten el mismo algoritmo con pequeñas diferencias: hay código duplicado y el orden de los pasos se puede romper por descuido.
Solución	La AbstractClass implementa el <i>template method</i> con la secuencia invariante y declara las operaciones primitivas (abstractas) que cada ConcreteClass implementa. La subclase provee los pasos; nunca el orden.
Cuándo	Para (a) implementar una sola vez las partes invariantes de un algoritmo y dejar a las subclases la conducta que varía; (b) factorizar y localizar en una clase común la conducta compartida entre subclases, para evitar duplicación — "un buen ejemplo de <i>refactorizar para generalizar</i> ": primero se identifican las diferencias en el código existente, se separan en operaciones nuevas y se reemplaza el código que difiere por un template method que las llama; (c) controlar las extensiones de las subclases: se define un template method que llama a operaciones <i>hook</i> en puntos específicos, permitiendo extensiones solo en esos puntos .
A favor	"Los template methods son una técnica fundamental de reutilización de código , particularmente importante en las bibliotecas de clases, porque son el medio para factorizar conducta común". Producen una estructura de control invertida , a veces llamada " el principio de Hollywood ": " no nos llames, nosotros te llamamos " — la clase padre llama a las operaciones de la subclase y no al revés. Distingue con precisión dos tipos de operaciones: las abstractas (deben sobrescribirse) y los hooks (pueden sobrescribirse; por defecto suelen no hacer nada) — "es importante especificar cuáles son cuáles: para reutilizar bien una clase abstracta, quien la subclasifique debe entender qué operaciones están diseñadas para ser sobrescritas".
En contra	Usa herencia , con todo lo que eso implica: el binding es estático (no se cambia el algoritmo en runtime) y la subclase queda expuesta a la implementación del padre — el "la herencia rompe el encapsulamiento" del capítulo 42.
Varía	Los pasos de un algoritmo.
Relacionados	Suele llamar Factory Methods . Se compara con Strategy : Template Method varía pasos por herencia; Strategy varía el algoritmo completo por composición (y por eso, en runtime). Los handlers de Chain of Responsibility suelen incluir al menos un template method. Es la inversión de control que, según el GoF, define lo que es un framework : "reutilizás el cuerpo principal y escribís el código que él llama".

58.11 Visitor — *objeto · comportamiento*

Intención	Representar una operación a realizar sobre los elementos de una estructura de objetos , permitiendo definir operaciones nuevas sin cambiar las clases de los elementos visitados.
Problema	Sobre una estructura de objetos (el árbol de un documento, un AST) hay que hacer operaciones muy distintas y no relacionadas — chequear ortografía, guionar, imprimir, calcular estadísticas — y meterlas todas en las clases de la estructura las contamina y obliga a recompilar todo por cada operación nueva.
Solución	Cada operación se vuelve un Visitor con un método VisitXxx por cada tipo concreto de elemento. Los elementos exponen Accept(visitor) y llaman al método que les corresponde — el doble despacho que caracteriza al patrón.

Cuándo	Cuando (a) la estructura contiene muchas clases con interfaces distintas y querés operar sobre ellas según su clase concreta; (b) hay muchas operaciones distintas y no relacionadas y querés evitar "contaminar" las clases con ellas — "Visitor permite mantener juntas las operaciones relacionadas definiéndolas en una clase", y si la estructura se comparte entre varias aplicaciones, poner las operaciones solo en las que las necesitan; (c) las clases de la estructura cambian poco pero se agregan operaciones seguido. Advertencia explícita: "si las clases de la estructura cambian a menudo, probablemente sea mejor definir las operaciones en esas clases".
A favor	1. Agregar operaciones nuevas es fácil: una operación nueva sobre toda la estructura es un visitor nuevo; si en cambio la funcionalidad estuviera desparramada, habría que cambiar cada clase. 2. Junta lo relacionado y separa lo no relacionado: la conducta afín se localiza en un visitor y los conjuntos no relacionados quedan en visitors distintos, lo que simplifica tanto las clases de los elementos como los algoritmos; las estructuras de datos propias de un algoritmo se esconden en el visitor. 4. Puede visitar a través de jerarquías de clases no relacionadas, algo que un iterador no puede.
En contra	3. Agregar clases concretas de elemento es difícil: cada ConcreteElement nuevo obliga a agregar una operación abstracta en Visitor y su implementación en cada ConcreteVisitor. De ahí la consideración clave para aplicarlo: ¿qué va a cambiar más — el algoritmo sobre la estructura, o las clases de la estructura? Si el conjunto de elementos crece seguido, el visitor se vuelve difícil de mantener y conviene poner las operaciones en las clases. Además puede requerir romper el encapsulamiento de los elementos para hacer su trabajo.
Varía	Las operaciones que se pueden aplicar a objetos sin cambiar sus clases.
Relacionados	Casi siempre se aplica sobre un Composite, y se combina con Iterator (uno recorre, el otro opera) y con Interpreter (para las operaciones sobre el árbol sintáctico). Es el ejemplo canónico de "objeto como argumento": el visitor es el argumento de una operación polimórfica Accept y nunca se considera parte de los objetos que visita. En Lexi resuelve el análisis (ortografía y guionado) sin complicar la estructura del documento. Resuelve las causas de rediseño 5 y 8.

Cierre — la tabla que resume el criterio de elección

Si tuvieras que quedarte con una sola herramienta de este catálogo, quedate con la pregunta de la Tabla 1.2 del GoF: "**¿qué quiero que pueda cambiar sin rediseñar?**". Familias de productos → **Abstract Factory**. Cómo se arma un compuesto → **Builder**. La subclase que se instancia → **Factory Method**. La clase que se instancia → **Prototype**. La instancia única → **Singleton**. La interfaz de un objeto → **Adapter**. Su implementación → **Bridge**. La estructura y composición → **Composite**. Las responsabilidades sin subclasificar → **Decorator**. La interfaz a un subsistema → **Facade**. El costo de almacenamiento → **Flyweight**. Cómo y dónde se accede → **Proxy**. Quién satisface la petición → **Chain of Responsibility**. Cuándo y cómo se satisface → **Command**. La gramática de un lenguaje → **Interpreter**. Cómo se recorre un agregado → **Iterator**. Quiénes y cómo interactúan → **Mediator**. Qué estado privado se guarda afuera → **Memento**. Cuántos dependen y cómo se actualizan → **Observer**. Los estados → **State**. El algoritmo → **Strategy**. Los pasos del algoritmo → **Template Method**. Las operaciones aplicables sin tocar las clases → **Visitor**.